

Shop360°: An Augmented Reality Platform for Immersive E-Commerce



Session: BSCS Spring 2022 to 2026

Project Advisor: Ms. Nimra Shafique.

Phase 1 - Project Advisor: Dr. Toqir Ahmad Rana.

Submitted By

Muhammad Azeem 70131514

Faris Salman 70132090

Department of Computer Science & IT
The University of Lahore
Lahore, Pakistan

Declaration

We have read the project guidelines and we understand the meaning of academic dishonesty, in particular plagiarism and collusion. We hereby declare that the work we submitted for our final year project, entitled **Shop360°: An Augmented Reality Platform for Immersive E-Commerce** is original work and has not been printed, published or submitted before as final year project, research work, publication or any other documentation.

Group Member 1 Name: **Muhammad Azeem**
SAP No: **70131514**

Signature:

Group Member 2 Name: **Faris Salman**
SAP No: **70132090**

Signature:

Statement of Submission

This is to certify that **Muhammad Azeem** Roll No. **70131514** and **Faris Salman** Roll No. **70132090** have successfully submitted the Phase 2 of the final year project named as: **Shop360°: An Augmented Reality Platform for Immersive E-Commerce**, at Computer Science & IT Department, The University of Lahore, Lahore Pakistan, to fulfill the partial requirement of the degree of **BS in Computer Science**.

Supervisor Name: Miss Nimra Shafique

Signature:

Date:

Dedication

This project is dedicated to my father, who taught me that the best kind of knowledge to have is which is learned for its own sake. It is also dedicated to my mother, who taught me that even the largest task can be accomplished if it is done one step at a time. This project is also dedicated to our teachers who taught us well, created our interest in such fields and made us capable enough to create something of our own. This project is also dedicated to us as for our motivation and dedication to the field of work.

Acknowledgement

We truly acknowledge the cooperation and help made by name of **Miss Nimra Shafique** of **The University of Lahore**. He has been a constant source of guidance throughout the course of this project. We are also thankful to our friends and families whose silent support led us to complete our project.

Date: _____

Abstract

Shop360° is a mobile shopping app that provides Augmented Reality (AR) to make online shopping better. The application allows customers to view and play with 3D product models in their real time environment with the phone's camera which allows them to "*see*" how something would look in their world before buying it. For example, customers can place furniture in their rooms with the help of Markerless AR.

The application is built with React Native for cross-platform use, and Expo Go is used in the design and testing phase. Augmented Reality features are implemented with ViroReact which allows Markerless AR experiences on Android devices that support ARCore. Product pictures and models are kept in Firebase Storage whereas user data, products, reviews, chats and wishlists of the users are kept with Firestore.

The app also includes Firebase Authentication to handle safe user login, sign-up and role-based access with Custom Claims. This allows normal users and admins to use the same app but with different access and features. Admins can add or modify products as well as handle user reviews and manage the app. Vendors can upload or edit the products whereas Users can browse products, add to a wishlist, write reviews, and preview them in AR.

Secure admin / vendor activities like product management and payment processing are accomplished through a Firebase backend that communicates with the app through secure APIs. Simple chat functionality for user-admin communication is also present within the app.

Shop360° aims to reduce the need for massive physical display rooms by helping businesses showcase their goods online. It is a cost-effective and modern way of helping businesses especially in Pakistan and connect with a wider audience while delivering an experiential and immersive shopping experience to its users. This app's tagline says it all: ***"See It. Try It. Buy It."***

Area of the Project

Augmented Reality (AR) – AR is used to display 3D models of products in the real world through the mobile camera.

Mobile Application Development – The app is built using React Native and runs on Android devices.

Cloud Computing – Firebase is used to store and manage data, including user information, product details, images, and 3D models in the cloud.

Web Services and APIs – The app communicates with a backend built in Firebase using secure APIs for admin functions.

E-Commerce – The app supports product browsing, wishlisting, and payment integration.

Technologies used

React Native, ExpoGo (Test), TypeScript, JavaScript, Firebase Authentication, Cloud Firestore, Firebase Storage, ViroReact, ARCore, ARKit, React Navigation, Google Sign-In, AsyncStorage, React Native Geolocation, React Native Document Picker, React Native Vector Icons, Jest, ESLint, Babel, Metro Bundler, Node.js, Gradle, CocoaPods, Kotlin, Java, React Native Safe Area Context, React Native Screens

List of Figures

Figure 1 Product Perspective Architectural Diagram	9
Figure 2 Use Case Diagram → User Registration	22
Figure 3 Use Case Diagram → User Login / Logout	23
Figure 4 Use Case Diagram → Password Reset	24
Figure 5 Use Case Diagram → Product Listing and Browsing	25
Figure 6 Use Case Diagram → Product Search and Filter	26
Figure 7 Use Case Diagram → View Product Details	27
Figure 8 Use Case Diagram → Wishlist Management	28
Figure 9 Use Case Diagram → Customer Real-Time Chat/Review	29
Figure 10 Use Case Diagram → Vendor Real-Time Chat/Review	30
Figure 11 Use Case Diagram → Admin Real-Time Chat/Review	31
Figure 12 Use Case Diagram → Admin / Vendor Product Management	32
Figure 13 Use Case Diagram → Push Notification	33
Figure 14 Use Case Diagram → Vendor / Admin Dashboard	35
Figure 15 Architectural Diagram	36
Figure 16 ERD	37
Figure 17 Data Flow Diagram Level 0	38
Figure 18 Data Flow Diagram Level 1	39
Figure 19 Class Diagram	40
Figure 20 Activity Diagram → User Registration	41
Figure 21 Activity Diagram → User Login	42
Figure 22 Activity Diagram → View Products	42
Figure 23 Activity Diagram → AR view of the product	43
Figure 24 Activity Diagram → View Products with AR	44
Figure 25 Activity Diagram → Search Product	45
Figure 26 Activity Diagram → Filter Product	46
Figure 27 Activity Diagram → Password Reset	47
Figure 28 Activity Diagram → Upload Review	48
Figure 29 Activity Diagram → Reply / Chat (Review)	49
Figure 30 Activity Diagram → Wishlist View and Management	50
Figure 31 Activity Diagram → User Profile View / Update	50
Figure 32 Activity Diagram → User Auth at Checkpoint	51
Figure 33 Activity Diagram → Admin Managing Users	52
Figure 34 Activity Diagram → Admin View User Chats & Reviews	53
Figure 35 Activity Diagram → Admin Product Management	53
Figure 36 Activity Diagram → Vendor Product Management	54

Figure 37 Sequence Diagram → Add to Cart	55
Figure 38 Sequence Diagram → Add to Wishlist.....	55
Figure 39 Sequence Diagram → Browse by Filter	56
Figure 40 Sequence Diagram → Chat with Vendor	56
Figure 41 Sequence Diagram → Checkout Process	57
Figure 42 Sequence Diagram → Product Browsing.....	57
Figure 43 Sequence Diagram → Submit Product Review.....	58
Figure 44 Sequence Diagram → View Cart	58
Figure 45 Sequence Diagram → View Product Details with AR.....	59
Figure 46 Sequence Diagram → View Wishlist.....	59
Figure 47 Sequence Diagram → Access Admin Dashboard	60
Figure 48 Sequence Diagram → Add New Product	60
Figure 49 Sequence Diagram → Ban and Unban User	61
Figure 50 Sequence Diagram → Delete Product.....	61
Figure 51 Sequence Diagram → Edit Existing Product	62
Figure 52 Sequence Diagram → View Product Reviews	63
Figure 53 Sequence Diagram → View Users	63
Figure 54 Sequence Diagram → Push Notifications	64
Figure 55 Sequence Diagram → Real-Time Chat	64
Figure 56 Sequence Diagram → Vendor Product Upload.....	65
Figure 57 Sequence Diagram → Admin Login	65
Figure 58 Sequence Diagram → Forgot Password & Password Reset.....	66
Figure 59 Sequence Diagram → User Login.....	66
Figure 60 Sequence Diagram → User Registration.....	67
Figure 61 Collaboration Diagram	68
Figure 62 State Transition Diagram for complete app.....	69
Figure 63 Component Diagram.....	70
Figure 64 Deployment Diagram	71
Figure 65 Valid Transition – Signed In (unverified) to Signed In (Verified) ST	105
Figure 66 User Authentication State Diagram ST	106
Figure 67 Order Status State Diagram ST	107
Figure 68 Ticket Support State Diagram ST.....	107
Figure 69 Cart State Diagram ST.....	108
Figure 70 AR View State Diagram ST	108

List of Tables

Table 1 Comparison Table.....	3
Table 2 Acronyms.....	6
Table 3 Abbreviations.....	6
Table 4 Functional Requirement → User Registration.....	10
Table 5 Functional Requirement → User Login.....	10
Table 6 Functional Requirement → User Logout.....	11
Table 7 Functional Requirement → Password Reset.....	11
Table 8 Functional Requirement → Product Browsing.....	12
Table 9 Functional Requirement → Search Product	12
Table 10 Functional Requirement → Filter Product.....	13
Table 11 Functional Requirement → View Product Details	13
Table 12 Functional Requirement → AR Product Preview.....	14
Table 13 Functional Requirement → User Authentication	14
Table 14 Functional Requirement → User Profile Management	15
Table 15 Functional Requirement → Wishlist	15
Table 16 Functional Requirement → Product Management	16
Table 17 Functional Requirement → Reviews Upload.	16
Table 18 Functional Requirement → Reviews View	17
Table 19 Functional Requirement → User Chats	17
Table 20 Functional Requirement → Admin View Chats/Reviews	17
Table 21 Use Case Table → User Registration	23
Table 22 Use Case Table → User Login / Logout.....	23
Table 23 Use Case Table → Password Reset	24
Table 24 Use Case Table → Product Listing.....	26
Table 25 Use Case Table → Product Search and Filter.....	26
Table 26 Use Case Table → View Product Details	27
Table 27 Use Case Table → Wishlist Management	28
Table 28 Use Case Table → Customer Real-Time Chats.....	29
Table 29 Use Case Table → Vendor Real-Time Chat or Reviews.....	30
Table 30 Use Case Table → Admin Real-Time Chat Viewing and Management	31
Table 31 Use Case Table → Admin / Vendor Product Management.....	32
Table 32 Use Case Table → Push Notification	34
Table 33 Use Case Table → Vendor / Admin Dashboard.....	35
Table 34 Email / Password Sign Up +ve Test Case Specification.....	72
Table 35 Email / Password Sign Up -ve Test Case Specification	73
Table 36 User Login / Logout +ve Test Case Specification	73
Table 37 User Login / Logout -ve Test Case Specification.....	74
Table 38 Password Reset +ve Test Case Specification.....	74

Table 39 Password Reset -ve Test Case Specification	75
Table 40 User Profile +ve Test Case Specification	75
Table 41 User Profile -ve Test Case Specification	75
Table 42 Role Based Access Control +ve Test Case Specification	76
Table 43 Role Based Access Control -ve Test Case Specification	76
Table 44 Browse Products +ve Test Case Specification	77
Table 45 Product Search and Filter +ve Test Case Specification	77
Table 46 Product Search and Filter -ve Test Case Specification	78
Table 47 Access AR View +ve Test Case Specification	78
Table 48 Access AR View -ve Test Case Specification	78
Table 49 Grant Camera Permission +ve Test Case Specification	79
Table 50 Grant Camera Permission -ve Test Case Specification	79
Table 51 Product Management +ve Test Case Specification	80
Table 52 Product Management -ve Test Case Specification	80
Table 53 Edit Product +ve Test Case Specification	81
Table 54 Delete Product +ve Test Case Specification	81
Table 55 User Reg / Login / Logout / Reset Password EP Table	82
Table 56 Invalid Email Format EP Table	82
Table 57 Invalid Email Format EP Table	83
Table 58 Empty Email EP Table	83
Table 59 Email Already is Use EP Table	83
Table 60 Valid Password EP Table	83
Table 61 Invalid Password EP Table	84
Table 62 Empty Password EP Table	84
Table 63 Weak Password EP Table	84
Table 64 Password Mismatch EP Table	85
Table 65 Valid Name EP Table	85
Table 66 Valid User Role EP Table	85
Table 67 Invalid User Role EP Table	86
Table 68 Valid Product Title EP Table	86
Table 69 Invalid Product Title EP Table	86
Table 70 Camera Permission Granted EP Table	86
Table 71 Camera Permission Denied EP Table	87
Table 72 Camera Permission Not Requested EP Table	87
Table 73 AR Tracking Ready EP Table	87
Table 74 AR Tracking Limited EP Table	87
Table 75 AR Tracking Unavailable EP Table	88
Table 76 AR Tracking Unknown EP Table	88
Table 77 Model Loaded Successfully EP Table	88
Table 78 Model Loading in Progress EP Table	89
Table 79 Model Loading Error EP Table	89

Table 80 Model Not Loaded EP Table	89
Table 81 Plane Detected EP Table.....	89
Table 82 Plane Detecting EP Table	90
Table 83 Plane Not Detected EP Table.....	90
Table 84 Plane Locked EP Table.....	90
Table 85 Valid Model Scale EP Table.....	91
Table 86 Invalid Model Scale EP Table	91
Table 87 Invalid Model Scale EP Table	91
Table 88 Valid Price (Positive) EP Table	91
Table 89 Invalid Price (Zero) EP Table	92
Table 90 Invalid Price (Negative) EP Table	92
Table 91 Invalid Price (Empty) EP Table.....	92
Table 92 User Reg / Login / Password Reset BVA Table	93
Table 93 Product Minimum Boundary - 1(Outside) BVA Table	93
Table 94 Password Minimum Boundary + 1 (Inside) BVA Table	93
Table 95 Password Minimum Empty (Absolute Minimum – Outside) BVA Table	94
Table 96 User Name Minimum Boundary (Inside) BVA Table.....	94
Table 97 User Name Minimum Boundary – 1 (Outside) BVA Table	94
Table 98 User Name Minimum Boundary + 1 (Inside) BVA Table	94
Table 99 User Name Typical Value (Inside) BVA Table.....	95
Table 100 Product Title Minimum Boundary (Inside) BVA Table.....	95
Table 101 Product Title Minimum Boundary – 1 (Outside) BVA Table	95
Table 102 Product Title Minimum Boundary + 1 (Inside) BVA Table.....	96
Table 103 Product Title Typical Value (Inside) BVA Table.....	96
Table 104 Model Scale Minimum Boundary (Inside) BVA Table.....	96
Table 105 Model Scale Minimum Boundary – 0.01 (Outside) BVA Table	96
Table 106 Model Scale Minimum Boundary (Inside) BVA Table.....	97
Table 107 Model Scale Minimum Boundary + 0.1 (Outside) BVA Table.....	97
Table 108 Model Scale Maximum Boundary – 0.1 (Inside) BVA Table	97
Table 109 Model Scale Typical Value (Inside) BVA Table.....	97
Table 110 AR Tracking Quality Ready (Inside) BVA Table	98
Table 111 AR Tracking Quality Limited – Insufficient Features (Inside) BVA Table	98
Table 112 AR Tracking Quality Limited – Excessive Motion (Inside) BVA Table	98
Table 113 AR Tracking Quality Ready Limited – Insufficient Light (Inside) BVA Table.....	99
Table 114 AR Tracking Quality Unavailable (Outside) BVA Table.....	99
Table 115 User Sign Up Decision Table	101
Table 116 User Sign In Decision Table	102
Table 117 Order Cancellation Request Decision Table.....	103
Table 118 Admin Dashboard Access Decision Table	103
Table 119 View AR Access Decision Table.....	104
Table 120 User Registration and Onboarding Use Case Test Table.....	109

Table 121 User Login Use Case Test Table	109
Table 122 Password Reset Use Case Test Table	110
Table 123 Browse and Search Products Use Case Test Table.....	110
Table 124 View Product Details and Add to Cart Use Case Test Table	111
Table 125 Manage Shopping Cart Use Case Test Table	111
Table 126 Checkout Process Use Case Test Table	112
Table 127 AR product Preview Use Case Test Table.....	112
Table 128 AR Model Interaction Use Case Test Table	112
Table 129 Admin Dashboard Access Use Case Test Table.....	113
Table 130 User Login Performance Test Table	128
Table 131 Home Screen Initial Load Performance Test Table.....	129
Table 132 AR View Initialization Performance Test Table	130
Table 133 Concurrent User Load Performance Test Table	130
Table 134 Extreme Network Interruption Stress Test	133
Table 135 Extreme Network Interruption Stress Test Table	134
Table 136 Firestore Transaction Conflict Stress Test Table.....	135
Table 137 Complete User Registration to First Purchase System Test	139
Table 138 AR Product Visualization and Purchase Integration System Test Table.....	142
Table 139 Multi-Product AR Comparison and Selection System Test Table	144

Table of Content

<i>Declaration</i>	i
<i>Statement of Submission</i>	ii
<i>Dedication</i>	iii
<i>Acknowledgement</i>	iv
<i>Abstract</i>	v
List of Figures.....	vii
List of Tables	ix
Chapter 1: Introduction to the Problem	1
1.1 Introduction	1
1.2 Purpose	1
1.3 Objective	2
1.4 Existing Solution	2
1.5 Proposed Solution	3
Chapter 2: Software Requirement Specification.....	4
2.1 Introduction	4
2.1.1 Purpose.....	4
2.1.2 Scope.....	4
2.1.3 Definitions, acronyms, and abbreviations.....	5
Definitions: -	5
Acronyms: -	6
Abbreviations: -	6
2.2 Overall description	7
2.2.1 Product perspective	7
2.2.2 Product functions	10
2.2.3 User characteristics	18
2.2.4 Constraints	18
2.2.5 Assumptions and dependencies	19
2.2.6 Apportioning of requirements.....	19

2.3	Specific requirements.....	20
2.3.1	Functional Requirements	20
2.3.2	Non-functional Requirements	21
Chapter 3:	Use Case Analysis.....	22
Chapter 4:	Design	36
4.1	Architecture Diagram.....	36
4.2	ERD with data dictionary	37
4.3	Data Flow diagram.....	38
4.3.1	The level 0.....	38
4.3.2	The level 1.....	39
4.4	Class Diagram	40
4.5	Activity Diagrams	41
4.6	Sequence Diagram	55
4.7	Collaboration Diagram.....	68
4.8	State Transition Diagram	69
4.9	Component Diagram	70
4.10	Deployment Diagram	71
Chapter 5: Testing.....		72
5.1	Test Case Specification.....	72
5.2	Black Box Test Cases	82
5.2.1	Equivalence Partitions (EP).....	82
5.2.2	Boundary Value Analysis	93
5.2.3	Decision Table Testing	100
5.2.4	State Transition Testing	105
5.2.5	Use Case Testing	109
5.3	White Box Test Cases.....	114
5.3.1	Cyclomatic complexity.....	114
5.4	Performance Testing.....	126
5.5	Stress Testing:.....	131
5.6	System Testing.....	136
5.7	Regression Testing.....	145
5.7.1	Selecting Regression Tests	145
5.7.2	Regression Tests	149

Chapter 6: Tools & Techniques	155
Chapter 7: Summary & Conclusion.....	158
7.1 Summary.....	158
7.2 Conclusion	158
Chapter 8: User Manual.....	160
Chapter 9: Lessons Learnt and Future Work	168
9.1 Lessons Learnt.....	168
9.2 Future Work.....	168
GitHub Repo.....	169
References	169
Appendix	171
Appendix A: User Roles and Access Controls	171
Appendix B: Technologies and Languages	171
Appendix C: AR Implementation Details.....	171
Appendix D: Project Setup	172
Appendix E: Testing and Debugging Tools	173
Appendix F: Deployment Instructions.....	173
Appendix G: AR Usage Instructions for Users	174
Appendix H: Security Practices.....	175
Appendix I: Performance Consideration	176

Chapter 1: Introduction to the Problem

1.1 Introduction

The e-commerce industry has been rapidly growing over the last few years. It has become easier to buy products as the e-shopping experience has made it more convenient for customers to reach a broader market and shop online. But one of the major problems still remains that in online marketing customers are facing an issue to visualize the products in real time [1][2]. This leads to confusion and makes the decision of buying trickier. Especially when buying products like furniture, etc. the major problem is of real-time dimensions of the product and inability to visualize the product in their own available space. This increases customer dissatisfaction and also results in higher return rates. If we look into businesses, companies that deal with furniture, sanitaryware, clothing, etc., spend a lot of money on expensive show-rooms for their products which not only increases the maintenance costs for them but also makes it more difficult to manage, especially for small scaled businesses.

Augmented Reality (AR) has been introduced to the market and has a potential solution to resolve these issues. AR would enable users to visualize virtual objects to the users in their real-time environment. **Shop360°** is an AR based mobile application that will enhance the online shopping experience by allowing its users to interact with the 3D models of products such as furniture [5], sanitaryware or even clothes, before purchasing. This would help them place the model of the product in their real-time environment and decide accordingly. By integrating AR into e-commerce, this project aims to reduce the gap between the physical and digital shopping experience [2].

1.2 Purpose

In e-commerce, customers often struggle with the decision of buying a product online. Uncertainties due to the inability of interacting with the product and visualization in real-time environment leads to increase in higher return rates or dissatisfaction. Maintenance of such expensive showrooms is also one of the major problems faced by companies.

These problems can be solved through this approach of Augmented Reality in e-commerce as users would be able to view and interact with a 3D model of the product in their real-time environment. Additionally, customers will not have to visit a physical store for product satisfaction which will decrease return rates and product dissatisfaction with convenience [6]. Companies or businesses would also not require an expensive physical showroom as their products can be viewed by the users through the mobile application which is an innovative alternate to physical showrooms. This would decrease the operational costs and would be really beneficial in local markets like in Pakistan where e-commerce is growing and businesses seek cost-effective ways to showcase their products.

By bridging this gap between traditional and digital shopping, **Shop360°** improves convenience and the decision-making of buying a while creating a more efficient and enjoyable retail experience.

1.3 Objective

The main objective of this project is to develop an Augmented Reality based mobile application which will enhance the online shopping experience by allowing users to visualize products in their real-time environment before purchasing a product. Following are the major objectives to aim for in this project: -

- Create an **Augmented Reality (AR)** based **Mobile Application** for **e-commerce** dealing with products such as furniture, sanitaryware, etc.
- Implement **product visualization** which will enable users to preview and interact products such as furniture, sanitaryware, etc. in Augmented Reality.
- **Optimize operational expenses** by offering companies of local market an alternate to large physical showrooms which would reduce expenditure or showrooms and improve product showcasing.
- Introduce Augmented Reality in the **local market** of **Pakistan**.

Enhance customer confidence by providing an interactive “*See It. Try It. Buy It*” experience to reduce uncertainty and increase **user satisfaction**.

1.4 Existing Solution

Currently in Pakistan, e-commerce shopping platforms such as Daraz. provide users with product images, descriptions and customer reviews to help them make purchasing decisions. But providing customers with images and descriptions is not enough for the realism of the product which leads to confusion and uncertainty. Some brands like IKEA Place or Amazon AR View offer product previews in AR, but the support is limited to certain categories and not widely available across all items. Plus, all these websites that are working on AR are providing services in e-commerce that are not functional here in Pakistan. Global e-commerce websites like Amazon have AR features but they do not offer an AR-based shopping experience here in Pakistan which limits the local accessibility of AR in the market.

Feature	Existing Solutions	Shop360°
AR Visualization	Limited to specific brands (e.g., IKEA, Snapchat)	Supports multiple product categories
E-Commerce Integration	Separate platforms for AR and shopping	Combines AR preview with purchasing
Market Availability	Mostly available in Western markets	Specifically designed for Pakistan & emerging markets
User Engagement	Static images and videos	Immersive, real-time product interaction

Table 1 Comparison Table

1.5 Proposed Solution

Shop360° is a mobile application that integrates Augmented Reality (AR) for online shopping which provides interactive and immersive product visualization [5]. This project is an innovative mobile application that allows users to preview products in a real-time environment before purchasing by also providing true-to-scale AR visualization for multiple product categories.

Shop360° is unlike the other apps of AR (like IKEA Place which is only limited to furniture) that are specific to their products such as furniture, etc. **Shop360°** supports a variety of categories which makes it a comprehensive AR shopping platform that combines AR shopping with e-commerce features in a single platform. All the features in an e-commerce website such as buying, wishlist, etc., would also be integrated in the project alongside Augmented Reality (AR).

At the present, Amazon and other international platforms have started integrating AR but the local platforms such as Daraz here in Pakistan do not offer such features. This project would fill this gap by providing an AR shopping application to the market having the future scalability to global markets [4][5].

Chapter 2: Software Requirement Specification

2.1 Introduction

2.1.1 Purpose

The objective of this Software Requirements Specification (SRS) document is to present with a detailed description of the functional and non-functional requirements for the implementation of the **Shop360°** mobile application. This document is intended to act as a reference for all stakeholders of the project such as the developers, testers, project managers, and future maintainers.

The main purpose of **Shop360°** is to offer users an Augmented Reality (AR)-based e-commerce website where they can see products (clothing, furniture, accessories, etc.) in the real world through their smartphone cameras prior to purchasing. This SRS will establish how the system will act, how it works with Firebase and how the system gets interacted by different roles of the users (admin or customer) and how the products would be rendered in real-time environment through the app.

The intended audience of this project would be the students developing this project which are the project stakeholders, Quality Assurance and the System Designers with our respected supervisor and the Testers and Project evaluators which would in this case be our respected evaluators assigned our project to evaluate and any personnel who want to check our product for future upgradation or scaling as for business purposes or educational purposes.

2.1.2 Scope

The project boundaries are discussed as follows:

- Users will be able to preview and interact with the 3D model of a product such as furniture, sanitaryware, etc. and see how do they look in real life.
- The app would provide all the basic e-commerce functionalities but with AR (Augmented Reality).
- The application would be built using **React Native** for mobile development and **Google Firestore & Firebase** is going to be used in Authentication, User Profile Managements, Products and their AR models storage, Chats & Reviews and Wishlist. Admin panel will also be dealt with Google Firestore for role-based auth.
- For the project, **ViroReact** would be utilized for AR rendering the models.
- The primary users are online shoppers looking for a more interactive shopping experience.
- Engagement of Businesses and retailers which are aiming to reduce showroom costs or even startups.
- Shop360° targets the Pakistani e-commerce market where online shopping is rapidly growing.

The limitations for the project are also mentioned as follows: -

- A mobile device that supports ARCore (Android) and can render Markerless AR models would be required.
- Only products like furniture, sanitaryware, etc. can be previewed in AR at this stage.
- The application would not support real-time AR size adjustments for custom fitting and the feature can be added in the future.
- Internet connectivity would be required for accessing product models and making purchases.

2.1.3 Definitions, acronyms, and abbreviations

Definitions: -

- **Augmented Reality (AR):** A technology that overlays digital objects (such as 3D models) onto the real world through a mobile device's camera.
- **React Native:** A framework for building cross-platform mobile apps using JavaScript and React.
- **Expo Go:** A tool for developing React Native apps quickly without needing to configure native code, useful during prototyping and testing.

Firebase: A Backend-as-a-Service (BaaS) platform by Google that offers services like authentication, real-time database, cloud storage, and hosting.

Firestore: A NoSQL cloud database provided by Firebase, used for storing app data in a scalable and real-time way.

Firebase Authentication: A service by Firebase that allows users to sign in using email, password, or third-party providers like Google.

Firebase Storage: A cloud storage solution by Firebase used for storing files such as images and 3D models.

Custom Claims: A feature in Firebase Authentication used to add user roles (like admin or customer) for role-based access control.

ViroReact: An open-source AR and VR framework for React Native that enables rendering of 3D models in real-world environments.

3D Model: A digital representation of an object in three dimensions, used in AR to display products virtually.

Markerless AR: A type of AR that does not require a printed marker (like a QR code) and instead uses the environment for placing virtual objects.

Hybrid Architecture: A software structure that combines multiple technologies or systems, like using Firebase for frontend/backend and for secure admin routes.

Wishlist: A feature that allows users to save favorite products for future reference.

Admin Dashboard: A backend interface for the app admin to manage products, users, payments, etc.

Chat Module: A feature that allows customers to communicate with the admin or support through messaging.

Role-Based Access Control (RBAC): A system that restricts app features based on user roles (like admin vs. customer).

Acronyms: -

<i>AR</i>	Augmented Reality
<i>UI</i>	User Interface
<i>UX</i>	User Experience
<i>API</i>	Application Programming Interface
<i>CRUD</i>	Create, Read, Update, Delete
<i>CLI</i>	Command Line Interface
<i>JWT</i>	JSON Web Token
<i>RBAC</i>	Role-Based Access Control
<i>IDE</i>	Integrated Development Environment
<i>SDK</i>	Software Development Kit
<i>FYP</i>	Final Year Project

Table 2 Acronyms

Abbreviations: -

<i>UOL</i>	University of Lahore
<i>DB</i>	Database
<i>Auth</i>	Authentication
<i>Config</i>	Configuration
<i>Dev</i>	Development
<i>Admin</i>	Administrator
<i>Desc</i>	Description
<i>App</i>	Application

Table 3 Abbreviations

2.2 Overall description

2.2.1 Product perspective

Shop360° is a self-contained mobile app built using React Native. It has integration with Google Firebase for authentication of users, real-time databases, and AR model storage as well. It also utilizes a Firebase backend for admin-specific secure functions like product management. The app doesn't use any other software systems and is self-contained entirely. All the interfaces and characteristics are given as follows:

- **System interfaces**
 - Firebase Authentication for managing users.
 - Firestore Database for storing products, reviews, chats, and user data.
 - Firebase Storage / Cloudinary for storing 3D AR model files.
 - Firebase Authentication for secure admin operations.
 - ViroReact library for rendering AR content within the App.
- **User interfaces** (Screenshots of the user interfaces are given in the user manual at chapter 5)
 - **Role-Based Routing** will direct users to either customer or admin interfaces
 - **Screens include:**
 - Login / Signup/Forgot Password?
 - Home Page with product listing.
 - Product Details with AR Preview.
 - Wishlist.
 - Reviews / Chats.
 - Admin Dashboard (Add / Edit / Delete Product, Manage Users, View Reviews).
- **Hardware interfaces**
 - Mobile Devices with Android operating Systems.
 - Camera access required for AR functionality.
- **Software interfaces**
 - Firebase SDK (Auth, Firestore, Storage).
 - ViroReact for AR model rendering.

- **Communications interfaces**
 - HTTPS for secure API communication.
 - Firebase WebSocket for real-time data syncing.
- **Memory**
 - The app must run efficiently on mobile devices with at least 4 GB RAM.
 - Firebase Storage quota managed dynamically.
- **Operations**
 - Standard mobile app installation.
 - No special operations are required.
 - Backups are managed by Firebase.
- **Site adaptation requirements.**
 - No special site adaptations are required for the app.
 - All data is cloud-hosted, allowing usage in any region.

Architectural Diagram for the complete perspective of the project is on the next page.

Architectural Diagram:

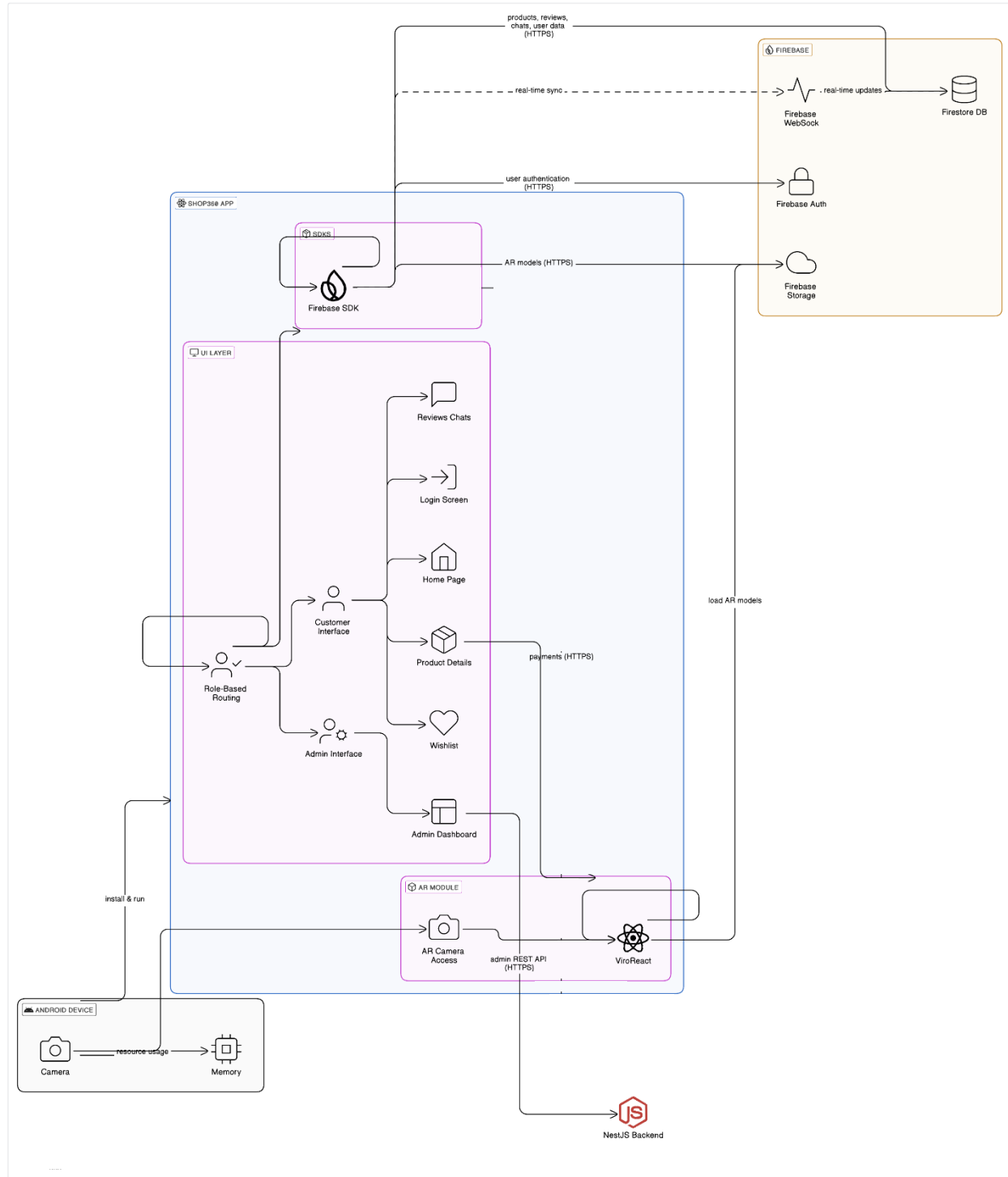


Figure 1 Product Perspective Architectural Diagram

2.2.2 Product functions

All the functional requirements for our project are defined below:

- **Module:** User Registration.

ID:	UR_01 (Customer) / UR_02 (Vendor)			
Name:	User Registration (Customer / Vendors)			
Description	Input	Output	Requirements	Basic Work Flow
Users should be able to register with email and password.	User inputs the credentials for account creation.	After verification and validation, a new user is formed.	Must enter valid credentials and verified through mail.	Credential Input by the user → Automatic mail verification for user → Verification → New user registered

Table 4 Functional Requirement → User Registration

- **Module:** User Login.

ID:	UL_01 (Admin) / UL_02 (Customer) / UL_03 (Vendor)			
Name:	User Login (Customer / Vendors)			
Description	Input	Output	Requirements	Basic Work Flow
Users should be able to login with verified email and password.	User inputs the credentials for account login.	User logs the app.	Must enter valid credentials and must be a verified user.	Credential Input by the user → Check if the user is validated or not → Login (Verified) / Account Verification Notification.

Table 5 Functional Requirement → User Login

- **Module:** User Logout.

ID:	ULo_01 (Admin) / ULo_02 (Customer) / ULo_03 (Vendor)			
Name:	User Logout.			
Description	Input	Output	Requirements	Basic Work Flow
Users should be able to logout the app.	User presses on the Logout button.	User is logged out and the session is now closed for the user.	User must be verified user and logged in.	User presses Logout → Session ended → User logged out.

Table 6 Functional Requirement → User Logout

- **Module:** Password Reset.

ID:	PR_01 (Admin) / PR_02 (Customer) / PR_03 (Vendor)			
Name:	Password Reset.			
Description	Input	Output	Requirements	Basic Work Flow
Users should be able to reset the password.	User clicks on “Forgot Password” / “Reset Password”	A mail verification is sent with the link to reset the password.	User must be a verified user.	User presses “Forgot Password” / “Reset Password” → Verification link with reset functionality sent to the user → User Password updated.

Table 7 Functional Requirement → Password Reset

- **Module:** Product Browsing (Guest and Verified User)

ID:	PB_01 (Admin) / PB_02 (Customer) / PB_03 (Vendor) / PB_04 (Guests)			
Name:	Product Browsing and Viewing			
Description	Input	Output	Requirements	Basic Work Flow
Allows all users (guests and verified) to browse the product catalog and view detailed product information. No login required for browsing.	User taps product listing or search query.	List of products or product details displayed	Internet Connectivity required and Firestore read access for all users.	User opens app → Product list fetched from Firestore → User selects a product → Product details are displayed including AR Preview button.

Table 8 Functional Requirement → Product Browsing

- **Module:** Search Product (Guest and Verified User)

ID:	SP_01 (Admin) / SP_02 (Customer) / SP_03 (Vendor) / SP_04 (Guests)			
Name:	Search Product (Guest and Verified User)			
Description	Input	Output	Requirements	Basic Work Flow
Allows all users (guests and verified) to search for the product. No login required for browsing.	User generates a search query.	List of products or product details displayed	Internet Connectivity required and Firestore read access for all users.	User opens app → Product list fetched from Firestore User search for a product → Product details are displayed including AR Preview button.

Table 9 Functional Requirement → Search Product

- **Module:** Filter Product

ID:	FP_01 (Admin) / FP_02 (Customer) / FP_03 (Vendor) / FP_04 (Guests)			
Name:	Filter Products.			
Description	Input	Output	Requirements	Basic Work Flow
The user should be able to filter the products by various criteria (price, etc.)	The user uses the filter functionality and enters the criteria to filter the products.	The product list is filtered out according to the criteria and displayed on the UI.	Internet Connectivity required and Firestore read access for all users.	Filter functionality is used → Criteria is entered → Products according to the criteria are displayed on the UI.

Table 10 Functional Requirement → Filter Product

- **Module:** View Product Details

ID:	VP_01 (Admin) / VP_02 (Customer) / VP_03 (Vendor) / VP_04 (Guests)			
Name:	View Product Details			
Description	Input	Output	Requirements	Basic Work Flow
The user should be able to view the product details in a separate component page for product description.	User clicks on a product.	Product Specific details page is opened.	Internet Connectivity required and Firestore read access for all users.	User clicks on a product → System fetches the data for the specific product → Product Specific details page is opened and displayed.

Table 11 Functional Requirement → View Product Details

- **Module:** AR Product Preview (Guest and Verified User)

ID:	AR_01			
Name:	AR Product Preview (Guest and Verified User)			
Description	Input	Output	Requirements	Basic Work Flow
Enables users to preview products in Augmented Reality using ViroReact. Available to all users without requiring login.	User selects AR preview on product details page.	User selects AR preview on product details page.	Internet Connectivity required and Firestore read access for all users as well as Camera Access.	User taps AR Preview button → AR model loaded from Firebase Storage → Model rendered using ViroReact

Table 12 Functional Requirement → AR Product Preview

- **Module:** User Authentication (At Checkout).

ID:	Auth_01			
Name:	User Authentication (At Checkout)			
Description	Input	Output	Requirements	Basic Work Flow
Makes guest users to register or login before making a purchase. Verification step is enforced only at checkout.	All the user details such as Name, Email, Password, Address, etc., is verified through Firebase Auth mail verification.	Verified user session token.	Firebase Authentication integration.	1. Guest proceeds to checkout → Prompted to login or register. 2. User provides credentials → Firebase Auth verifies. 3. Upon success, user completes purchase.

Table 13 Functional Requirement → User Authentication

- **Module:** User Profile View and Management

ID:	UP_01			
Name:	User Profile View and Management.			
Description	Input	Output	Requirements	Basic Work Flow
Allows viewing and editing user information if verified.	Profile data.	Updated user data.	None as a guest can also use the app except the checkout functionality	User updates fields → Firestore document updated.

Table 14 Functional Requirement → User Profile Management

- **Module:** Wishlist View and Management (Verified Users)

ID:	WL_01			
Name:	Wishlist View and Management.			
Description	Input	Output	Requirements	Basic Work Flow
Allows logged-in users to view, add or remove products from their wishlist. Guests cannot save wishlist.	Product ID from UI interaction.	Firestore update to user's wishlist subcollection.	Firestore read/write access restricted to authenticated users	1. User taps wishlist icon on a product. 2. App verifies authentication → updates wishlist in Firestore.

Table 15 Functional Requirement → Wishlist

- **Module:** Product Management (Admin/Vendor)

ID:	PM_01 (Admin) / PM_02 (Vendor)			
Name:	Product CRUD.			
Description	Input	Output	Requirements	Basic Work Flow
Admin/Vendor can create, update and delete products.	Product title, description, AR Model and the price.	New/Updated Firestore document.	Must be a verified admin user.	Admin fills form → API call to Firebase SDK → Firestore update.

Table 16 Functional Requirement → Product Management

- **Module:** Reviews upload.

ID:	RC_01			
Name:	Review upload and view.			
Description	Input	Output	Requirements	Basic Work Flow
Allows real-time product reviews for customers.	Provide with a review of a product	Displayed to user via Firestore sync.	Must be a verified user to give a review.	User sends text → Firestore document upload → Displayed in UI.

Table 17 Functional Requirement → Reviews Upload.

- **Module:** Reviews view.

ID:	RV_01			
Name:	Review viewed by Vendors.			
Description	Input	Output	Requirements	Basic Work Flow
Allows real-time product reviews for vendors.	View a review and reply in chat.	Displayed to user via Firestore sync.	Must be a verified user to give a review.	User sends text → Firestore document upload → Displayed in UI.

Table 18 Functional Requirement → Reviews View

- **Module:** User Chats.

ID:	UC_01 (Customer) / UC_02 (Vendor)			
Name:	User Chats (Customer and Vendor).			
Description	Input	Output	Requirements	Basic Work Flow
Allows real-time chats for customers and vendors.	Customer creates a chat with a vendor / Vendor replies to a message.	Displayed to users via Firestore sync.	Must be a verified user to start a chat.	User sends text → Firestore document upload → Displayed in UI.

Table 19 Functional Requirement → User Chats

- **Module:** Admin View User Chats and Reviews.

ID:	AV_01 (Admin)			
Name:	Admin View User Chats and Reviews.			
Description	Input	Output	Requirements	Basic Work Flow
Allows the admin to keep check and view user reviews and chats.	Admin fetches the review/chat data.	Displayed to Admin.	Must be a verified admin.	Admin requests for a customer/vendor chat → Firestore Storage returns the chats → Displayed in UI.

Table 20 Functional Requirement → Admin View Chats/Reviews

2.2.3 User characteristics

There would be two types of users for the app and their required characteristics are mentioned as follows.

1. General Users (Customers):

These users can have a basic level of education with an experience of familiar shopping apps as minimal technical skills are required to operate the app.

2. Admin Users:

These users must have an Intermediate to advanced level of educational level as they would have to deal with the form to fill out while updating a new product with its relevant AR models and pictures. They would also be required to have familiarity for content management. The technical expertise should be moderate so that they would be able to manage the backend data.

2.2.4 Constraints

The following mentioned would be the constraints for the project and some of them may change after time when implemented accordingly:

- **Regulatory Policies:** The app must comply with data protection laws. Pakistan does not have any stand-alone laws for data protection but the user detail confidentiality must remain in the app.
- **Hardware Limitations:** The app must work on smartphones with at least 4GB RAM and a functional rear camera for AR.
- **Interfaces to Other Applications:** Firebase SDK Console.
- **Audit Functions:** Admin has privileges to manage content.
- **Language Requirements:** JavaScript / TypeScript.
- **Reliability:** 99% uptime for everything.
- **Security:** Firebase security rules.

2.2.5 Assumptions and dependencies

To know about the dependencies and the assumptions to make before implementation is important for a project so that the developers know that their project is dependent on such factors. Those assumptions and dependencies for the project are mentioned as follows:

- Users must have a stable internet access.
- Firebase services must remain operational.
- Firebase SDK backend is deployed securely.
- AR will function using ViroReact and proper permissions are allowed on device like camera access.

2.2.6 Apportioning of requirements

Apportioning is the process of breaking down high level requirements to basic different parts or modules of the system which will help in distributing responsibilities across subsystems and components for a better understanding regarding the functionality of our project. The following is the breakdown of the total tasks:

- **Authentication & Authorization** → Firebase Authentication.
- **AR Functionality** → Firestore / Cloudinary access + ViroReact integration in the React Native App.
- **Product Display & Browsing** → Firestore + React Native UI.
- **Admin Product Management** → Firestore Storage for AR models.
- **Chat & Reviews** → Firestore sync (real-time updates).
- **Wishlist Management** → Firestore subcollections.

For Future Updates:

- **Purchase/Payment Module:** Stripe integration (Test Mode) & PayFast.pk (Production mode)
- **Order History & Tracking:** For insights on user behavior as well as for the users to keep track on their previous orders.
- **Push Notifications & Offers:** Admin triggered promotional messages to users regarding the offers.

2.3 Specific requirements

In this section, all the functional and non-functional requirements have been mentioned as to verify that all the system actually satisfies the functionality of the product. These requirements are mentioned and discussed below:

2.3.1 Functional Requirements

The specific features and the capabilities that our system must possess to function are as follows:

- User Registration.
- User Login / Logout.
- Password Reset.
- User Profile Management.
- Role Based Access Control (Admin / Verified User / Guest).
- Product Browsing.
- Product Search and Filter.
- View Product Details.
- AR Product Preview.
- Wishlist Management (Customer).
- Chat with vendor.
- Product Reviews.
- Product Management (Admin).
- Firebase Integration.
- Push Notifications.
- Authentication and Token Verification.
- Admin Dashboard Access.

These all would be the requirements the product should must have.

2.3.2 Non-functional Requirements

All the non-functional requirements for the product are mentioned as follows:

- **Usability:**
 - Intuitive navigation between app and a UI with a clean design.
 - AR is implemented seamlessly with no issues and bugs using ViroReact.
 - **Reliability:**
 - Firebase ensures 99% availability.
 - Firebase SDK is hosted with health monitoring.
 - **Performance:**
 - AR loading optimized with lazy-loading of 3D Models.
 - Real-time sync using Firestore WebSockets.
 - **Design Constraints:**
 - App developed in React Native CLI (Not Expo).
 - AR limited to supported device of Android.
 - **Portability:**
 - Backend REST APIs usable by future versions and up-scalable.
 - **Maintainability:**
 - Modular Code Structure.
 - Cloud Functions and Firestore rules easy to update.
 - **License Agreement:**
 - Uses open-source libraries: ViroReact, Firebase SDK, React Native.
-

Chapter 3: Use Case Analysis

All the use cases are based on the roles available in the app and as the app have roles with common functionalities, an advanced version of use cases (scenarios) have been provided here according to the roles.

- User Registration Use Case Diagram.

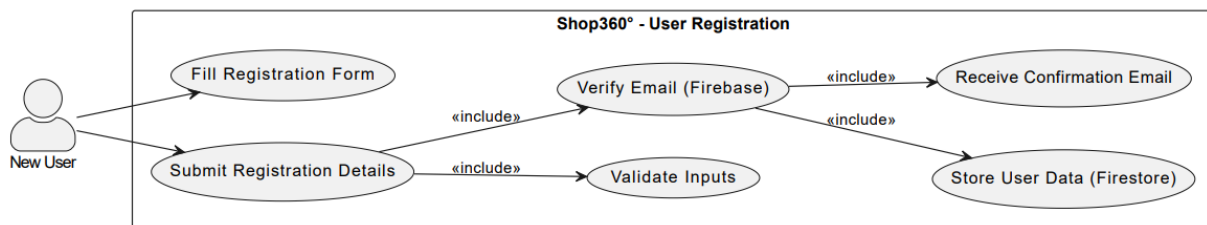


Figure 2 Use Case Diagram → User Registration

- User Registration Use Case diagram detail table.

Use Case ID	UC_01	
Use Case Name	User Registration	
Description	A new user can register on the app for being a verified user.	
Primary Actor	Customer / Vendor	
Secondary Actor	System	
Pre-Condition	Must enter valid details and must be connected to the internet.	
Post-Condition	Gets verification email for verification	
Basic Flow	Actor Action	System Action
	User enters the registration details and submits	Details are verified and stored in Firestore and a confirmation mail is sent.
Alternate Flow	Notification for registration failed is sent.	

Table 21 Use Case Table → User Registration

- User Login / Logout Use Case Diagram.

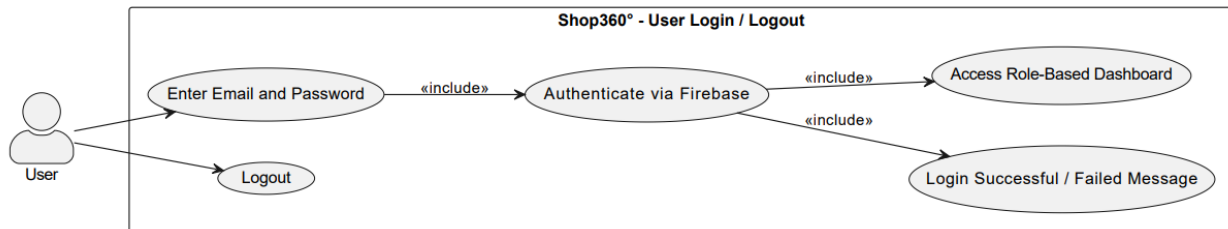


Figure 3 Use Case Diagram → User Login / Logout

- User Login / Logout Use Case diagram detail table.

Use Case ID	UC_02 (Login) / UC_03 (Logout).	
Use Case Name	User Login / Logout Functionality.	
Description	Users are able to login / logout from the app.	
Primary Actor	User (Verified Customer / Vendor)	
Secondary Actor	System.	
Pre-Condition	User must enter credentials for login and must be logged in for logout.	
Post-Condition	User session starts (login) / ends (logout).	
Basic Flow	Actor Action	System Action
	User enters login credentials / User logs out.	System checks for a verified account and logs in and starts a user session / System logs the user out and ends the session.
Alternate Flow	Notification for invalid credentials / Logout unsuccessful.	

Table 22 Use Case Table → User Login / Logout

- Password Reset Use Case (Scenario) Diagram.

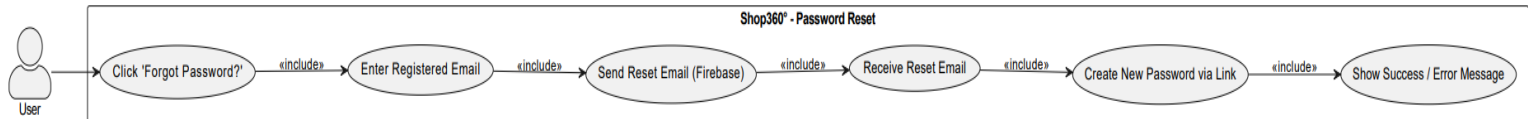


Figure 4 Use Case Diagram → Password Reset

- Password Reset Use Case diagram detail table.

Use Case ID	UC_04	
Use Case Name	Password Reset (Admin / Vendor / Customer).	
Description	The user (verified) can change their password.	
Primary Actor	User (Admin / Vendor / Customer).	
Secondary Actor	System.	
Pre-Condition	The user must be a verified user and respond on email verification for password reset.	
Post-Condition	Firebase storage updates the saved password for the user.	
Basic Flow	Actor Action	System Action
	User resets the password through mail verification.	Password updated notification is sent and updated in the storage.
Alternate Flow	Notification for unable to update password is sent to the user.	

Table 23 Use Case Table → Password Reset

- Product Listing and Browsing Use Case (Scenario) Diagram.

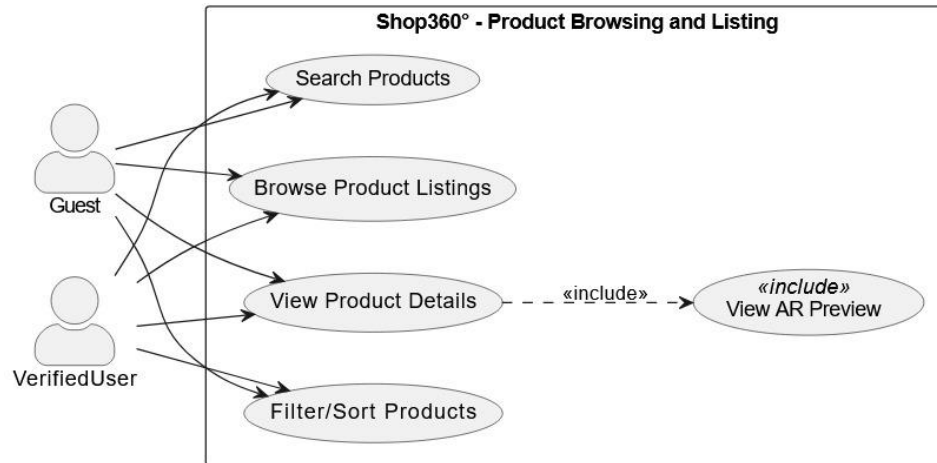


Figure 5 Use Case Diagram → Product Listing and Browsing

- Product Listing and Browsing Use Case (Scenario) diagram detail table.

Use Case ID	UC_05	
Use Case Name	Product Listing and Browsing.	
Description	User (Guest or Verified) can view the product list and browse through the products including the AR view functionality as well.	
Primary Actor	Users (Guest or Verified).	
Secondary Actor	System.	
Pre-Condition	Users are connected to the internet and have allowed permissions such as camera access for AR view.	
Post-Condition	The request is generated for the specific product to be viewed and the relevant data is fetched from the Storage and displayed at the user end.	
Basic Flow	Actor Action	System Action
	The user views the products and browse a product to view its description or even view in AR.	The system fetches the data from the Storage and displays the details for the relevant product.
Alternate Flow	In an unprecedented error in fetching, page not found page would be displayed.	

Table 24 Use Case Table → Product Listing

- Product Search and Filter Use Case Diagram.

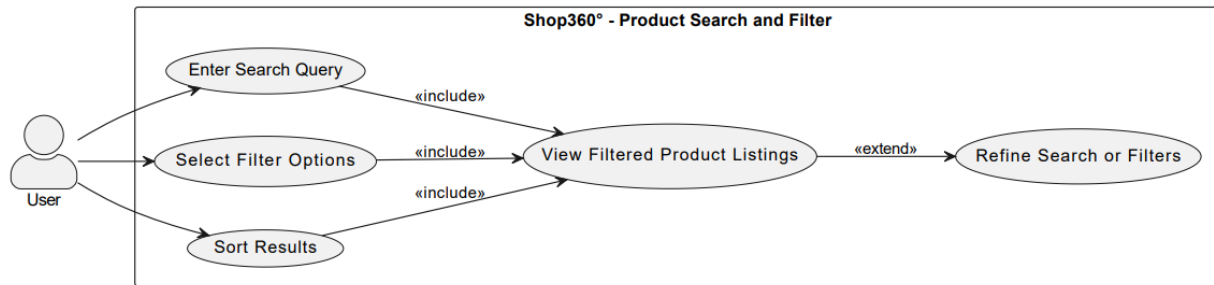


Figure 6 Use Case Diagram → Product Search and Filter

- Product Search and Filter Use Case diagram detail table.

Use Case ID	UC_06	
Use Case Name	Product Search and Filter.	
Description	The uses searches for a specific product or filter through a criterion.	
Primary Actor	User.	
Secondary Actor	System.	
Pre-Condition	The user must be connected to the internet.	
Post-Condition	The app successfully fetches the relevant products.	
Basic Flow	Actor Action	System Action
	The user searches for a product or applies a filter.	The system provides with the relevant data.
Alternate Flow	Notification for unable to search / apply filter is shown.	

Table 25 Use Case Table → Product Search and Filter

- View Product Details Use Case Diagram.

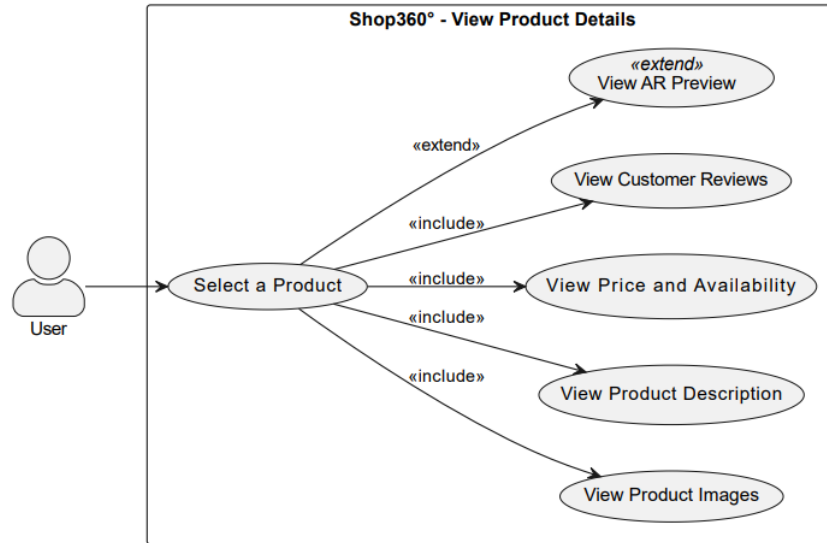


Figure 7 Use Case Diagram → View Product Details

- View Product Details Use Case diagram detail table.

Use Case ID	UC_07	
Use Case Name	View Product Details.	
Description	The user is able to view the details for the product.	
Primary Actor	User.	
Secondary Actor	System.	
Pre-Condition	The user must be connected to the internet.	
Post-Condition	The system fetches the data of the specific product and displays on UI.	
Basic Flow	Actor Action	System Action
	The user selects a product and views the details.	The system fetches the data of the specific product and displays on UI.
Alternate Flow	Page not found / Unable to get the product data Notification is shown.	

Table 26 Use Case Table → View Product Details

- Wishlist Management Use Case Diagram.

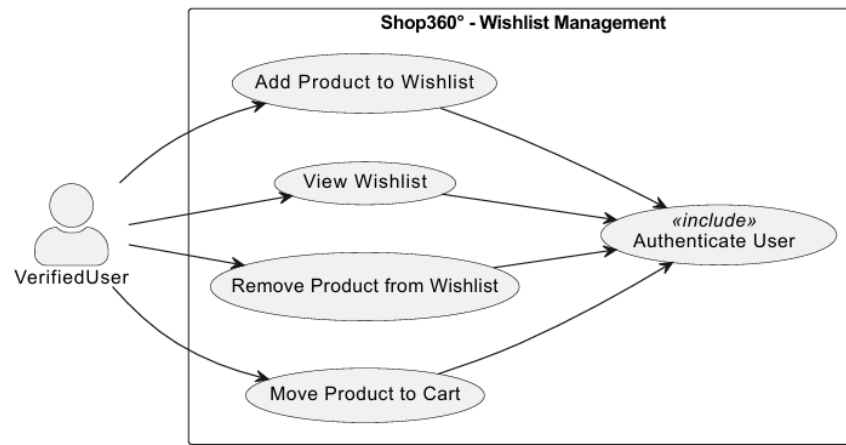


Figure 8 Use Case Diagram → Wishlist Management

- Wishlist Management Use Case diagram detail table.

Use Case ID	UC_08	
Use Case Name	Wishlist Management.	
Description	Verified Customers can manage their own wishlist.	
Primary Actor	Verified Customer.	
Secondary Actor	System.	
Pre-Condition	The user must be verified and clicks the “Add to Wishlist” button in a products description page.	
Post-Condition	The product is added in the wishlist in the Firestore Storage for that specific user.	
Basic Flow	Actor Action	System Action
	The user clicks the “Add to Wishlist” button in a products description page.	The product is added in the wishlist in the Firestore Storage for that specific user.
Alternate Flow	Unable to fetch your wishlist notification.	

Table 27 Use Case Table → Wishlist Management

- Customer real-time chat / reviews Use Case Diagram.

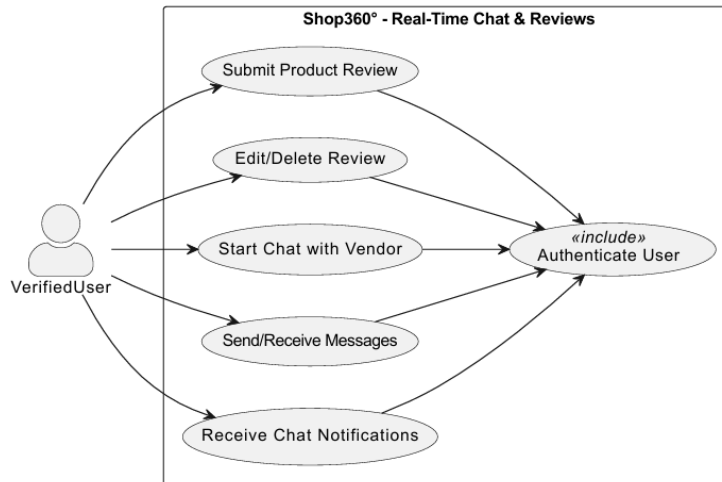


Figure 9 Use Case Diagram → Customer Real-Time Chat/Review

- Customer real-time chat / reviews Use Case diagram detail table.

Use Case ID	UC_09	
Use Case Name	Customer real-time chat / reviews.	
Description	The user can chat or even provide reviews in real-time.	
Primary Actor	Verified User (Customer).	
Secondary Actor	System.	
Pre-Condition	The user must be verified and the user uploads a chat or review.	
Post-Condition	Through Firestore sync (WebSocket), the review or chat message section is updated successfully.	
Basic Flow	Actor Action	System Action
	The user uploads a chat message or review.	Firestore sync updates the review or chat section to the updated one.
Alternate Flow	Unable to fetch chats / Unable to upload your message notification.	

Table 28 Use Case Table → Customer Real-Time Chats

- Vendor real-time chat / reviews Use Case Diagram.

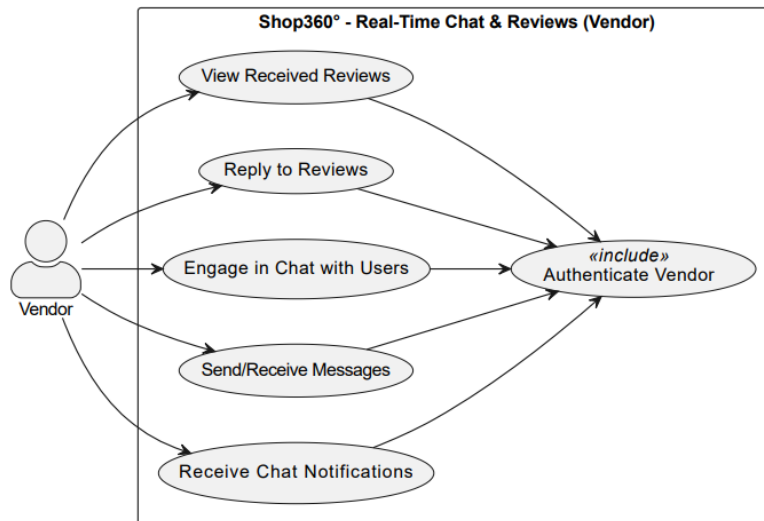


Figure 10 Use Case Diagram → Vendor Real-Time Chat/Review

- Vendor real-time chat / reviews Use Case diagram detail table.

Use Case ID	UC_10	
Use Case Name	Vendor real-time chat / reviews.	
Description	This use case deals with the real-time chats of a customer with the vendor.	
Primary Actor	User (Vendor).	
Secondary Actor	System.	
Pre-Condition	The user must be a verified Vendor.	
Post-Condition	Firestore is working properly and the message is saved in the Storage and updated in the UI in real-time.	
Basic Flow	Actor Action	System Action
	The vendor replies a chat and submits the message.	That message is pushed in the storage for that user and is showed in the UI.
Alternate Flow	Unable to fetch chats / Unable to upload your message notification.	

Table 29 Use Case Table → Vendor Real-Time Chat or Reviews

- Admin real-time chat / reviews Use Case Diagram.

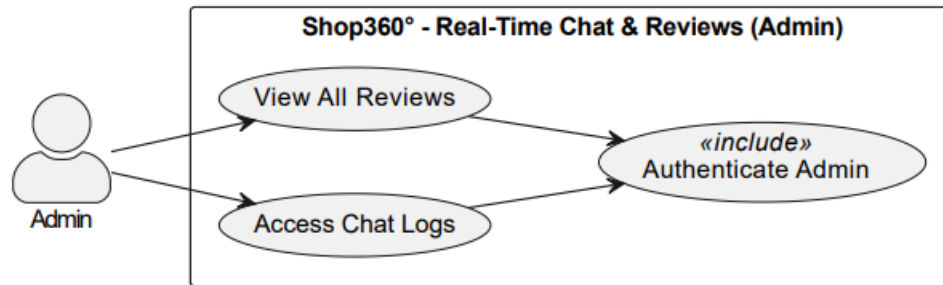


Figure 11 Use Case Diagram → Admin Real-Time Chat/Review

- Admin real-time chat / reviews Use Case diagram detail table.

Use Case ID	UC_11	
Use Case Name	Admin real-time chat / reviews.	
Description	Admin can view all the messages (chats and reviews) uploaded by the customer or the vendor.	
Primary Actor	Admin	
Secondary Actor	System	
Pre-Condition	Admin must be verified.	
Post-Condition	Admin gets real-time access to the messages uploaded by the verified users.	
Basic Flow	Actor Action	System Action
	Admin requests the message pool of a vendor or a customer.	The system in response returns all the chats by the vendor or the customer from Storage.
Alternate Flow	Unable to fetch chats / Unable to upload your message notification.	

Table 30 Use Case Table → Admin Real-Time Chat Viewing and Management

- Admin/Vendor Product Management Use Case (Scenario) Diagram.

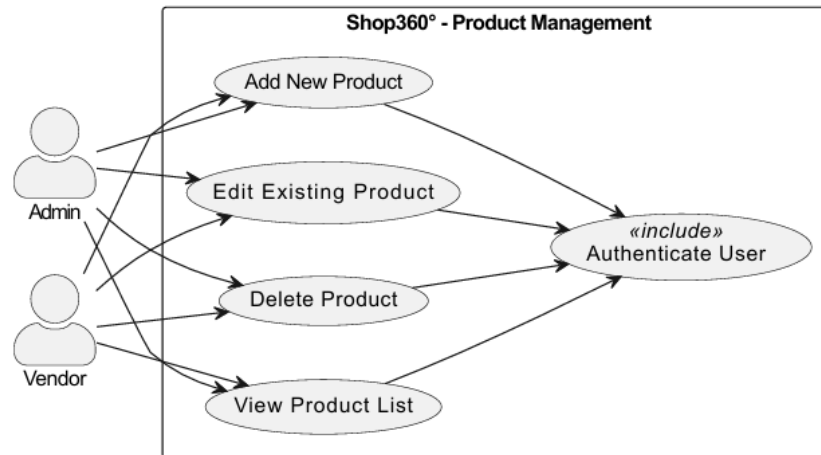


Figure 12 Use Case Diagram → Admin / Vendor Product Management

- Admin/Vendor Product Management Use Case (Scenario) diagram detail table.

Use Case ID	UC_12	
Use Case Name	Admin/Vendor Product Management.	
Description	Admin and Vendor both can manage the CRUD operations for a product.	
Primary Actor	Verified Admin/Vendor.	
Secondary Actor	System.	
Pre-Condition	Must be verified.	
Post-Condition	Admin / Vendor make any changes to it and that change is updated in the storage for a product.	
Basic Flow	Actor Action	System Action
	Admin/Vendor upload or update a product.	The system stores the details of the product in Storage.
Alternate Flow	If the vendor is getting an issue regarding uploading, Admin will be contacted.	

Table 31 Use Case Table → Admin / Vendor Product Management

- Push Notification Use Case (Scenario) Diagram.

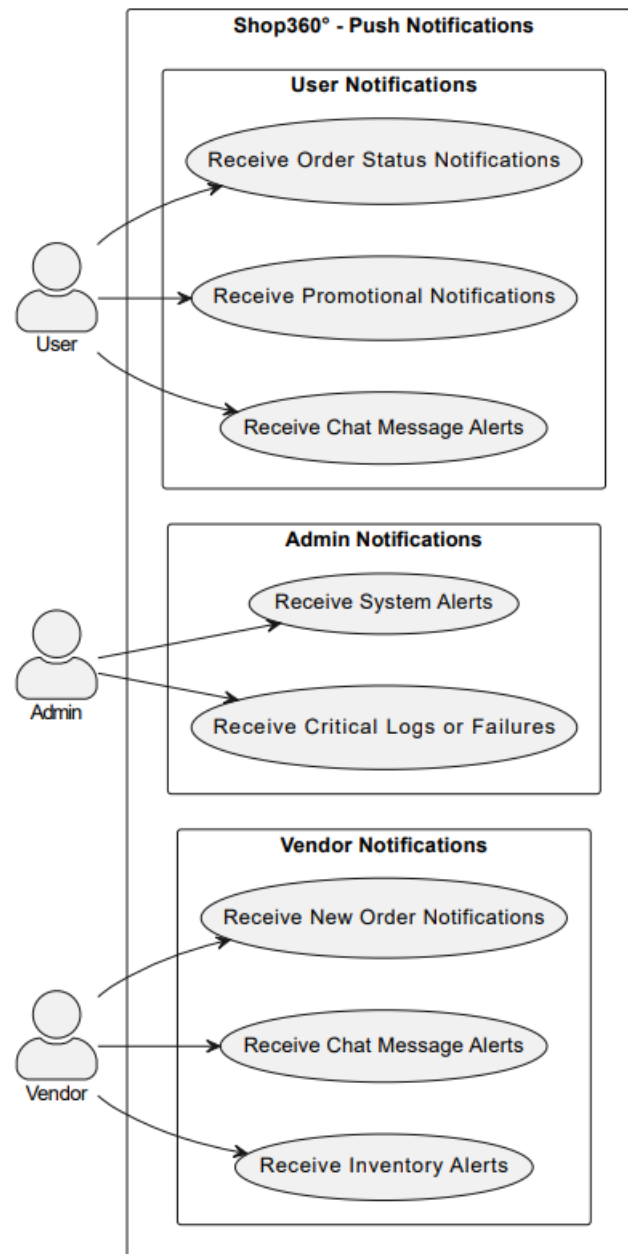


Figure 13 Use Case Diagram → Push Notification

- Push Notification Use Case (Scenario) diagram detail table.

Use Case ID	UC_13 (User) / UC_14 (Admin) / UC_15 (Vendor)	
Use Case Name	Push Notifications.	
Description	Push Notifications for alerts will be generated for the user.	
Primary Actor	Firebase Cloud Messaging (FCM)	
Secondary Actor	System.	
Pre-Condition	Notifications are ON and Receives an alert and Device is registered with Firebase Cloud Management.	
Post-Condition	System pushes a notification for the app.	
Basic Flow	Actor Action	System Action
	A message is generated for the user.	The system generates a notification and an alert for the user.
Alternate Flow	In case of OFF Notification or the device is offline, no alert is generated.	

Table 32 Use Case Table → Push Notification

- Vendor / Admin Dashboard Use Case (Scenario) Diagram.

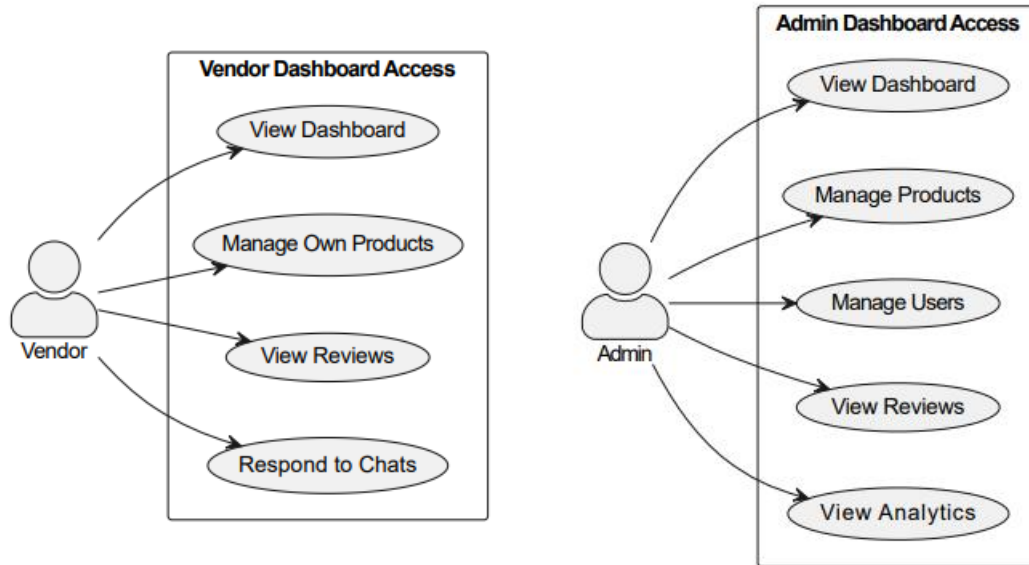


Figure 14 Use Case Diagram → Vendor / Admin Dashboard

- Vendor / Admin Dashboard Use Case (Scenario) diagram detail table.

Use Case ID	UC_16 / UC_17	
Use Case Name	Vendor / Admin Dashboard.	
Description	Separate Dashboards for Vendors and Admin,	
Primary Actor	User (Admin / Vendor)	
Secondary Actor	System.	
Pre-Condition	The user must be verified.	
Post-Condition	The user is routed to their respective dashboards according to their roles.	
Basic Flow	Actor Action	System Action
	User enters credentials while login.	User is routed to their individual dashboard.
Alternate Flow	If user credentials are invalid, a notification is displayed on the UI.	

Table 33 Use Case Table → Vendor / Admin Dashboard

Chapter 4: Design

4.1 Architecture Diagram

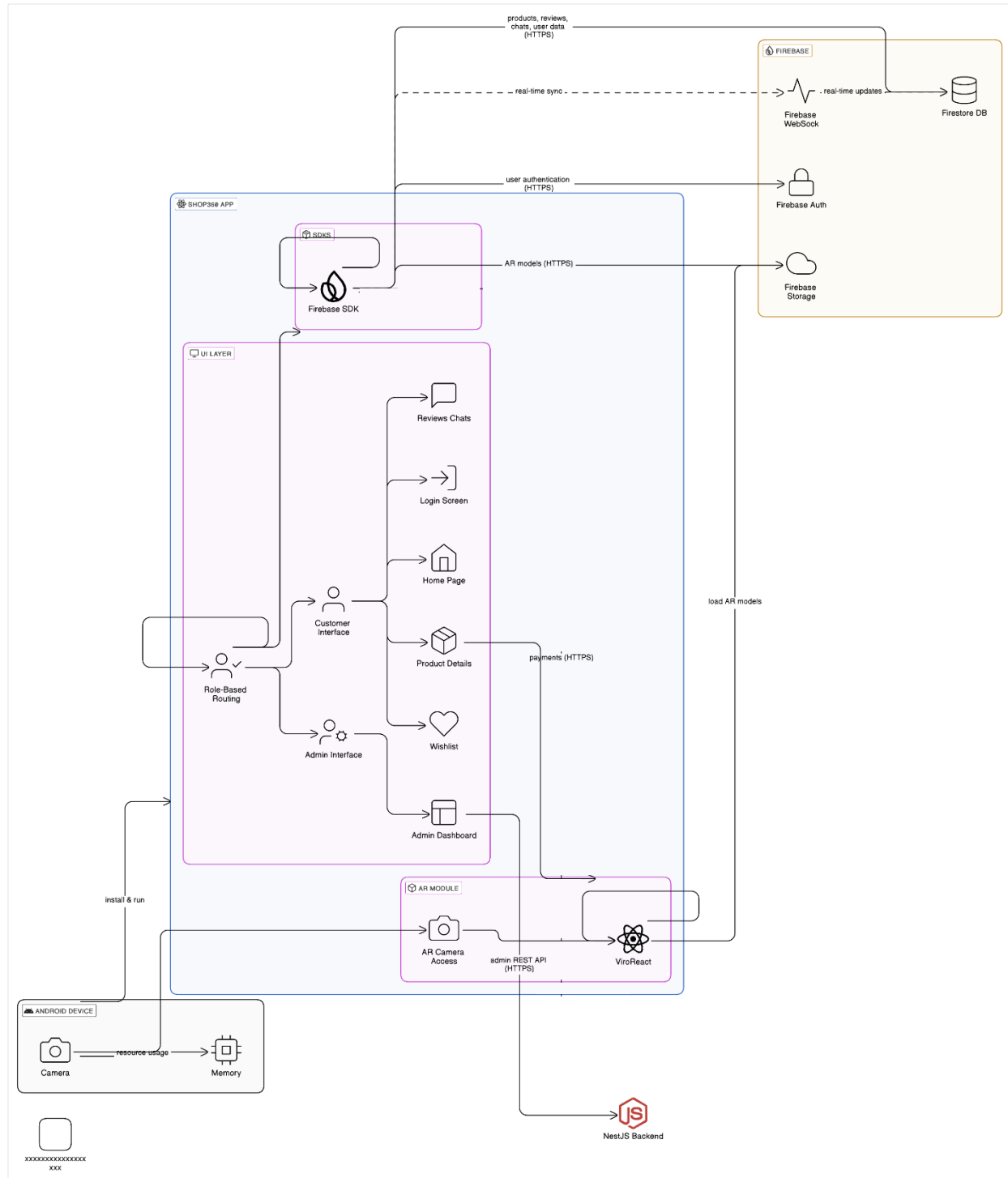


Figure 15 Architectural Diagram

4.2 ERD with data dictionary

There may be changes in data variables in time when implementing the AR functionality in the second part of the FYP.

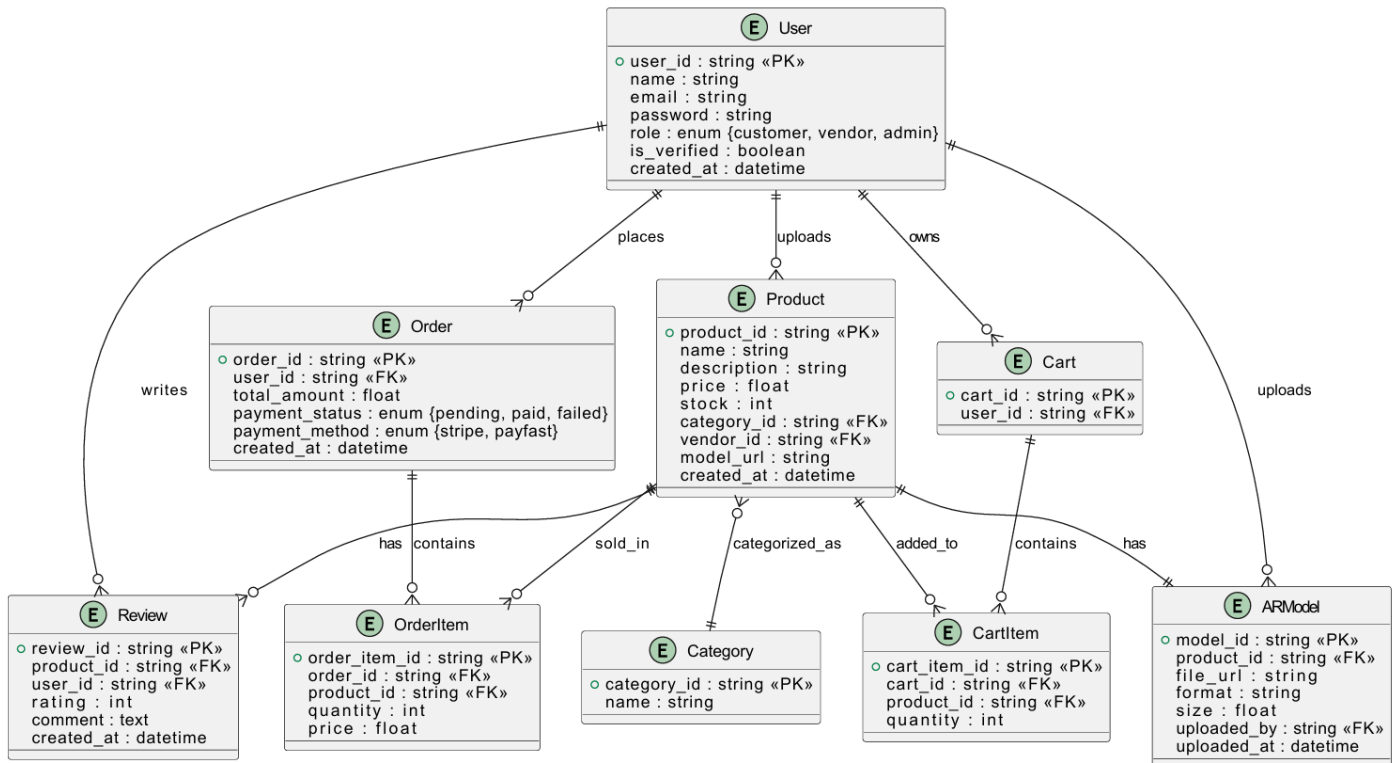


Figure 16 ERD

4.3 Data Flow diagram

Data flow diagram includes two levels and both of them are given below:

4.3.1 The level 0

The flow of information inside the system is defined in this level as shown:

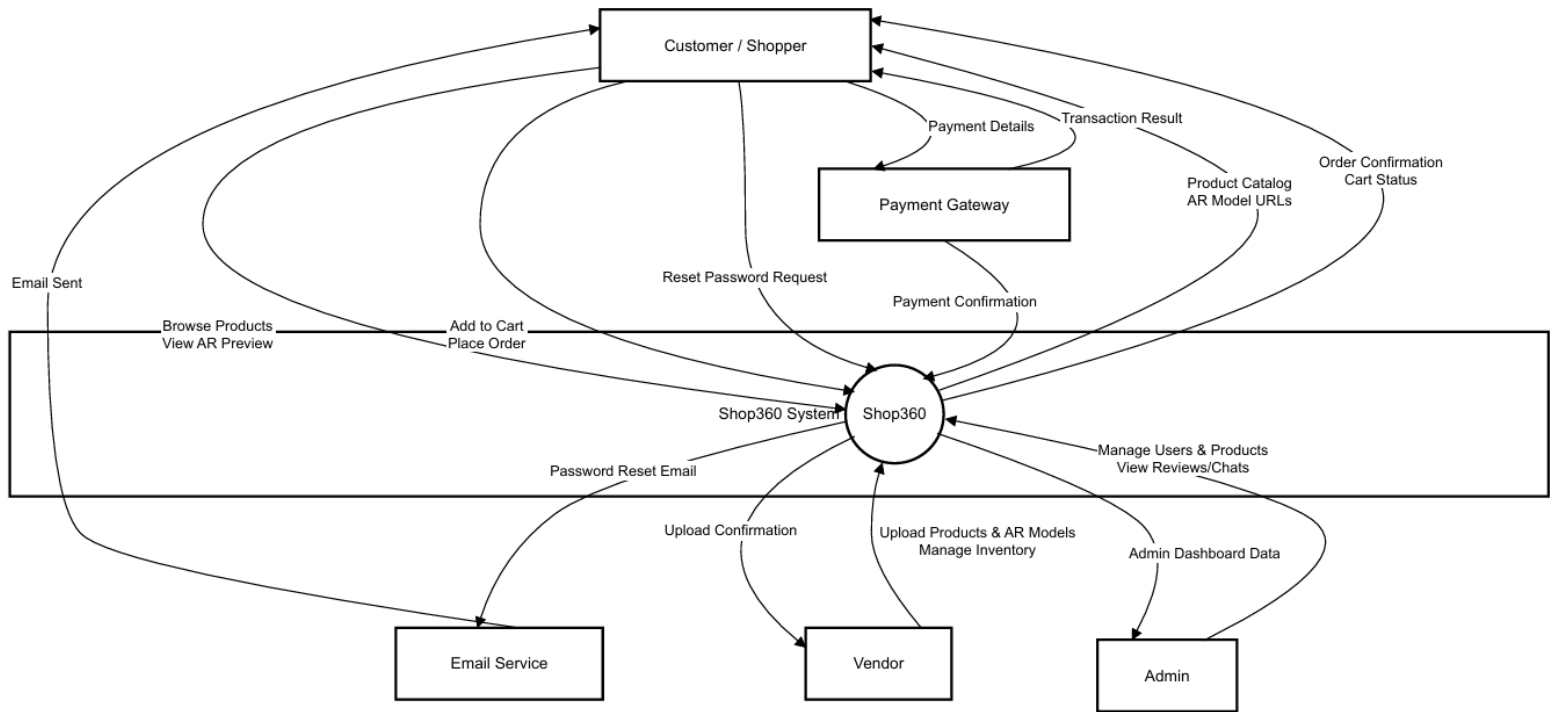


Figure 17 Data Flow Diagram Level 0

4.3.2 The level 1

The flow of information outside the system is defined in this level as shown below:

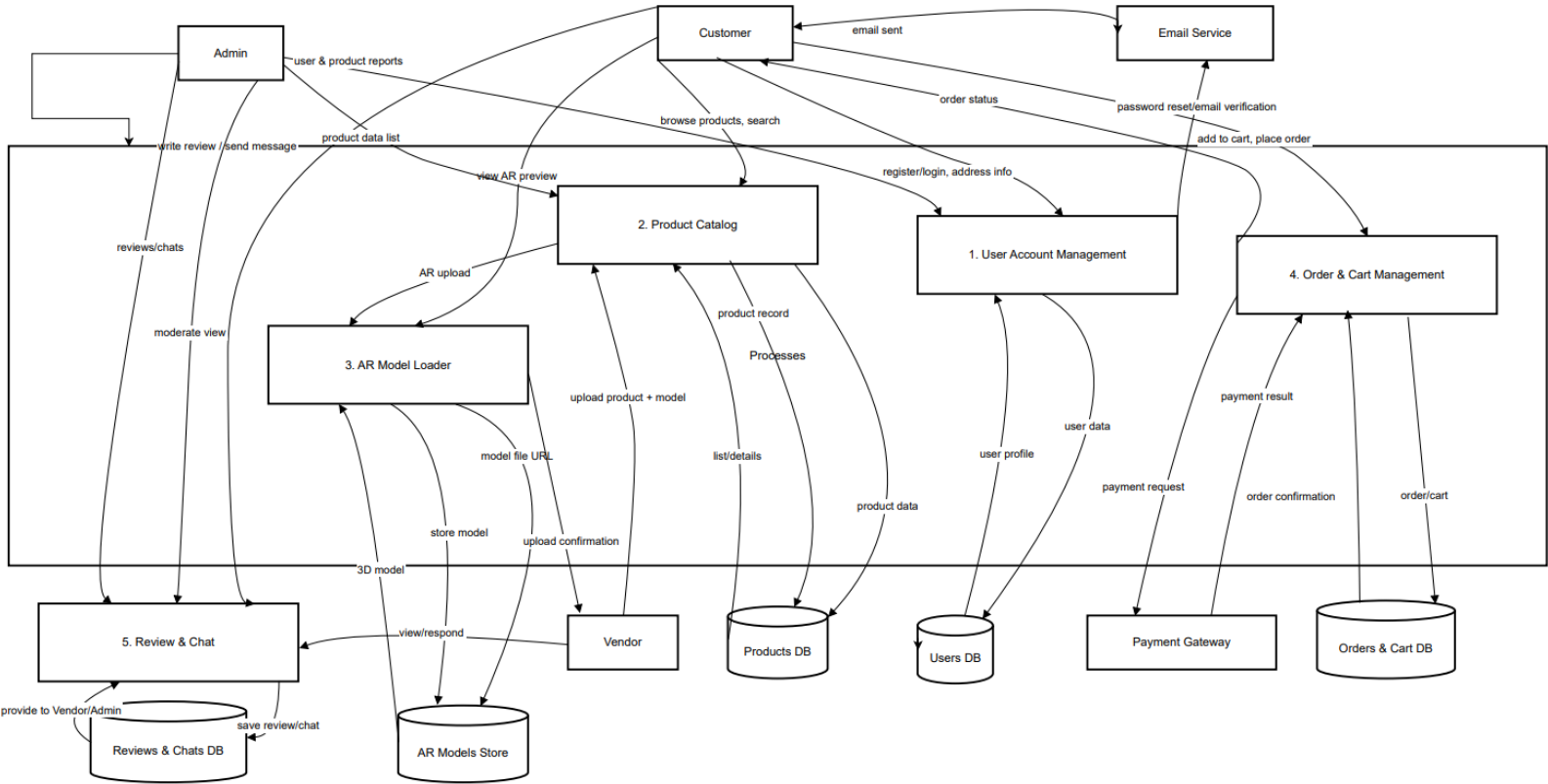


Figure 18 Data Flow Diagram Level 1

4.4 Class Diagram

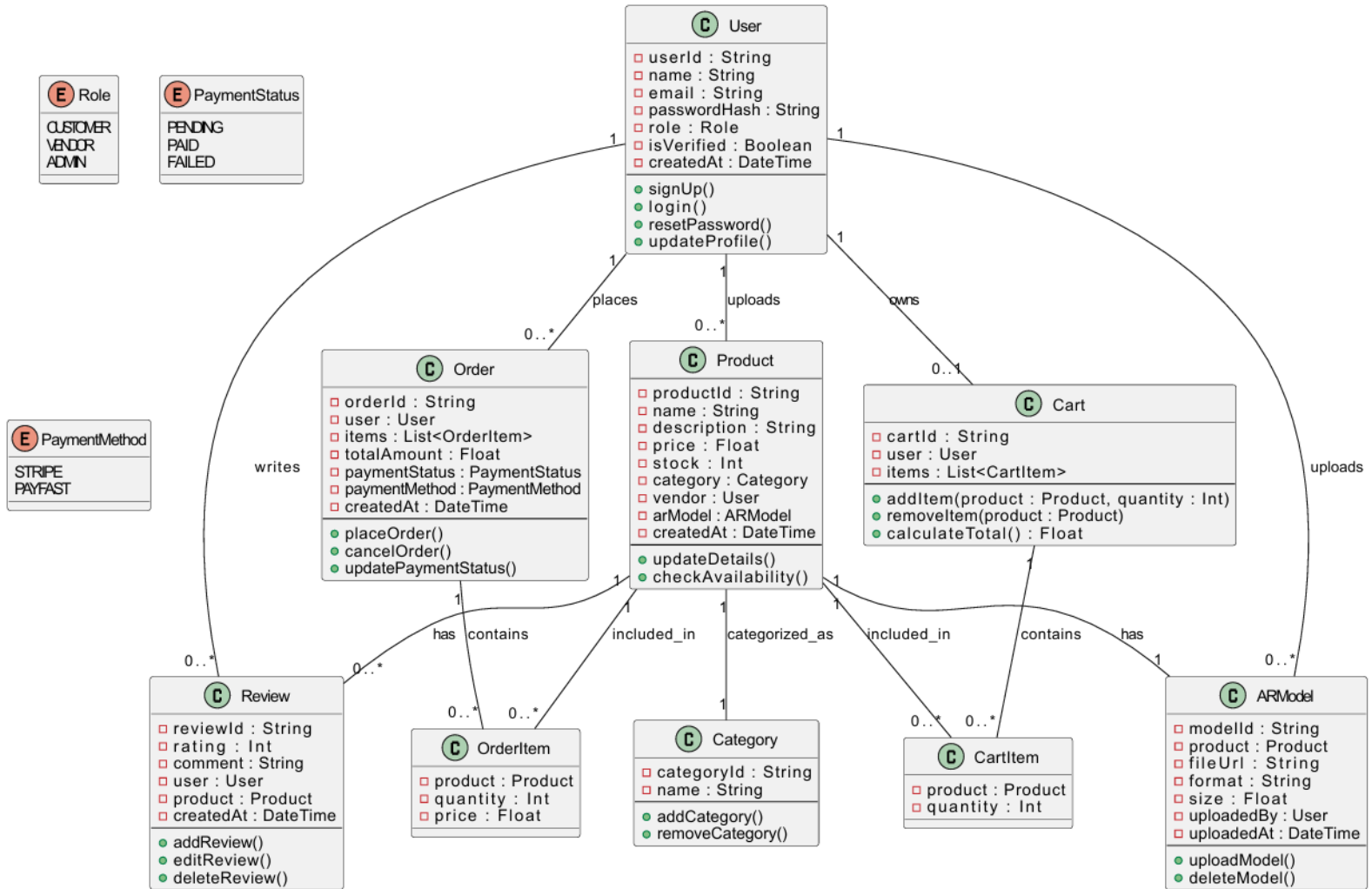


Figure 19 Class Diagram

4.5 Activity Diagrams

The activity Diagrams are mentioned and in detail for every aspect of the functionality of the application.

- User Registration Activity Diagram

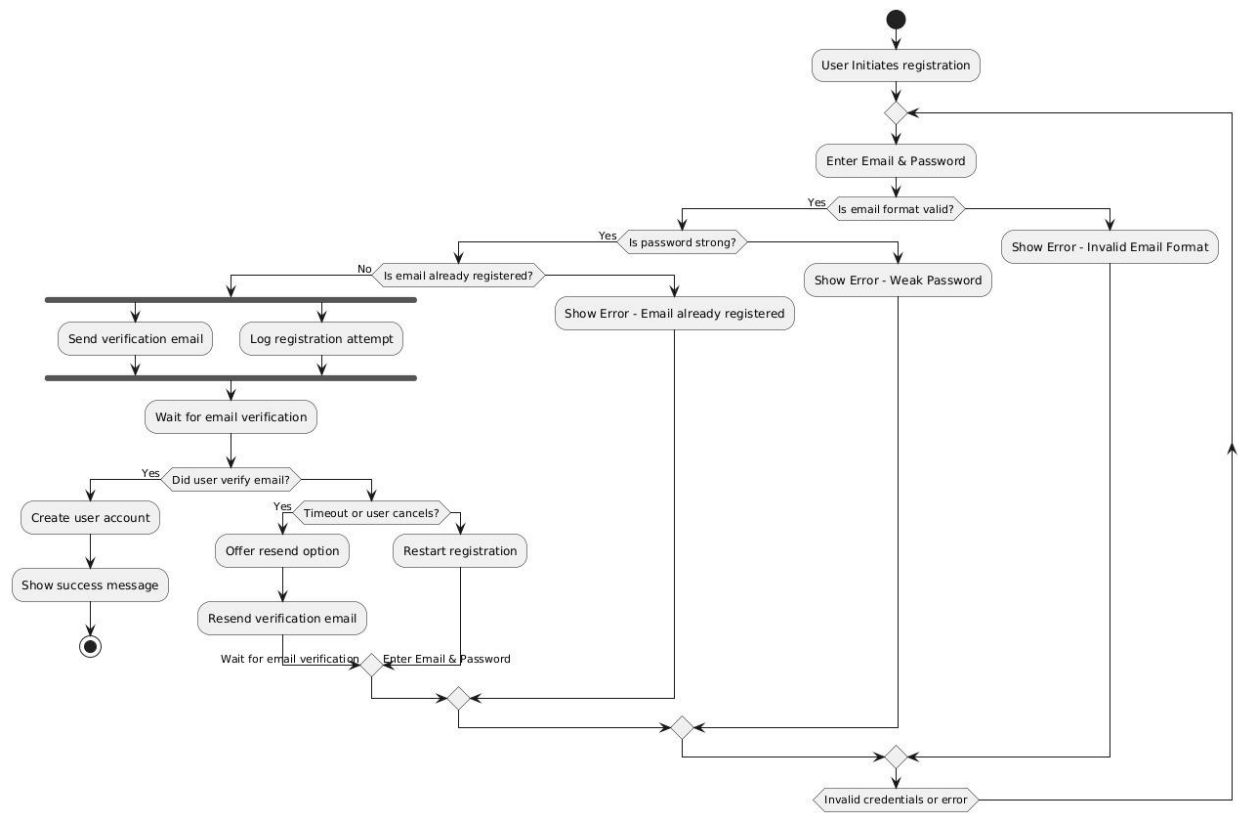


Figure 20 Activity Diagram → User Registration

- User Login Activity Diagram

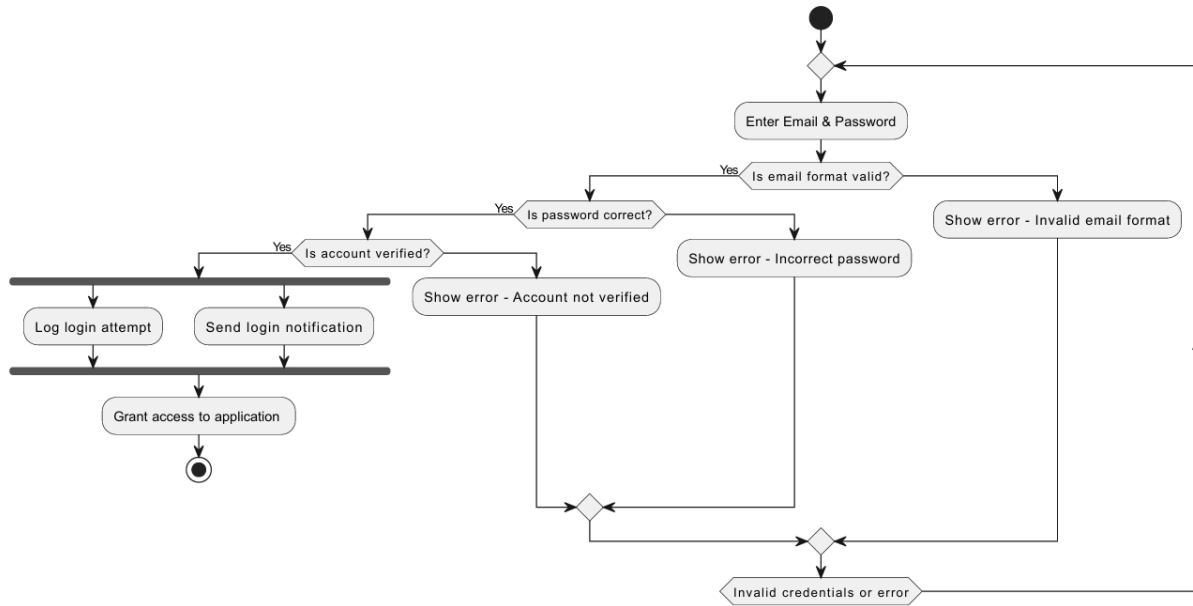


Figure 21 Activity Diagram → User Login

- View Products Activity Diagram

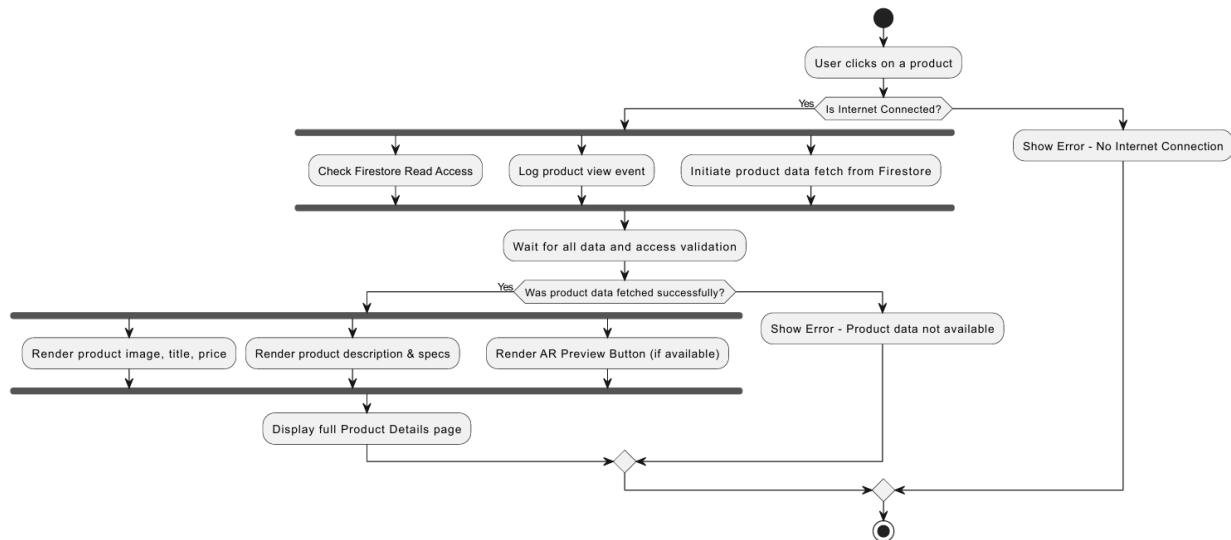


Figure 22 Activity Diagram → View Products

- AR view of the product Activity Diagram

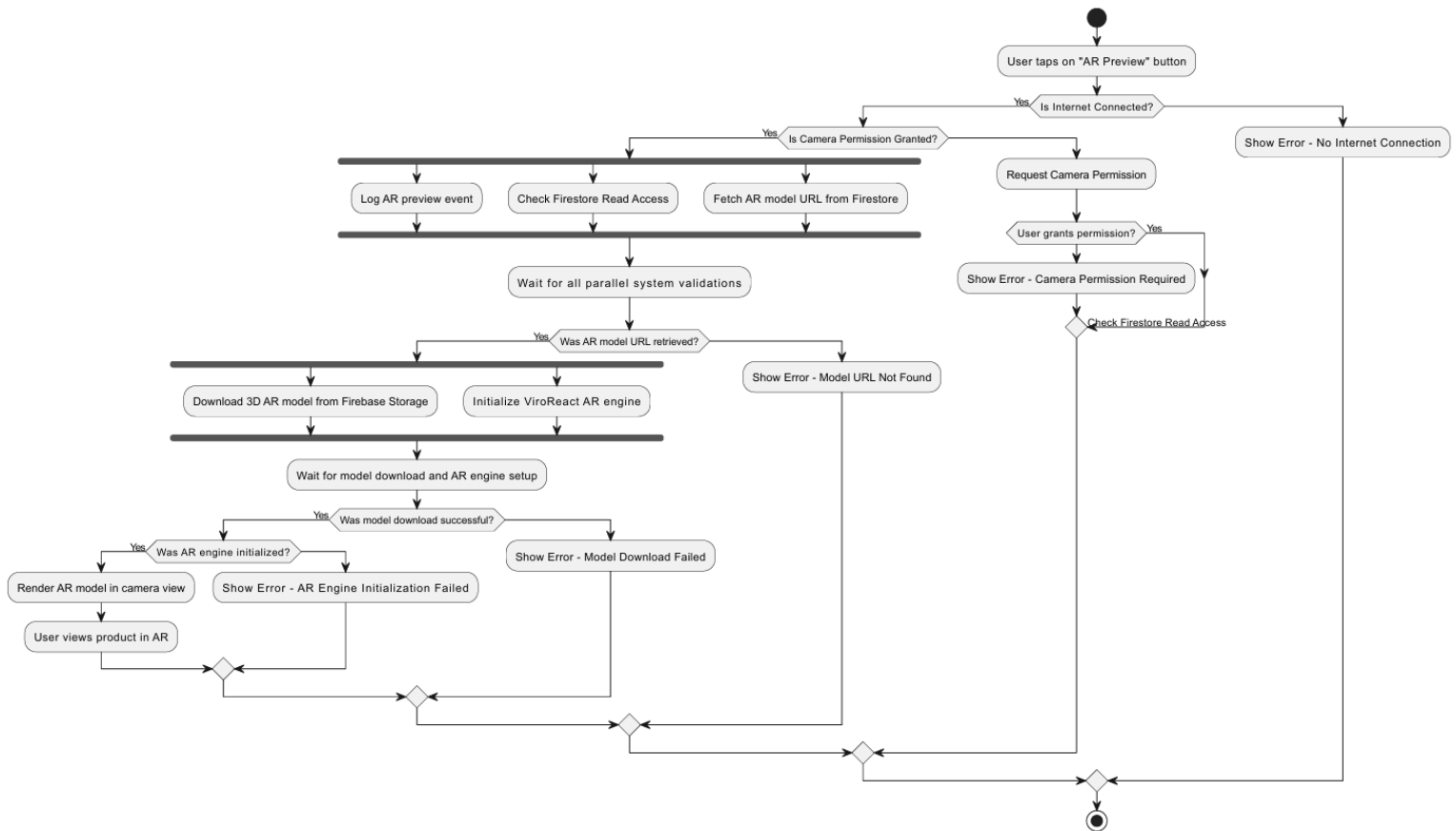


Figure 23 Activity Diagram → AR view of the product

- View products with AR (complete) Activity Diagram

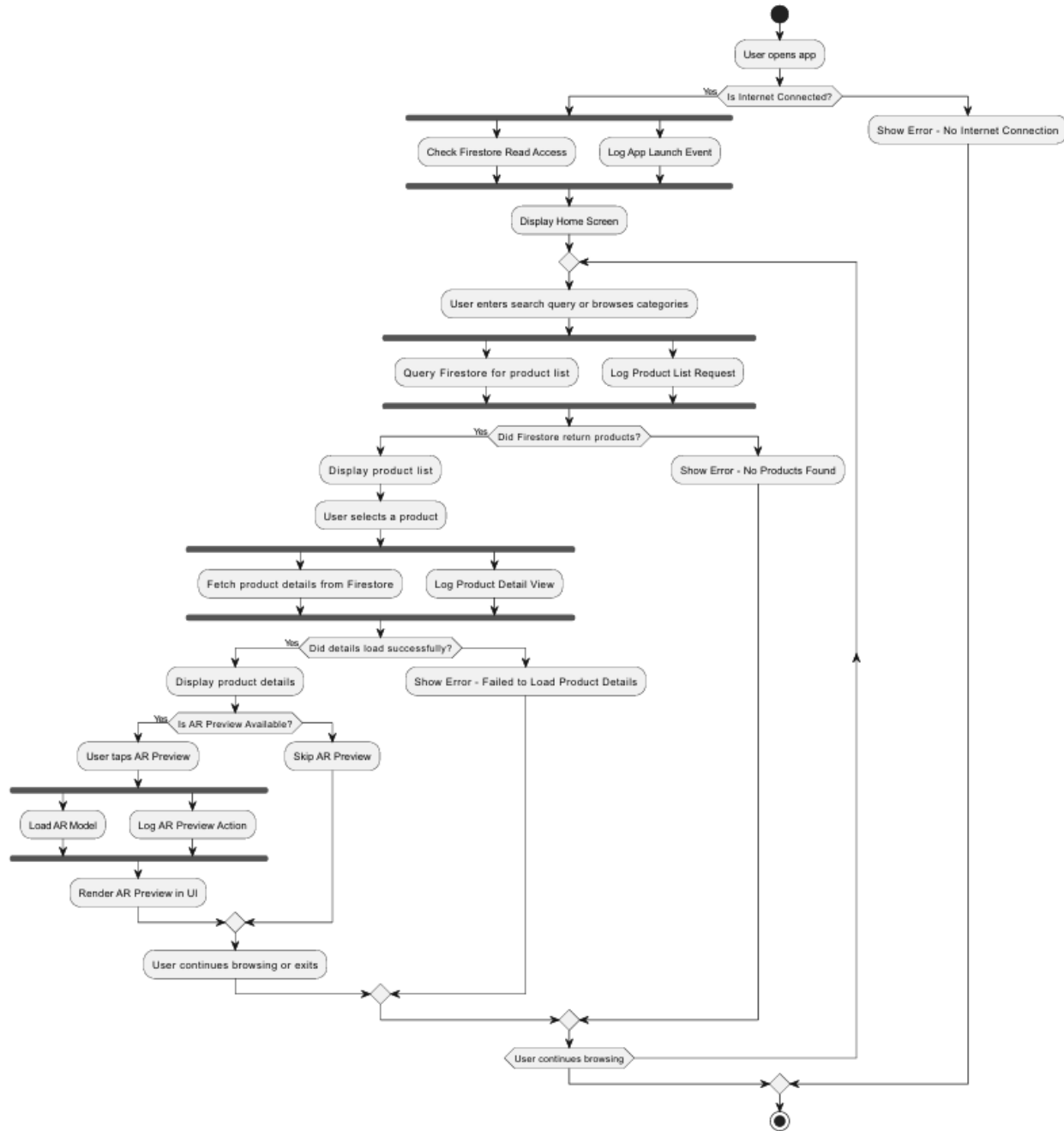


Figure 24 Activity Diagram → View Products with AR

- Search Product Activity Diagram

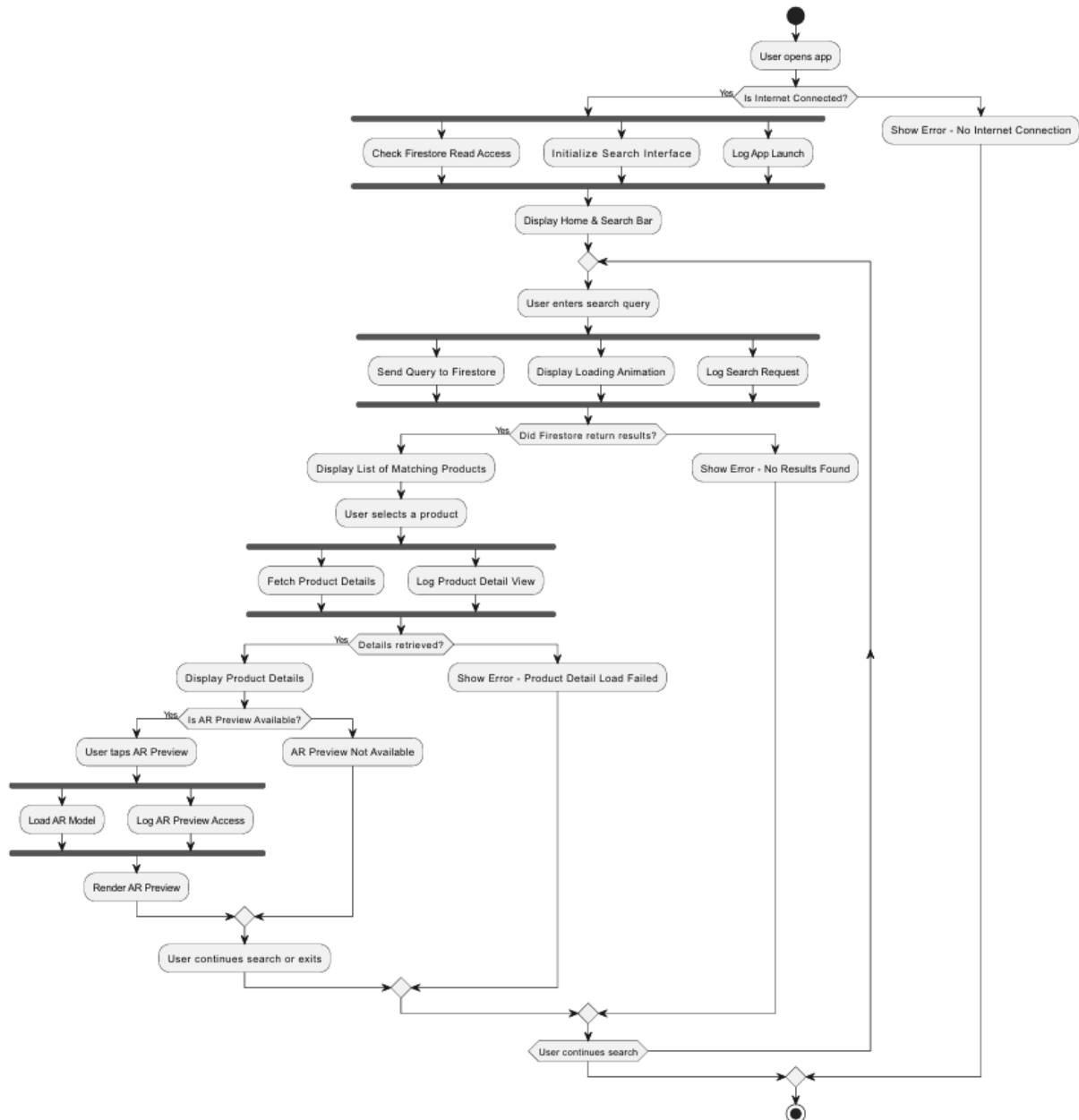


Figure 25 Activity Diagram → Search Product

- Filter Product Activity Diagram

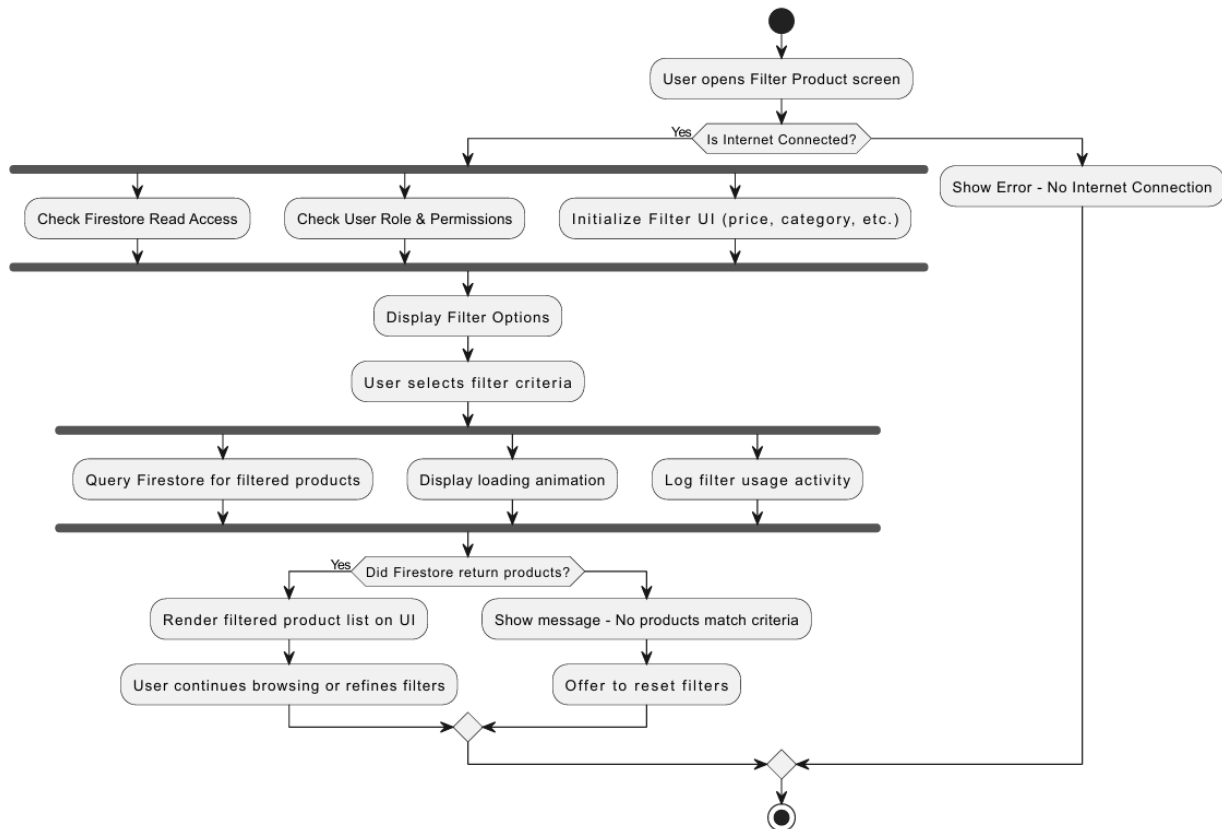


Figure 26 Activity Diagram → Filter Product

- Password Reset Activity Diagram

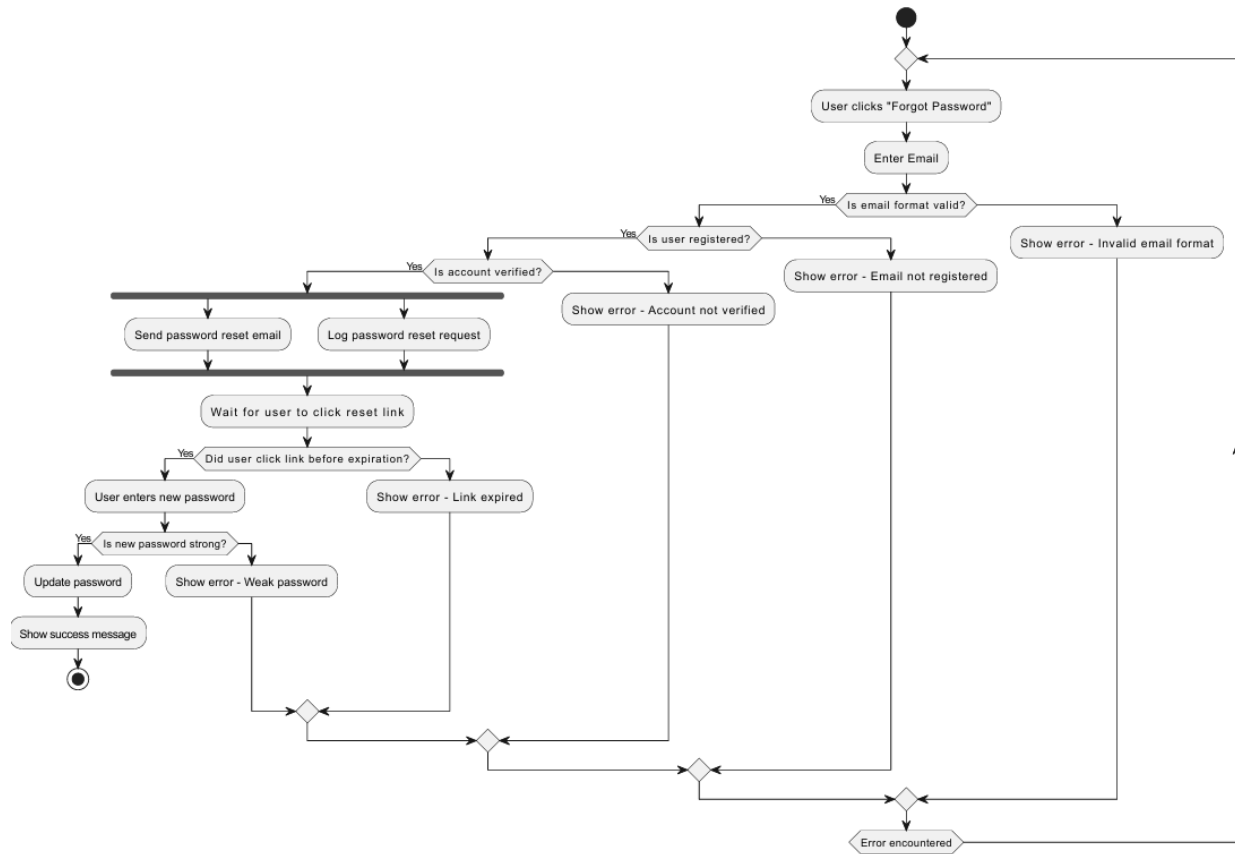


Figure 27 Activity Diagram → Password Reset

- Upload Review Activity Diagram

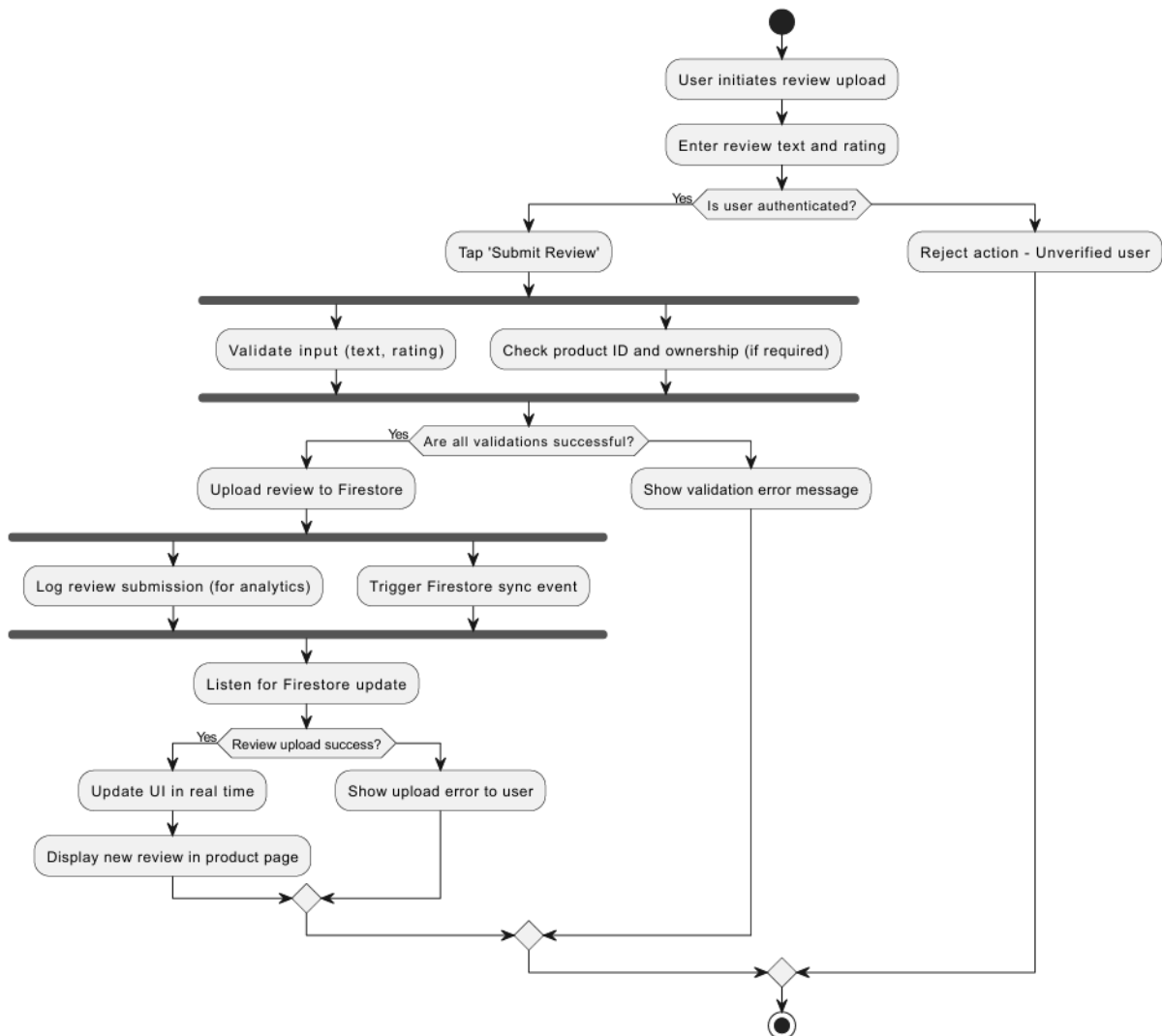


Figure 28 Activity Diagram → Upload Review

- Reply / Chat (Review) Feature Activity Diagram

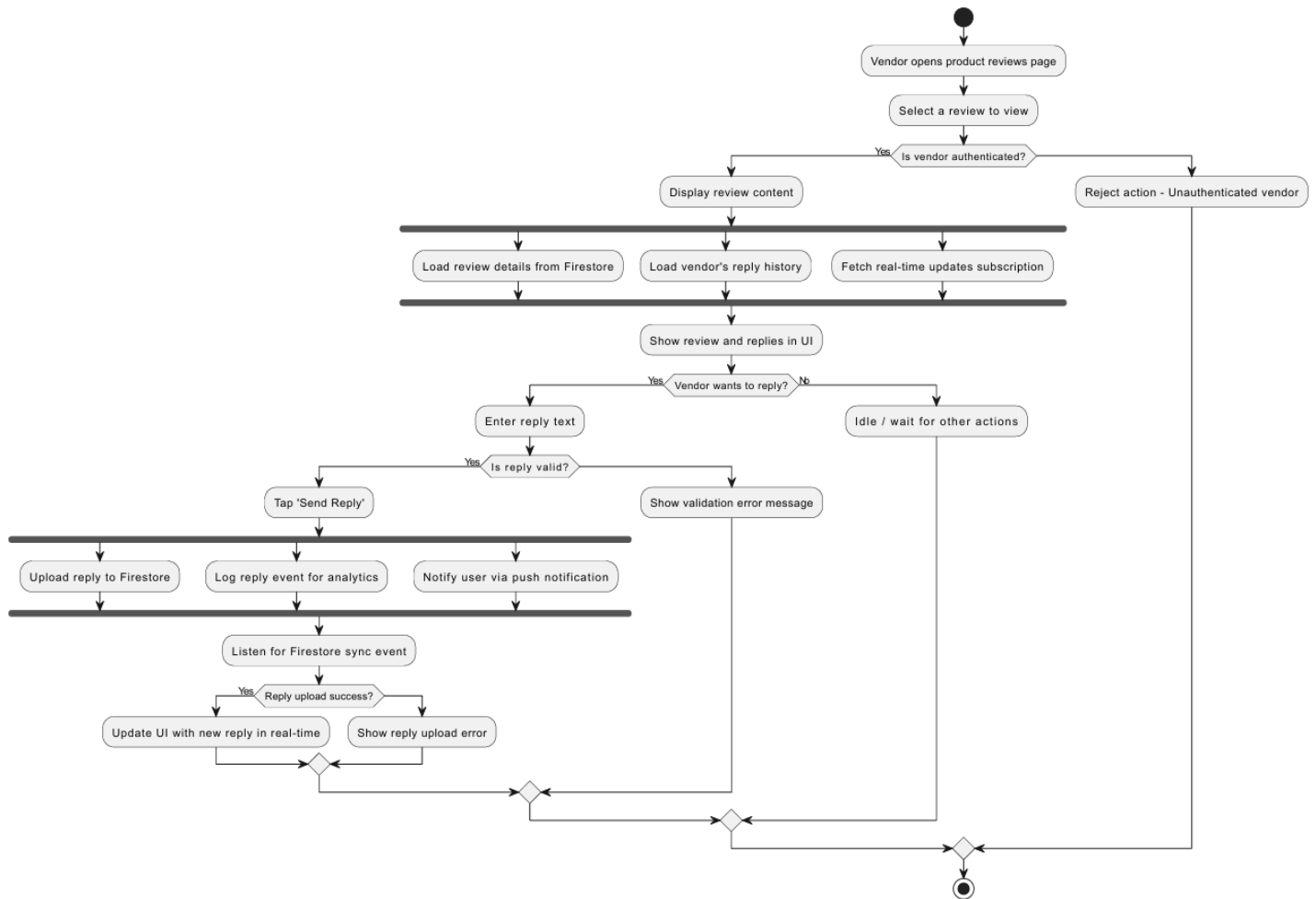


Figure 29 Activity Diagram → Reply / Chat (Review)

- Wishlist View and Management Activity Diagram

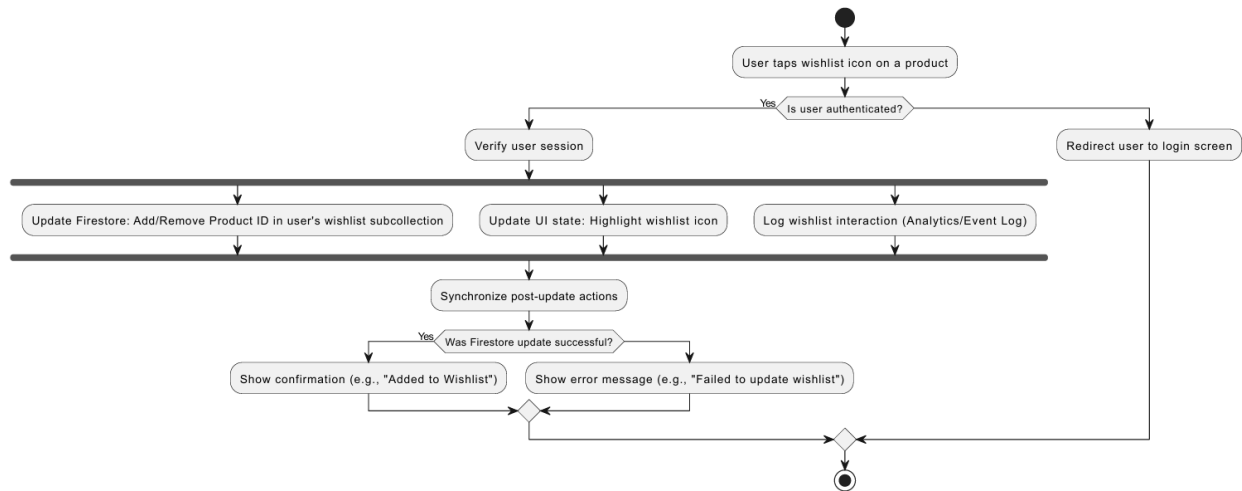


Figure 30 Activity Diagram → Wishlist View and Management

- User Profile View/Update Activity Diagram

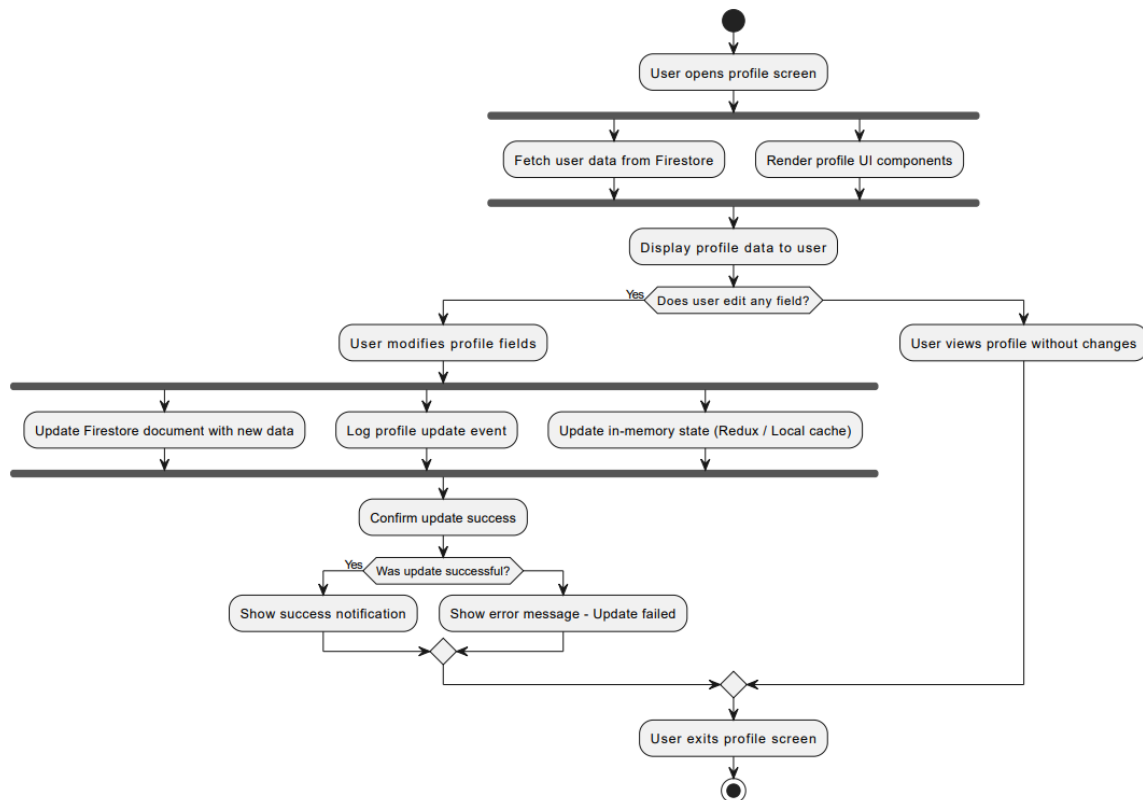


Figure 31 Activity Diagram → User Profile View / Update

- User Authentication at Checkout (Payment) Activity Diagram

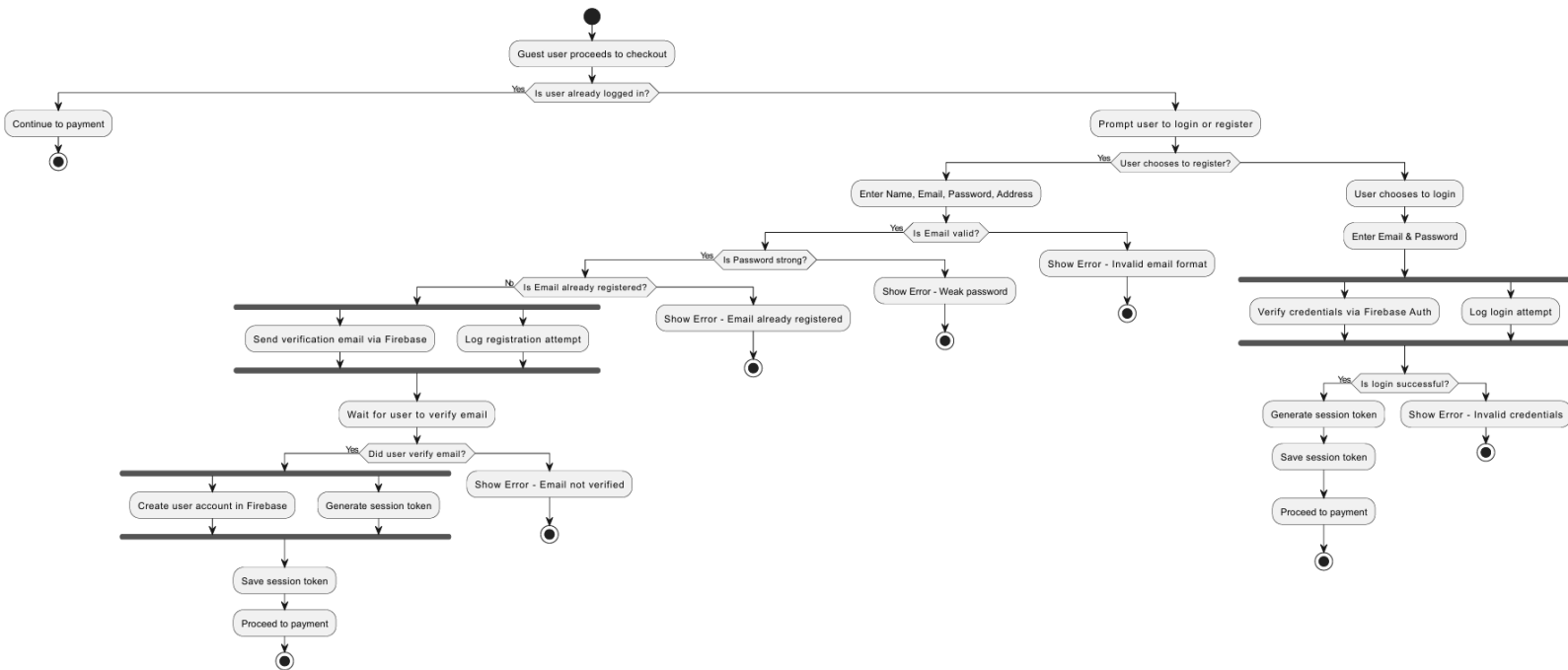


Figure 32 Activity Diagram → User Auth at Checkpoint

- Admin Managing Users Activity Diagram

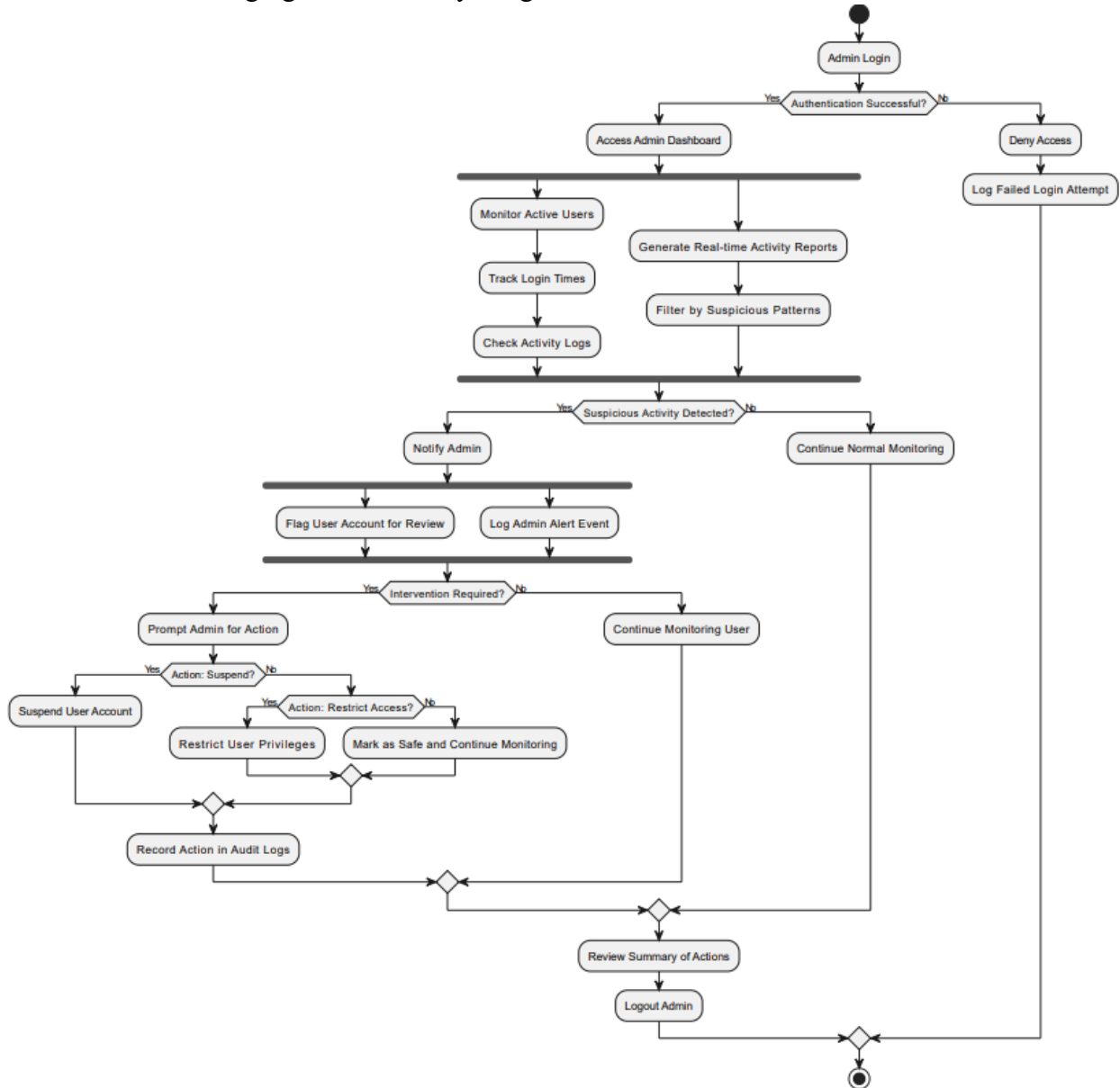


Figure 33 Activity Diagram → Admin Managing Users

- Admin View User Chats & Reviews Activity Diagram

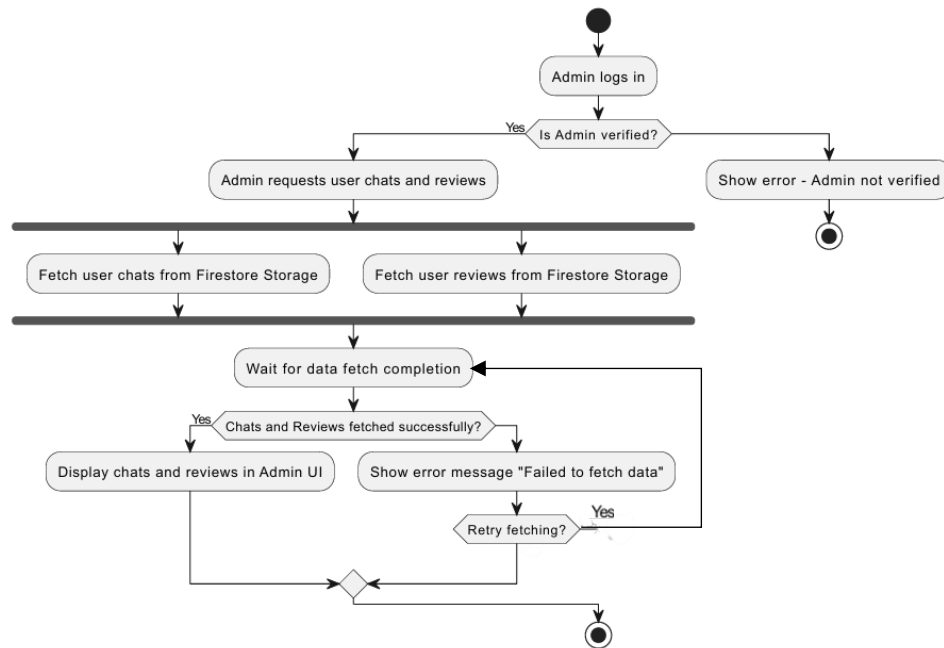


Figure 34 Activity Diagram → Admin View User Chats & Reviews

- Admin Product Management

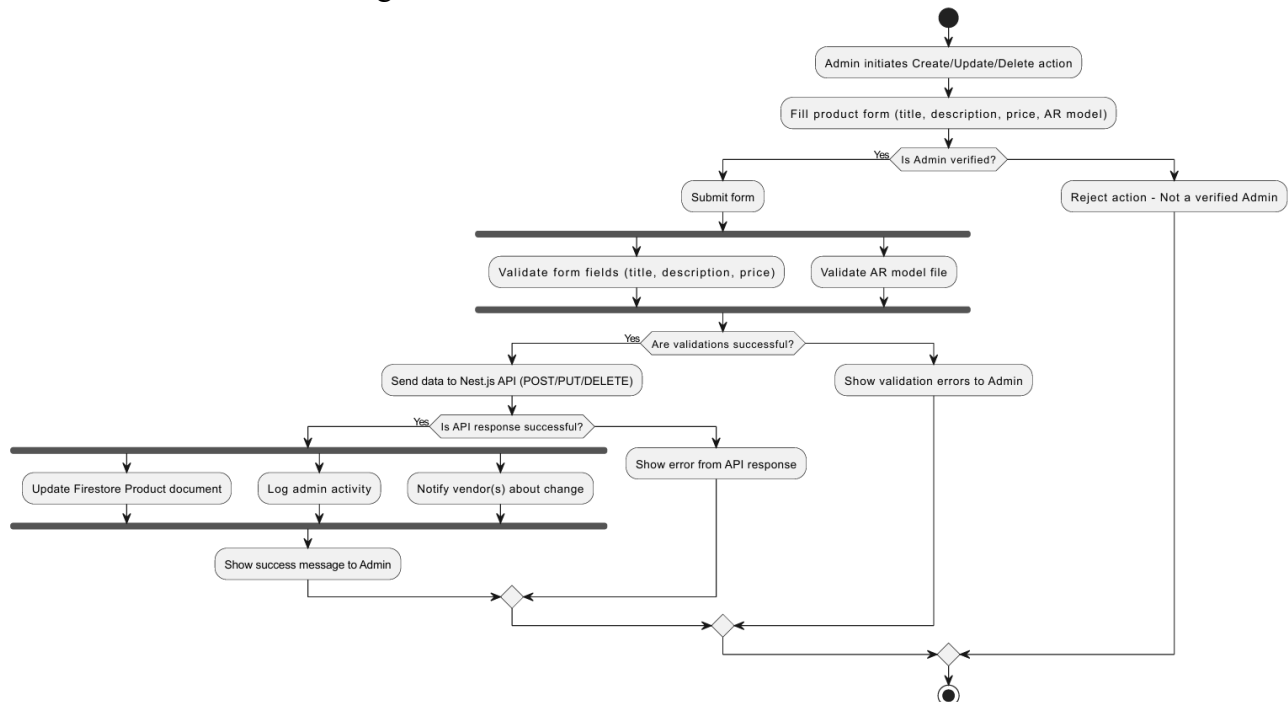


Figure 35 Activity Diagram → Admin Product Management

- Vendor Product Management

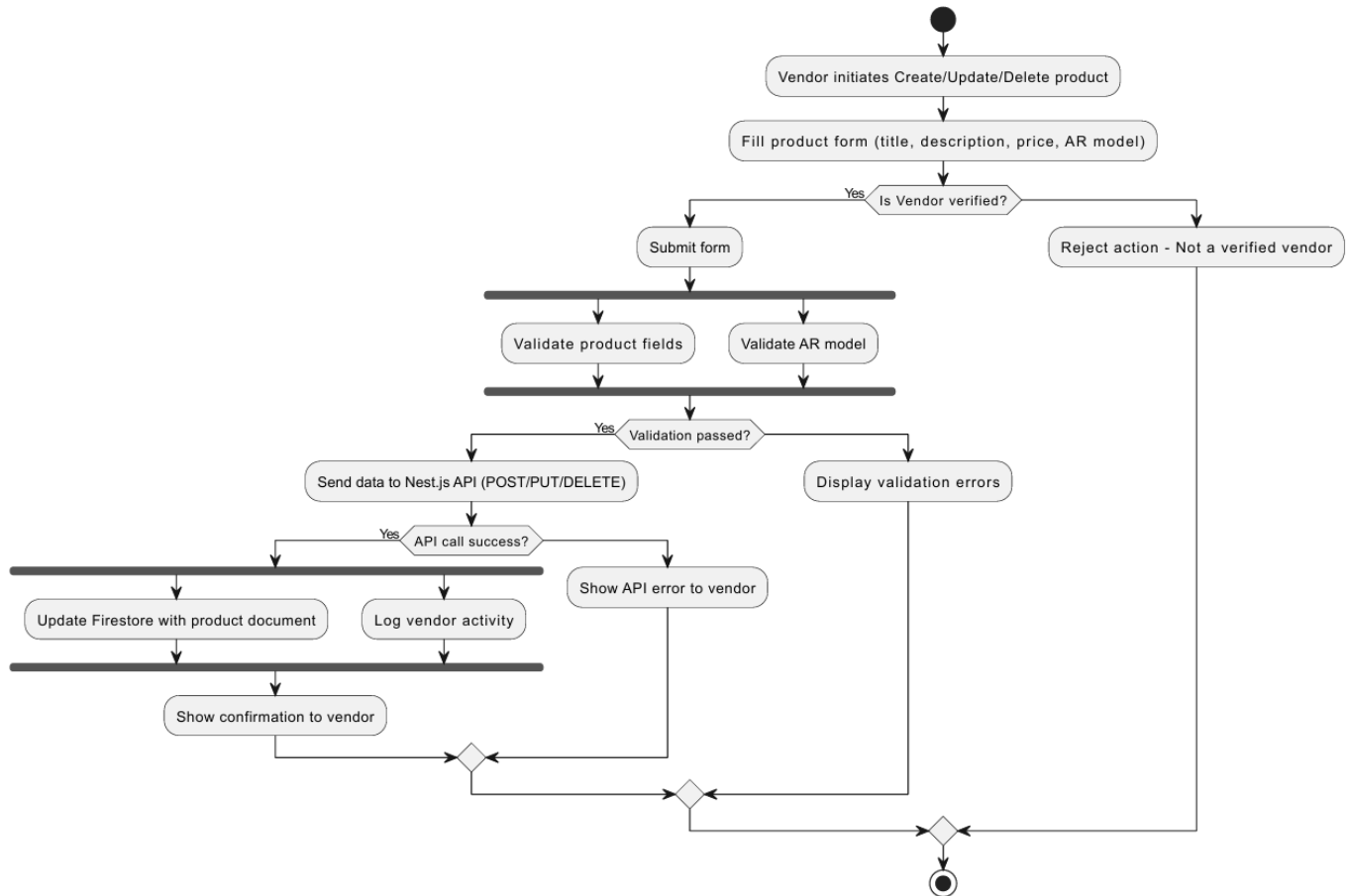


Figure 36 Activity Diagram → Vendor Product Management

4.6 Sequence Diagram

- **Customer Functionalities Sequence Diagrams**

- Add to Cart Sequence Diagram

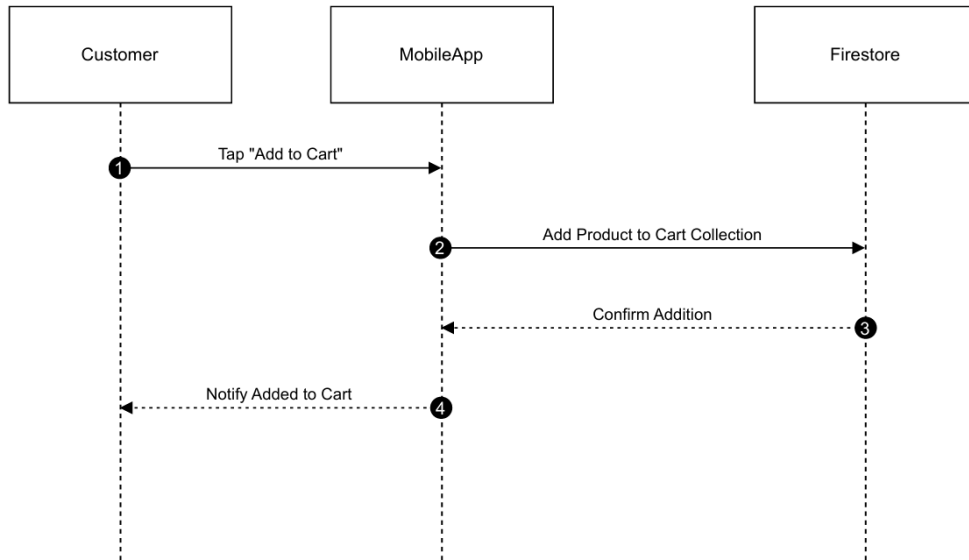


Figure 37 Sequence Diagram → Add to Cart

- Add to Wishlist Sequence Diagram

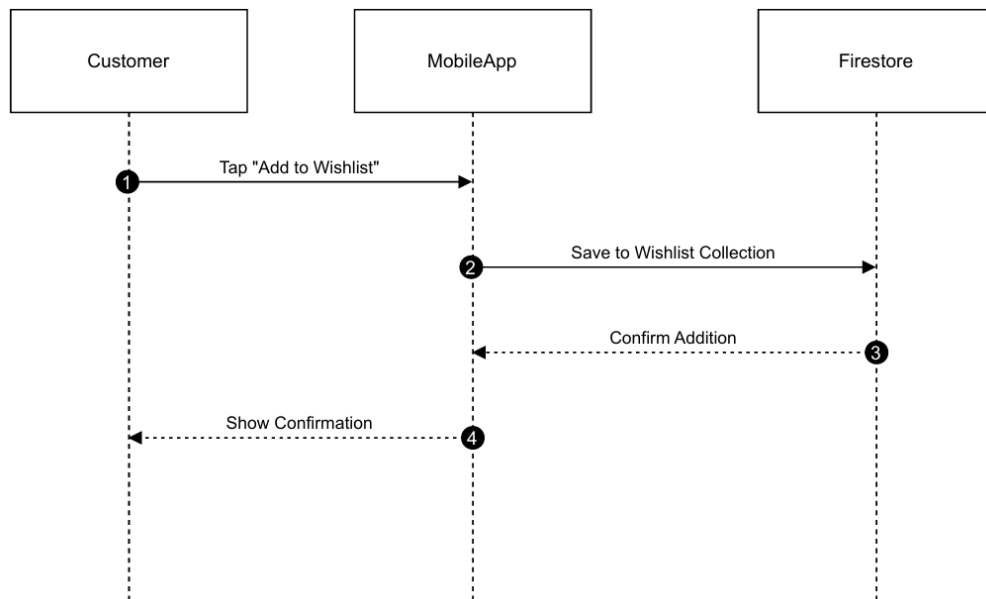


Figure 38 Sequence Diagram → Add to Wishlist

○ Browse by Filter Sequence Diagram.

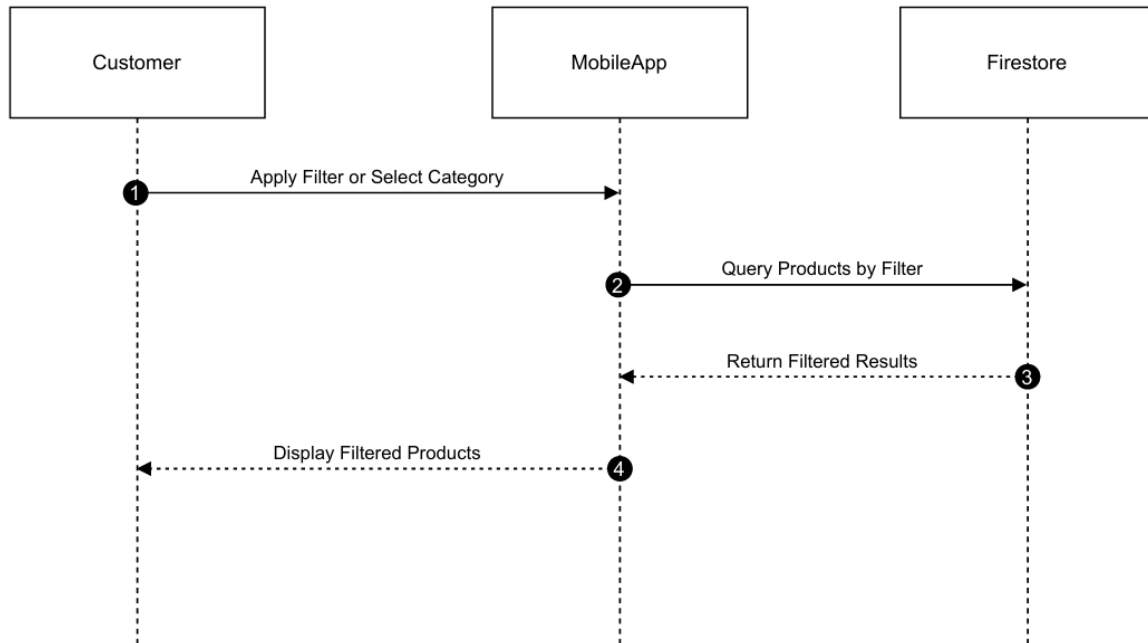


Figure 39 Sequence Diagram → Browse by Filter

○ Chat with Vendor Sequence Diagram

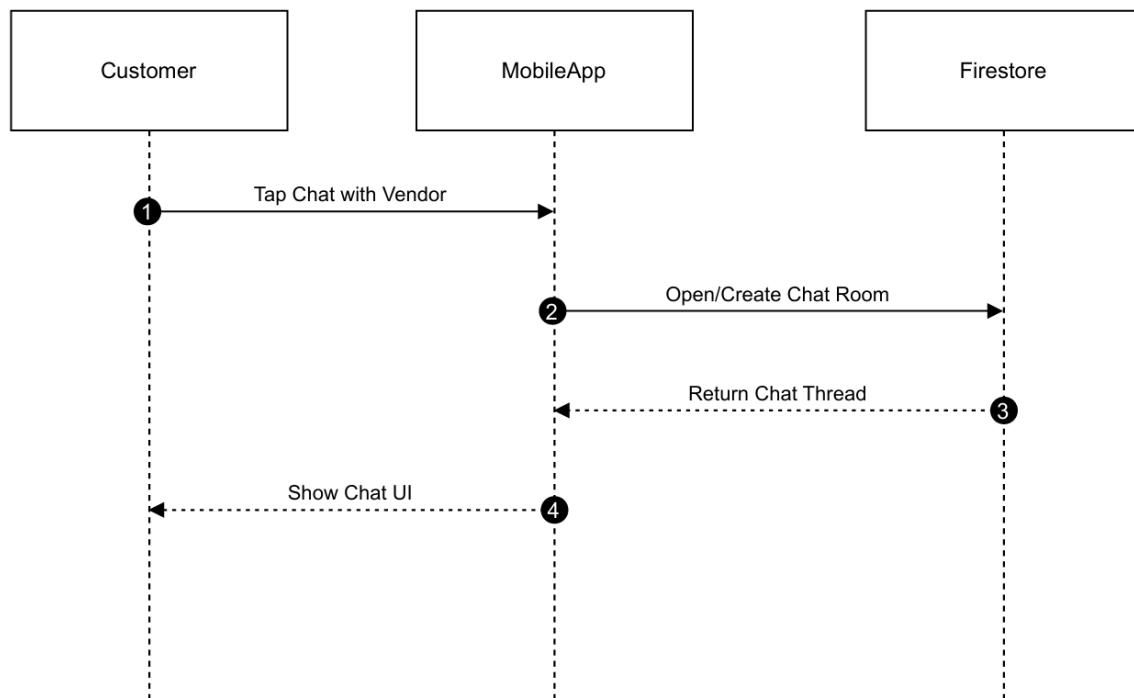


Figure 40 Sequence Diagram → Chat with Vendor

○ Checkout Process Sequence Diagram

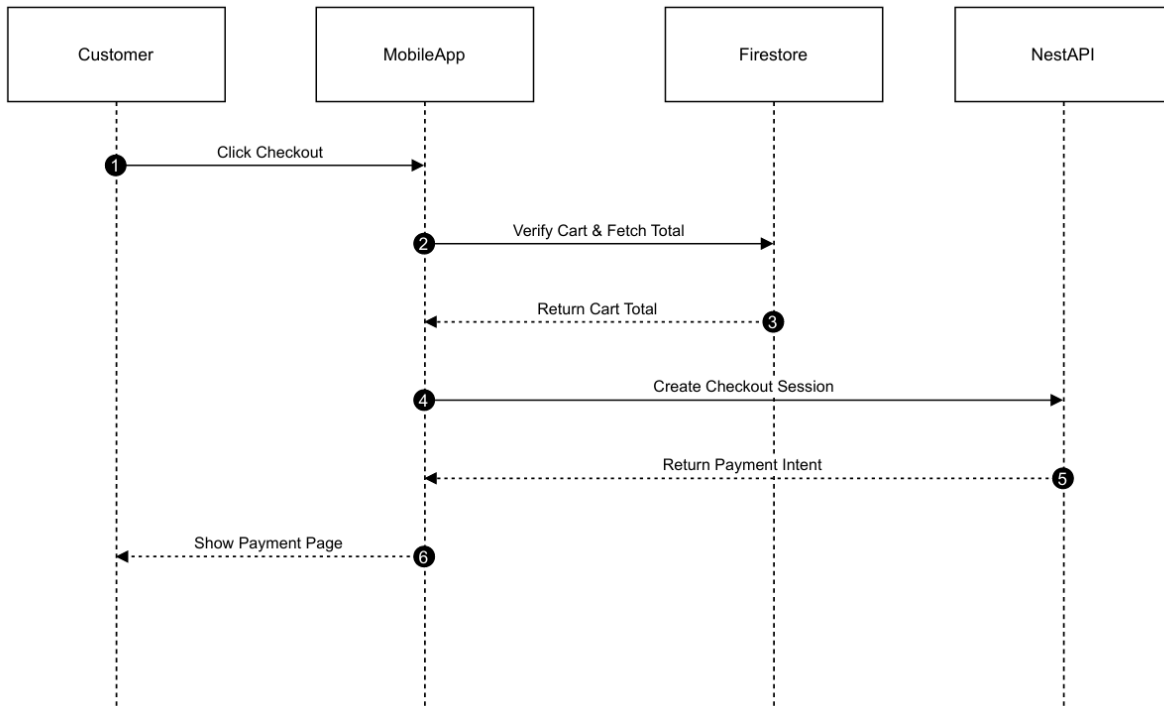


Figure 41 Sequence Diagram → Checkout Process

○ Product Browsing Sequence Diagram

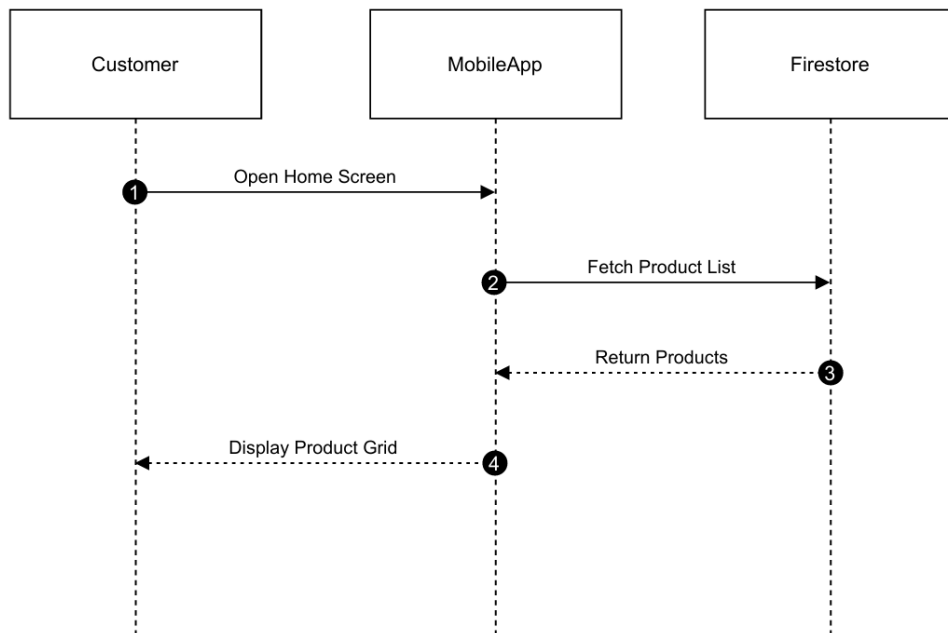


Figure 42 Sequence Diagram → Product Browsing

○ Submit Product Review Sequence Diagram

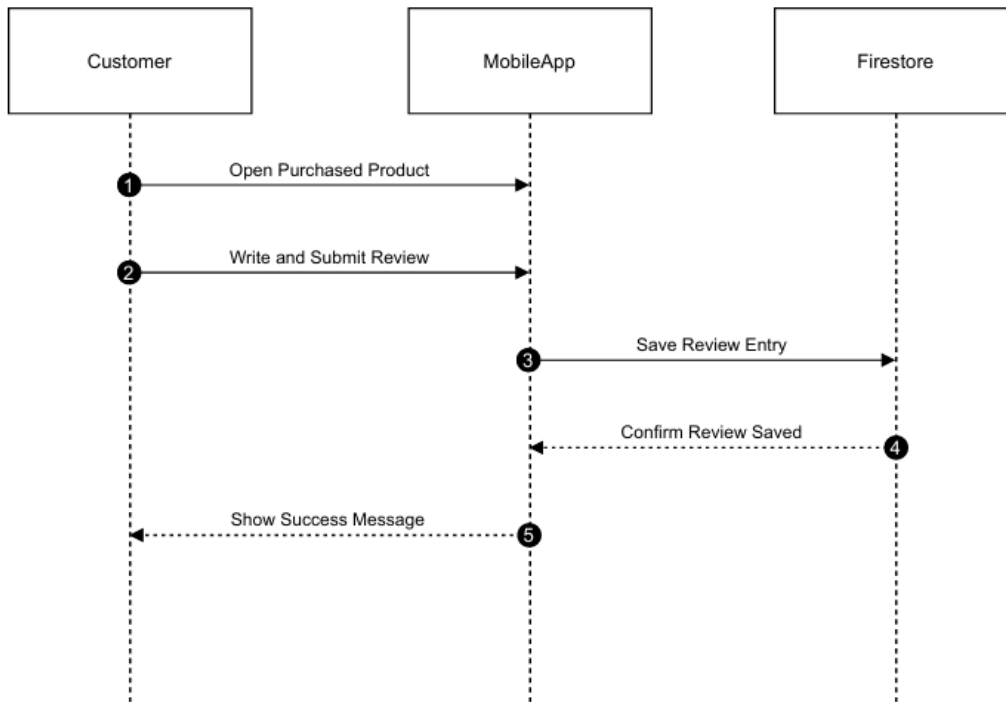


Figure 43 Sequence Diagram → Submit Product Review

○ View Cart Sequence Diagram

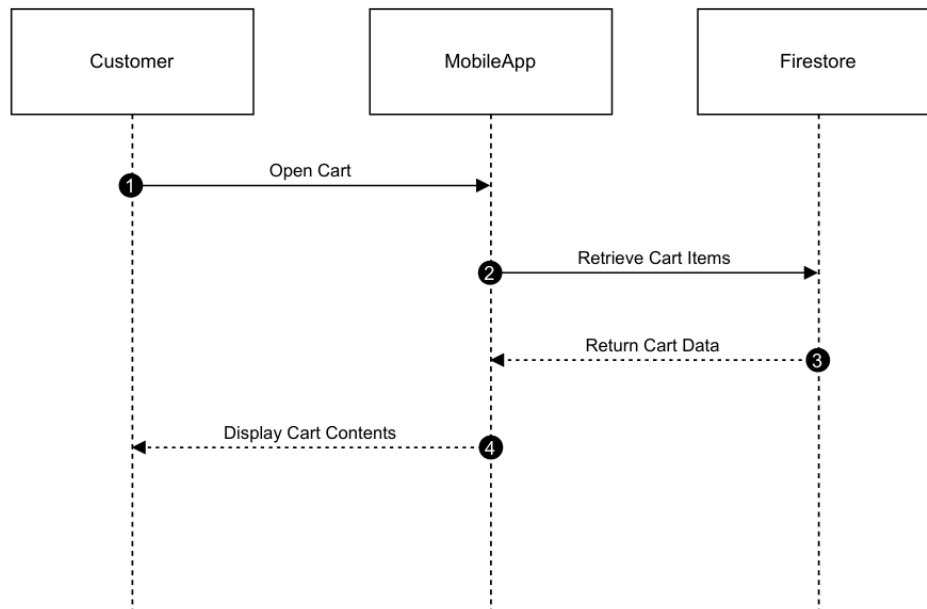


Figure 44 Sequence Diagram → View Cart

○ View Product Details with AR Sequence Diagram

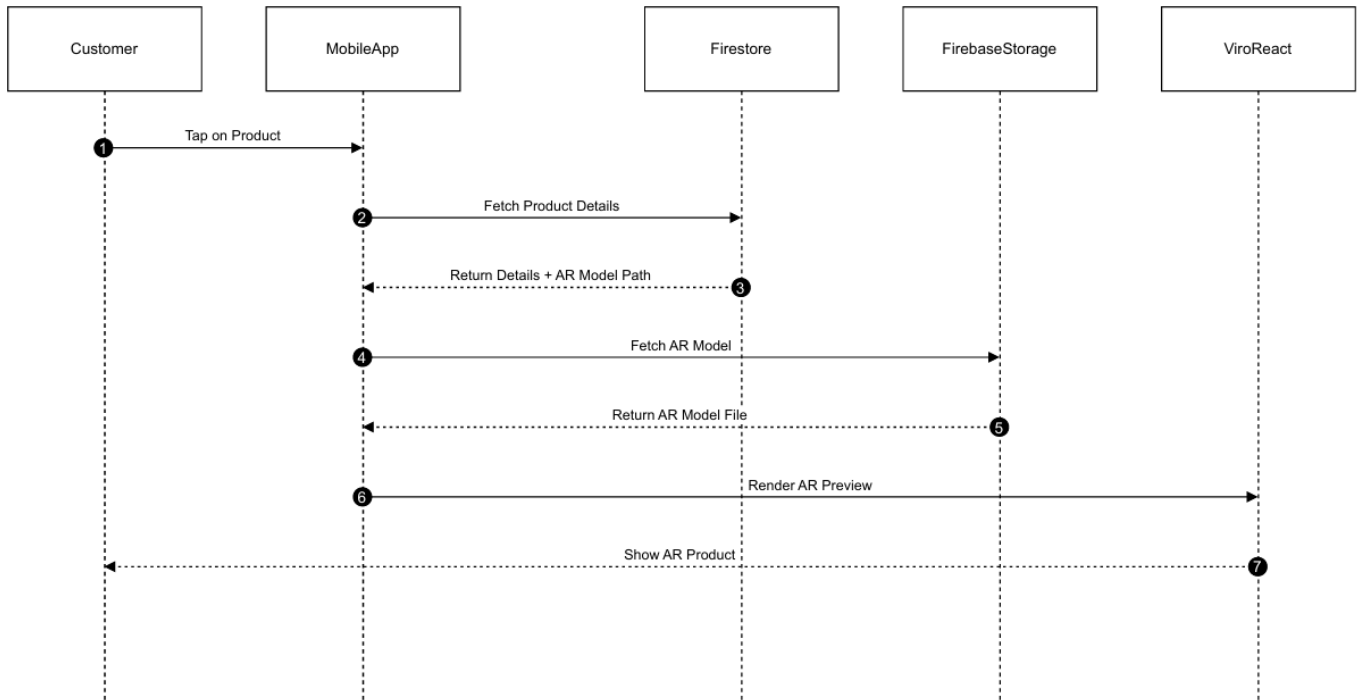


Figure 45 Sequence Diagram → View Product Details with AR

○ View Wishlist Sequence Diagram

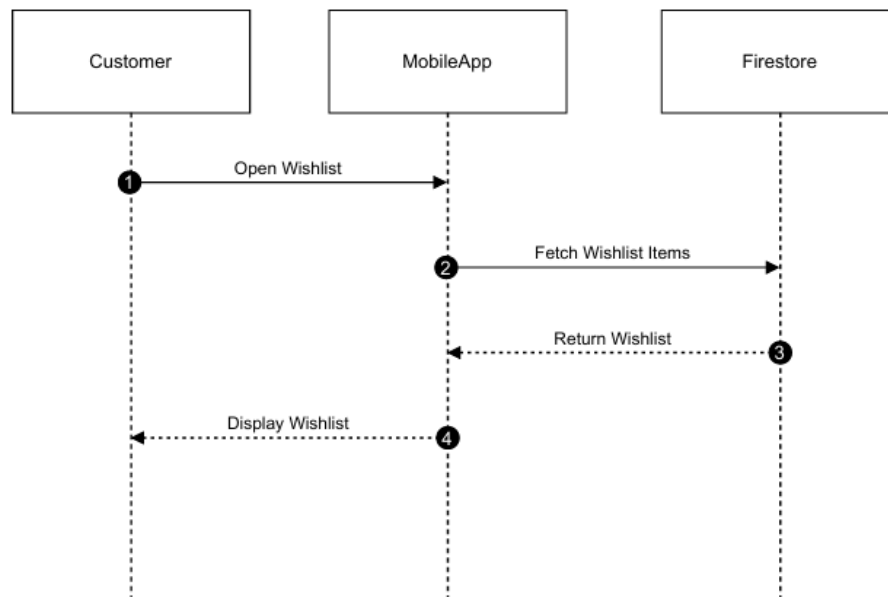


Figure 46 Sequence Diagram → View Wishlist

- **Admin / Vendor Functionality Sequence Diagrams**

- Access Admin Dashboard Sequence Diagram

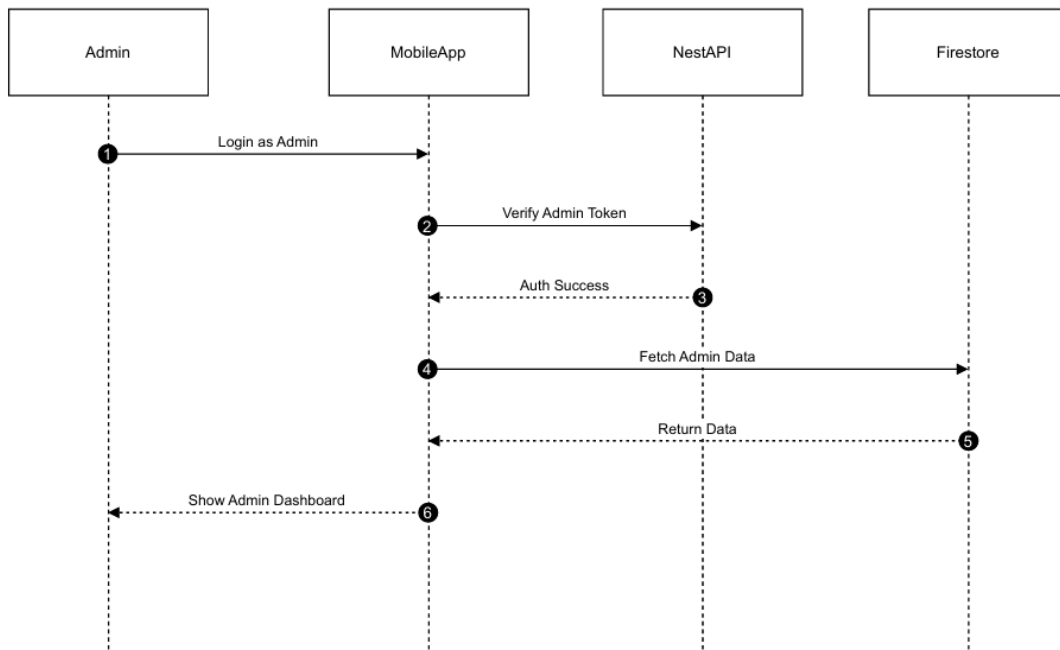


Figure 47 Sequence Diagram → Access Admin Dashboard

- Add New Product Sequence Diagram

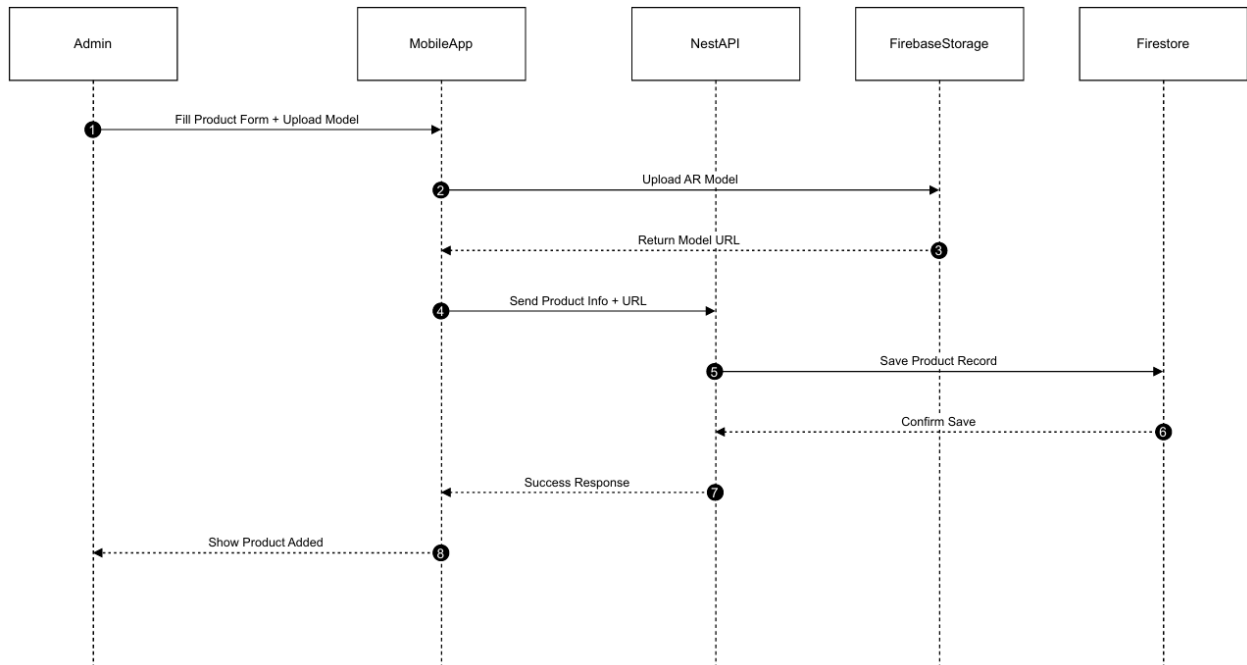


Figure 48 Sequence Diagram → Add New Product

○ Ban and Unban User Sequence Diagram

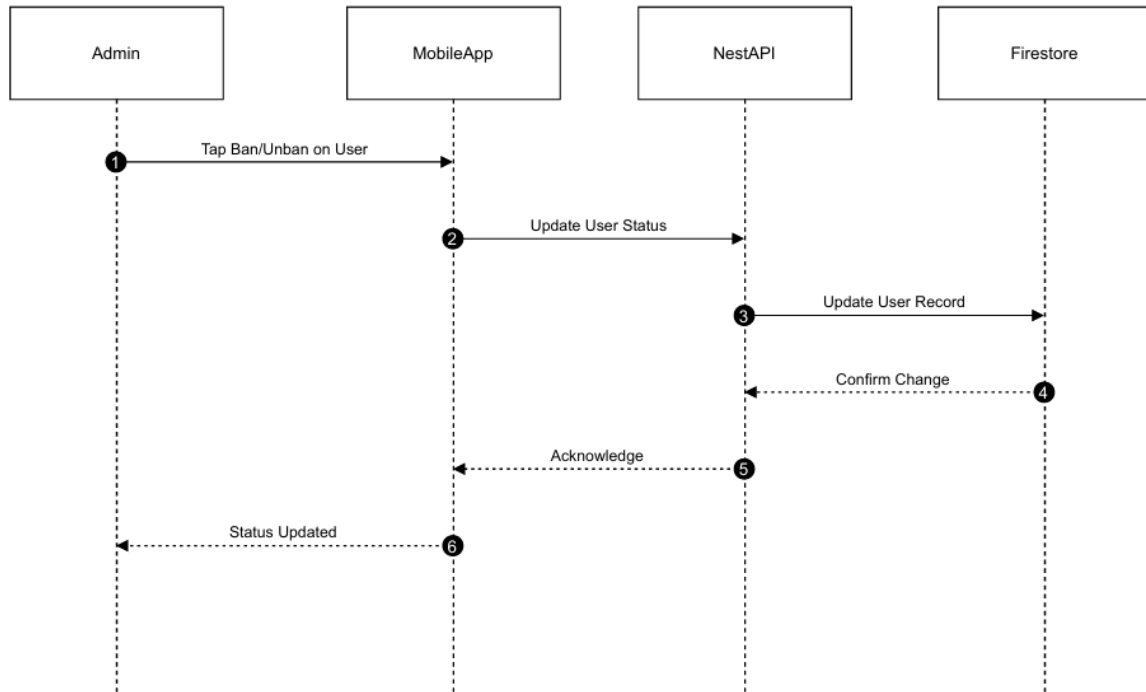


Figure 49 Sequence Diagram → Ban and Unban User

○ Delete Product Sequence Diagram

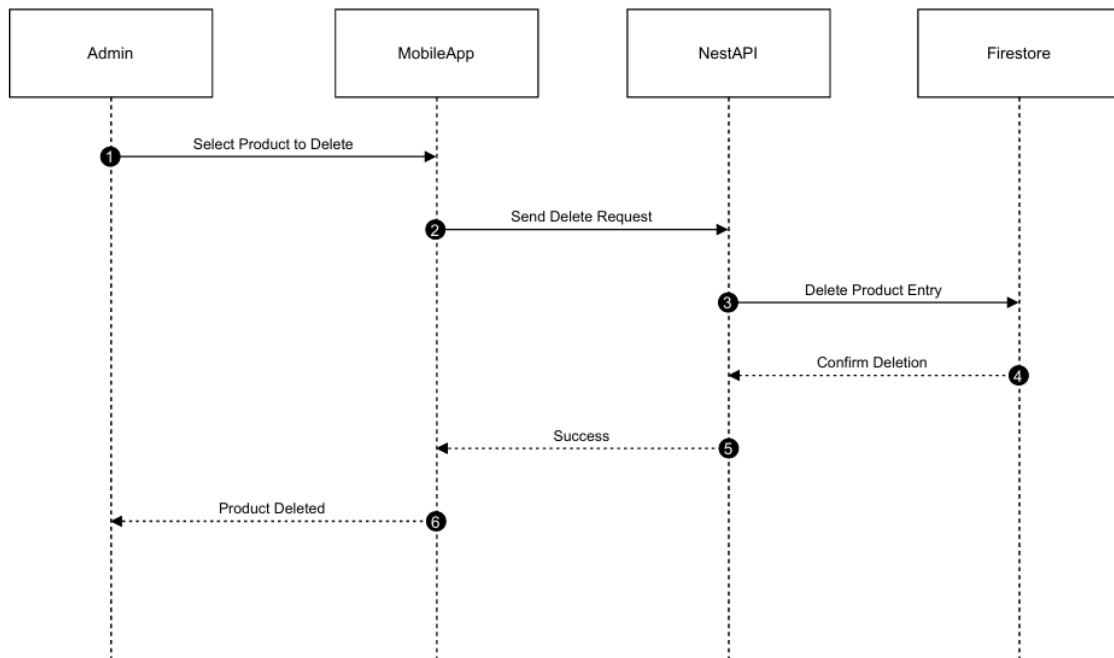


Figure 50 Sequence Diagram → Delete Product

○ Edit Existing Product Sequence Diagram

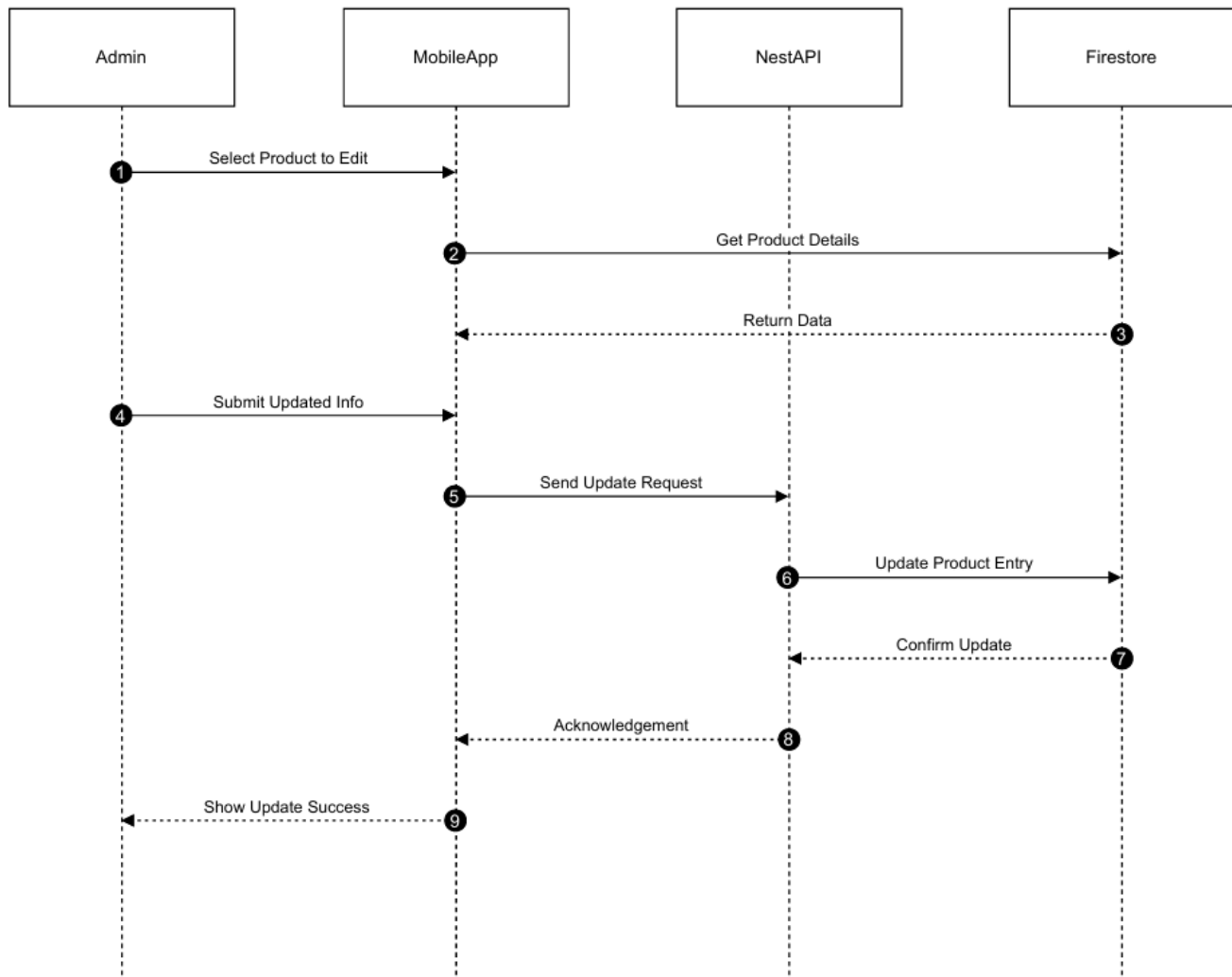


Figure 51 Sequence Diagram → Edit Existing Product

○ View Product Reviews Sequence Diagram

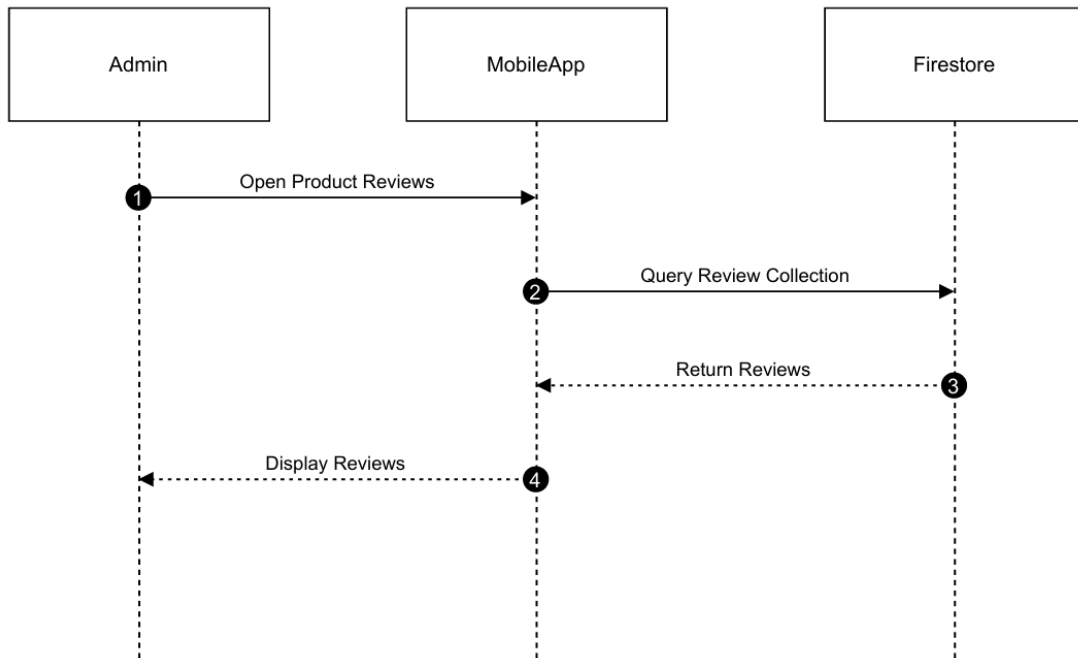


Figure 52 Sequence Diagram → View Product Reviews

○ View Users Sequence Diagram

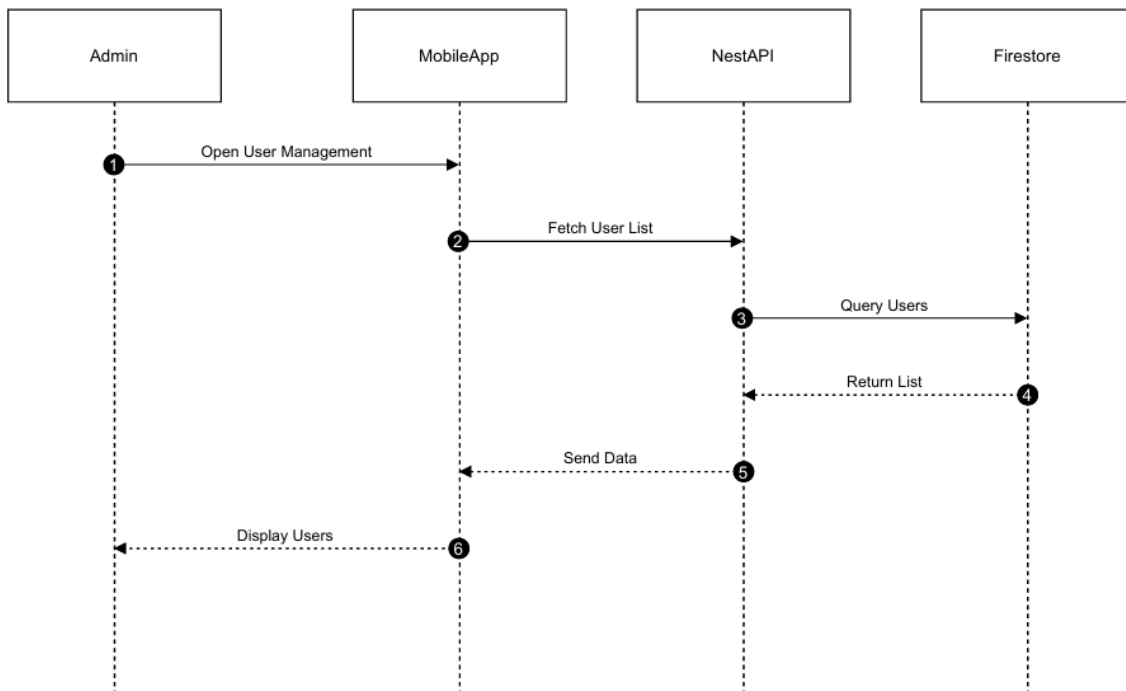


Figure 53 Sequence Diagram → View Users

- **Other Functionalities Sequence Diagrams**

- Push Notifications Sequence Diagram.

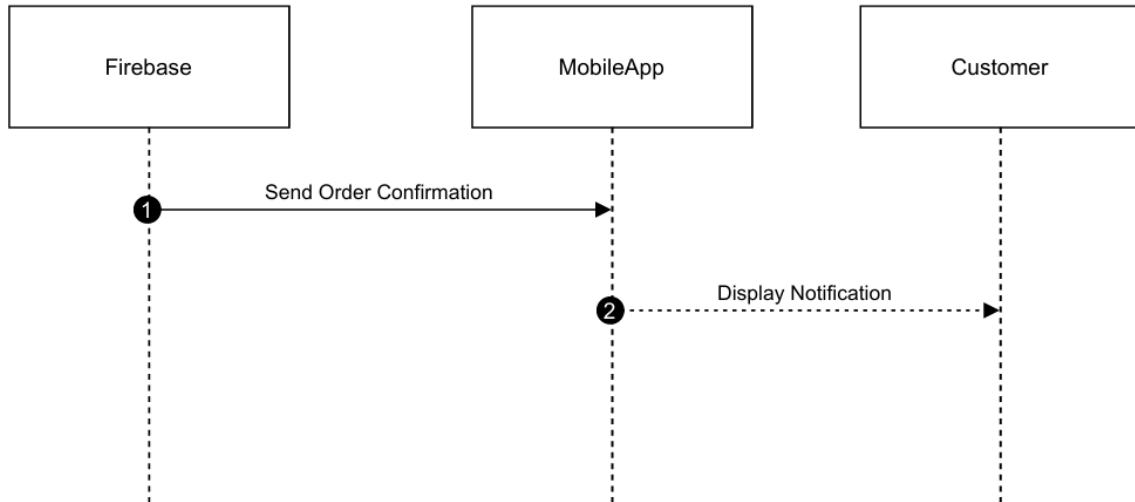


Figure 54 Sequence Diagram → Push Notifications

- Real-Time Chat Sequence Diagram.

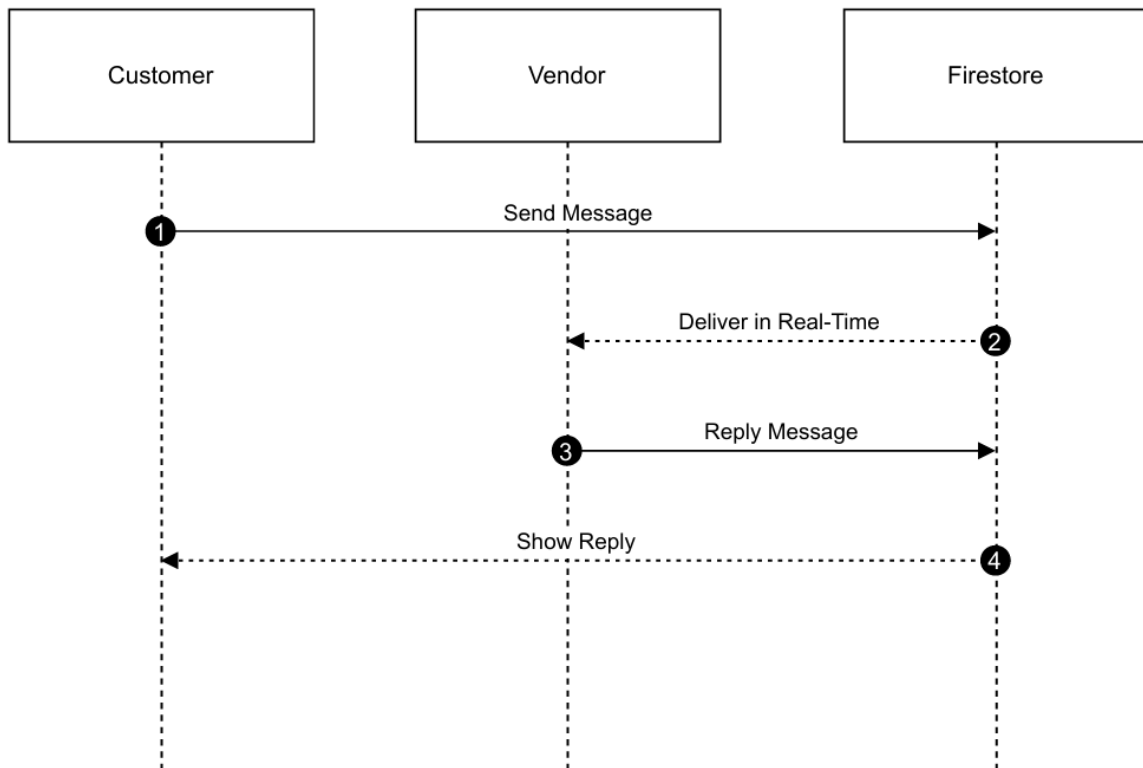


Figure 55 Sequence Diagram → Real-Time Chat

○ Vendor Product Upload Sequence Diagram.

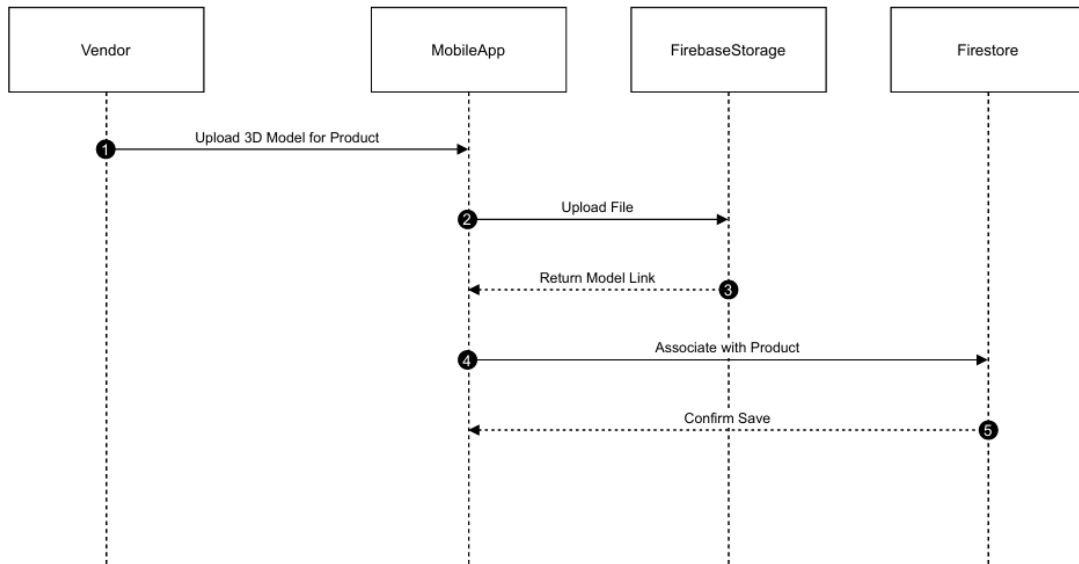


Figure 56 Sequence Diagram → Vendor Product Upload

• User Account Management Sequence Diagrams

○ Admin Login Sequence Diagram.

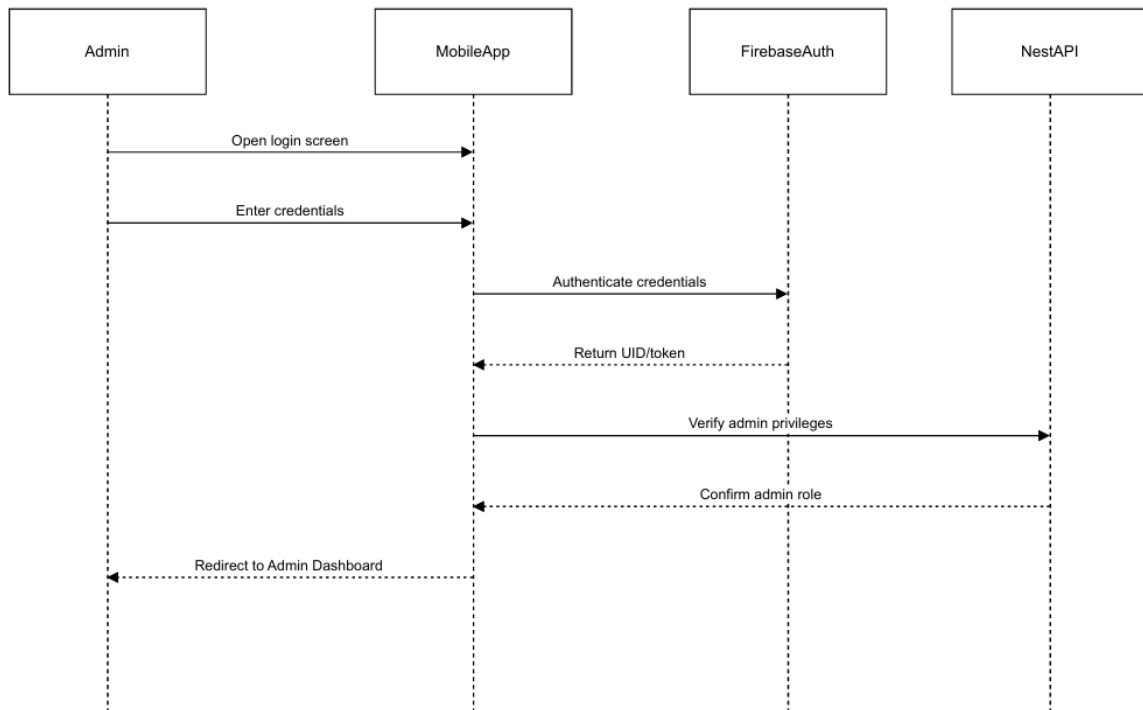


Figure 57 Sequence Diagram → Admin Login

○ Forgot Password & Password Reset Sequence Diagram.

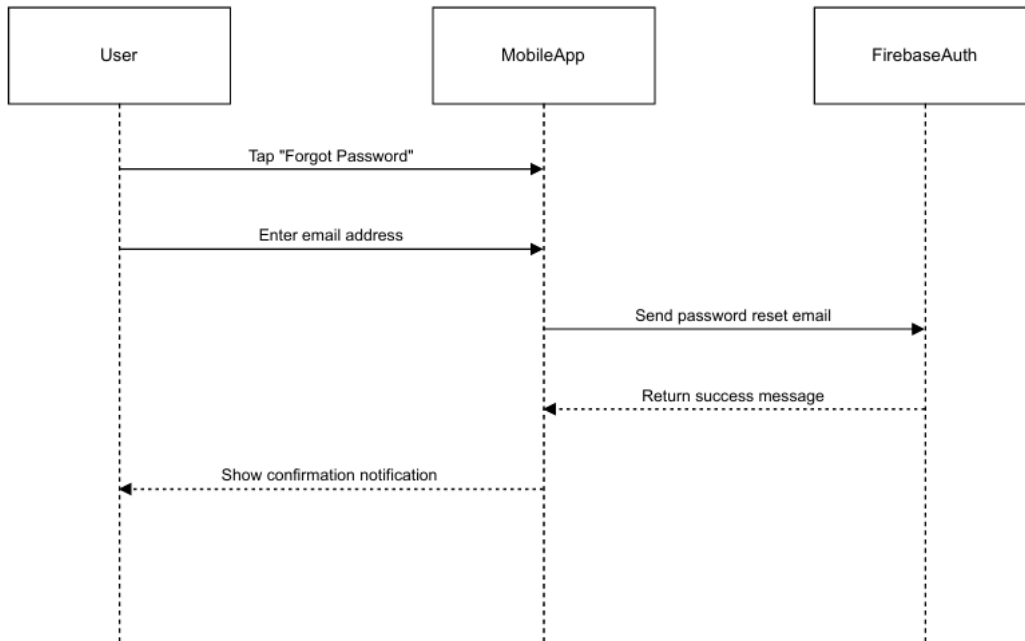


Figure 58 Sequence Diagram → Forgot Password & Password Reset

○ User Login Sequence Diagram.

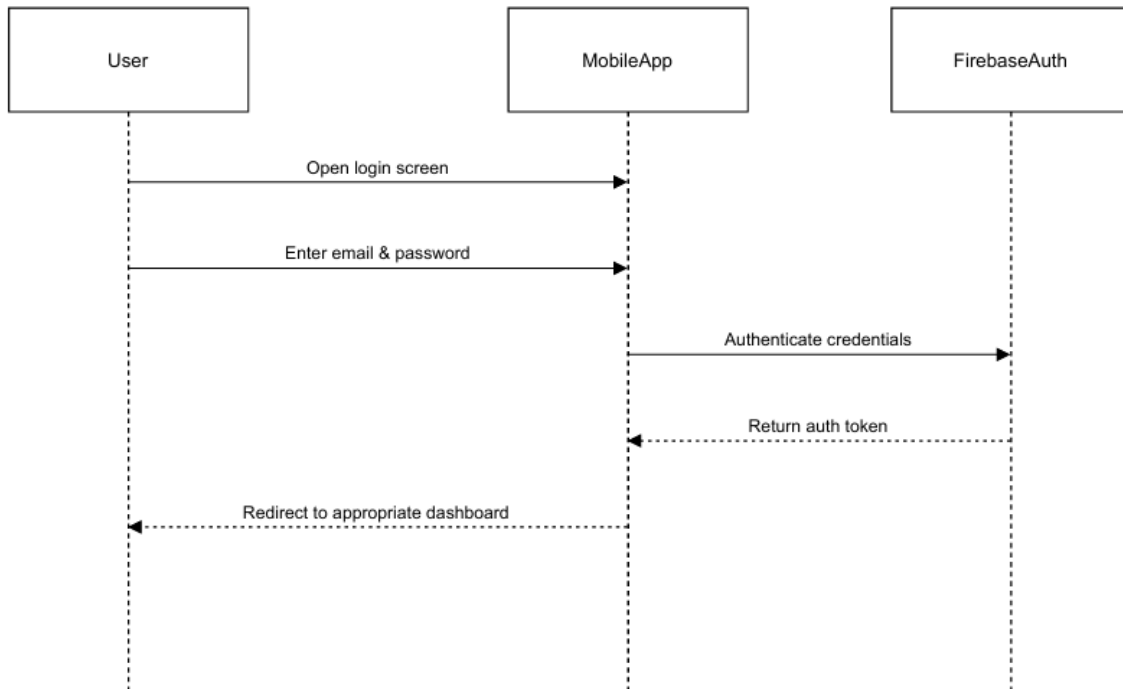


Figure 59 Sequence Diagram → User Login

○ User Registration Sequence Diagram.

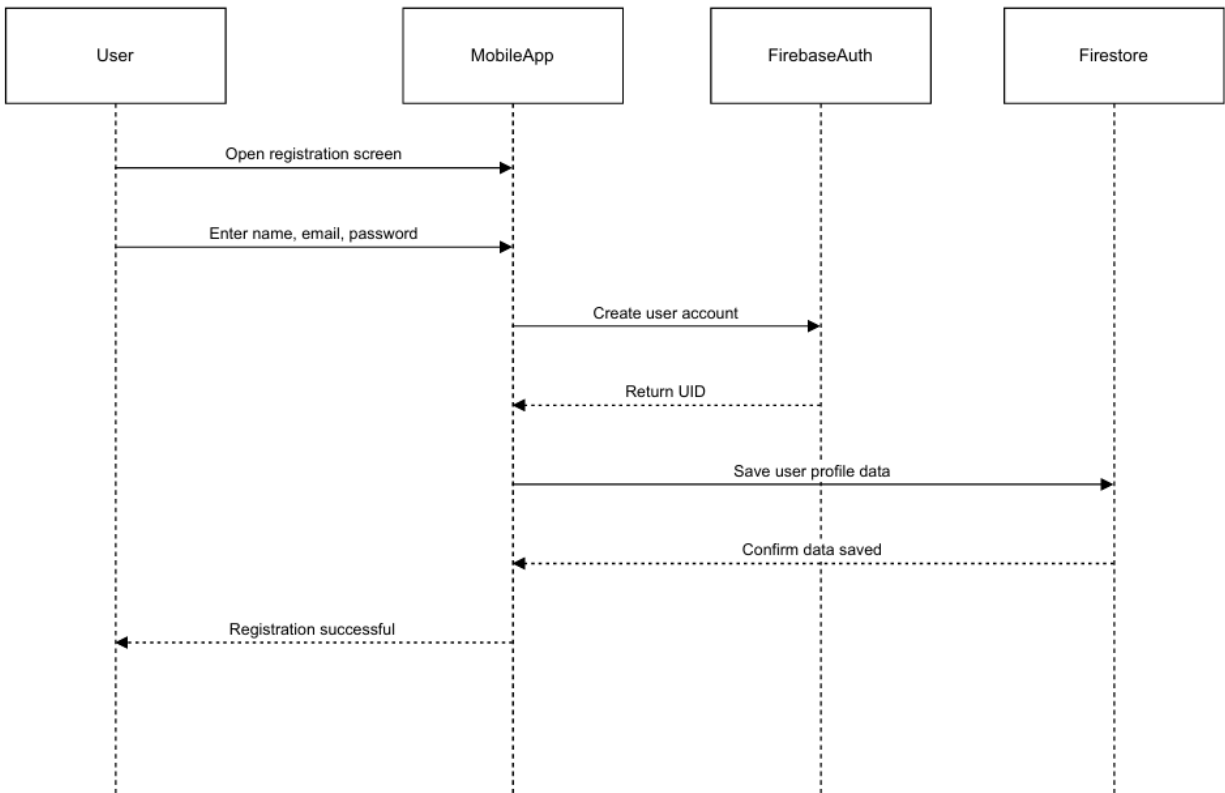


Figure 60 Sequence Diagram → User Registration.

4.7 Collaboration Diagram

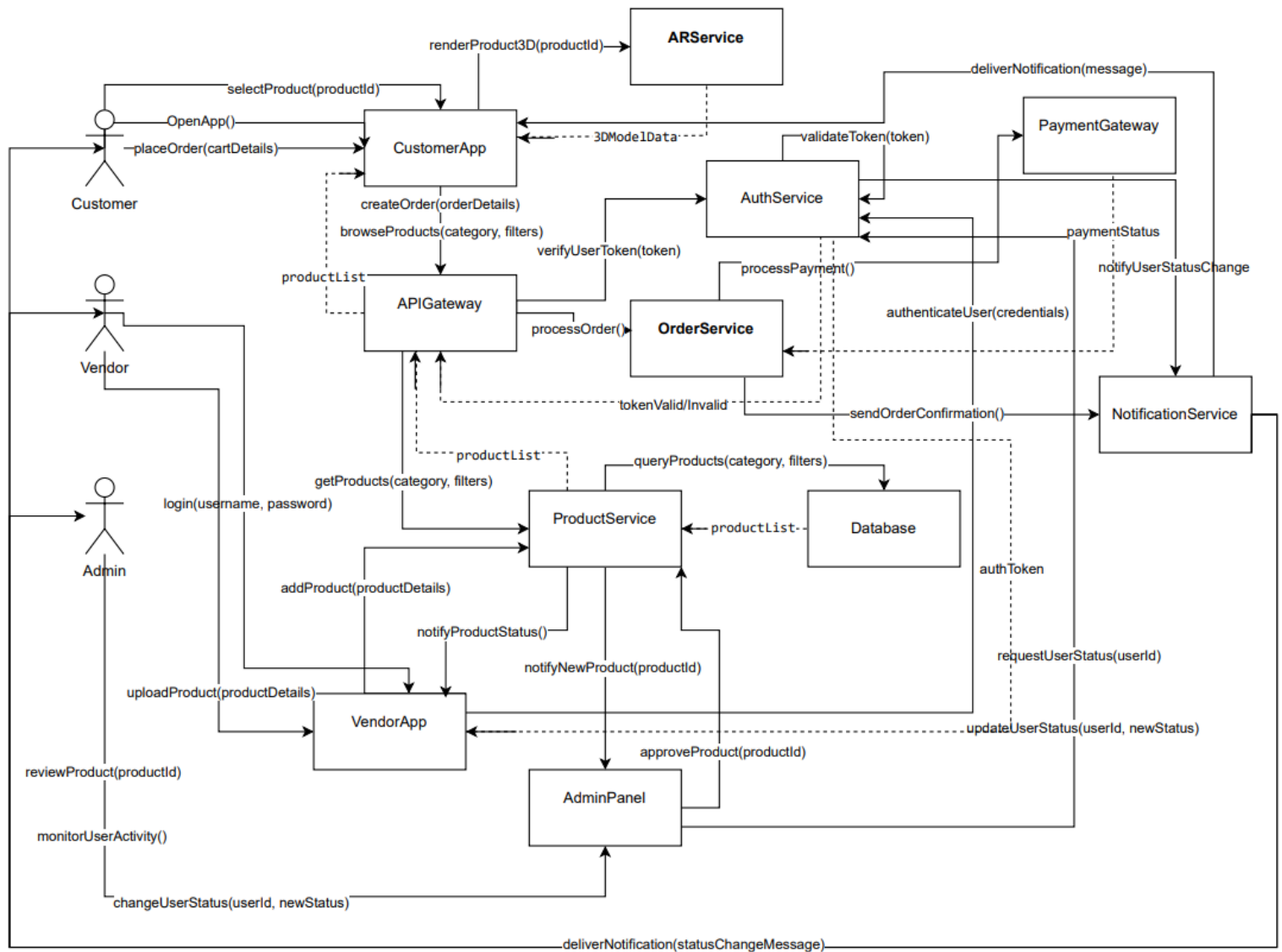


Figure 61 Collaboration Diagram

4.8 State Transition Diagram

A complete State Transition Diagram for the working flow of the app a for the customer is given below.

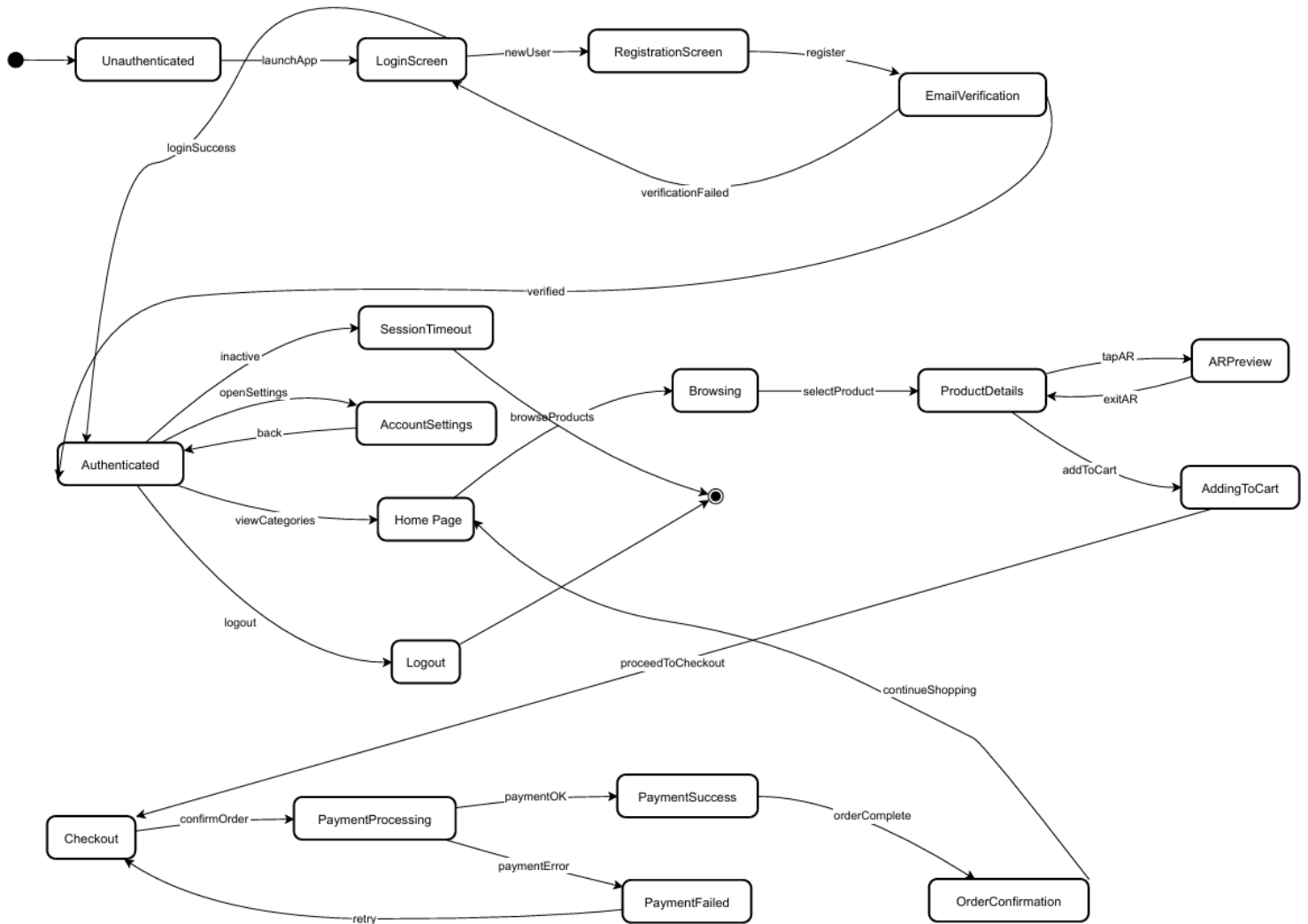


Figure 62 State Transition Diagram for complete app

4.9 Component Diagram

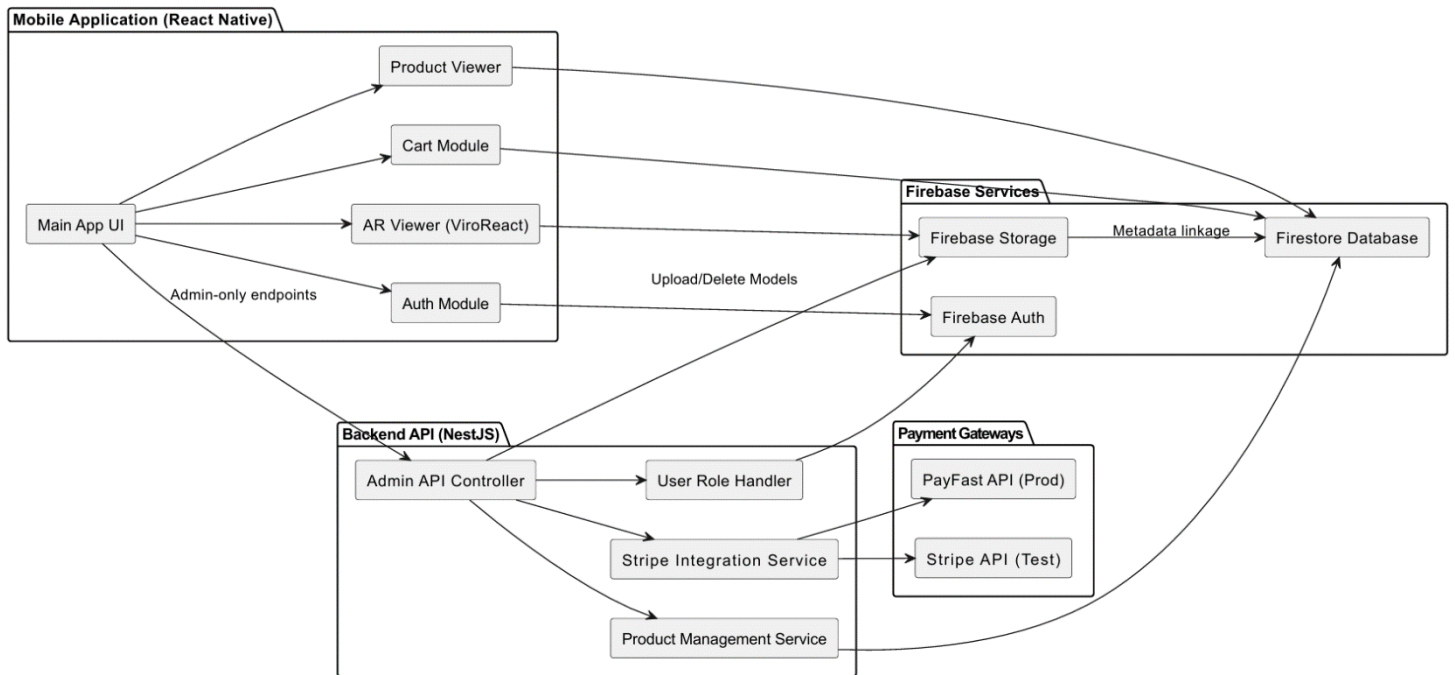


Figure 63 Component Diagram

4.10 Deployment Diagram

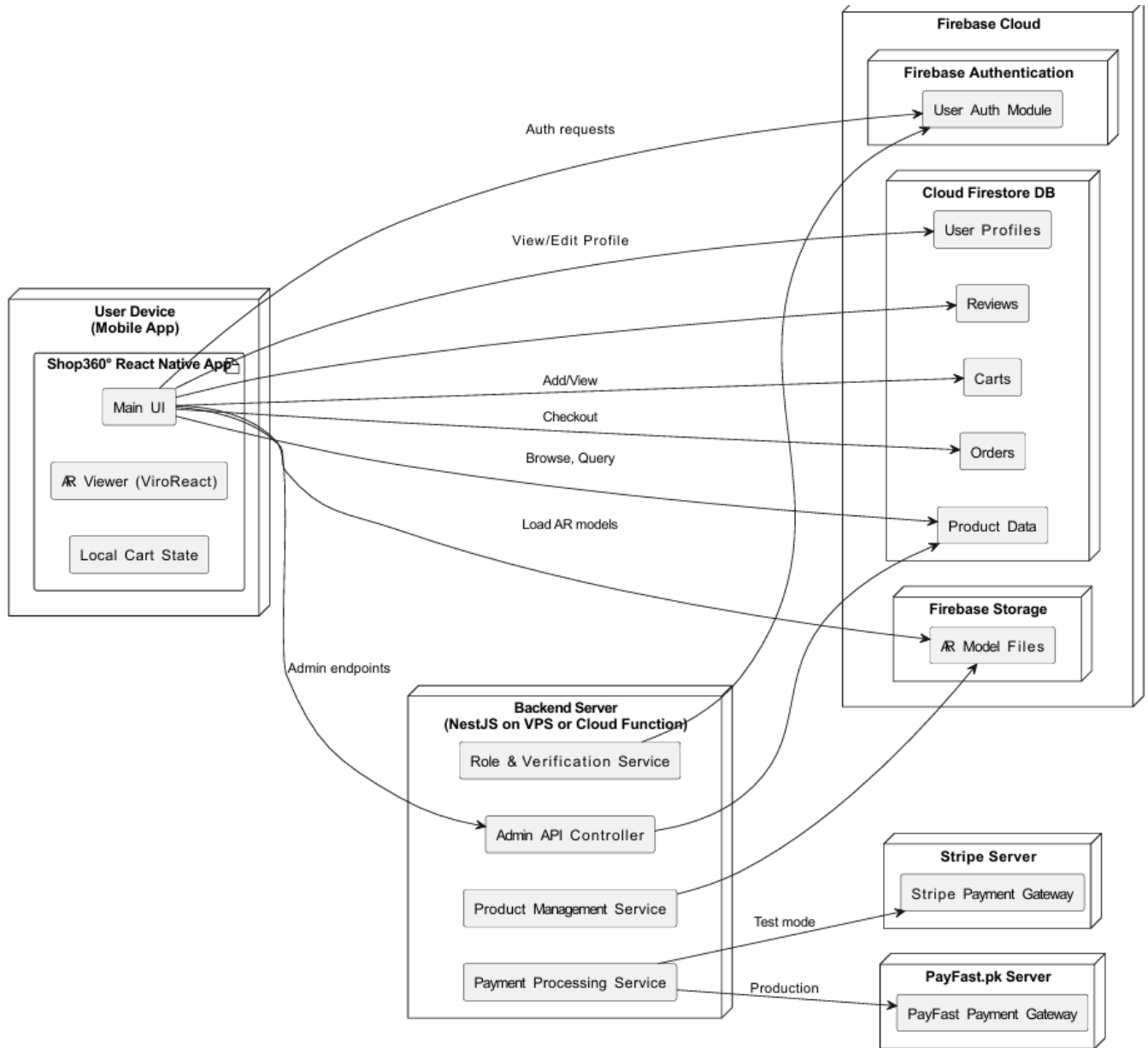


Figure 64 Deployment Diagram

Chapter 5: Testing

5.1 Test Case Specification

- Email / Password Sign Up

Positive Test Case	
ID	TC-FR1.1-POS-001
Priority	High
Description	Verify users can successfully sign up with valid email and password
Reference	FR1.1
Users	Guest User
Pre-Requisites	A. App is installed and launched B. User is on Login/Signup screen C. User has a valid email address not already
Steps	A. Navigate to Signup screen B. Enter valid email address (e.g., testuser@example.com) C. Enter valid password (minimum 6 characters) D. Enter name E. Tap "Sign Up" button
Input	Email: testuser@example.com, Password: Test123!, Name: Test User
Expected Results	User accounts are created, verification email is sent, user is redirected to email verification screen
Status	Tested, Passed

Table 34 Email / Password Sign Up +ve Test Case Specification

Negative Test Case	
ID	TC-FR1.1-NEG-001
Priority	High
Description	Verify System rejects sign up with invalid email format
Reference	FR1.1
Users	Guest User
Pre-Requisites	A. App is installed and launched B. Mobile device is connected to Internet. C. User is on SignUp Screen
Steps	A. Navigate to Signup screen B. Enter invalid email address (e.g., invalidemail) C. Enter valid password

	D. Enter name E. Tap "Sign Up" button
Input	Email: invalidemail, Password: Test123!, Name: Test User
Expected Results	Error message displayed indicating invalid email format, sign up fails
Status	Tested, Passed

Table 35 Email / Password Sign Up -ve Test Case Specification

- User Login / Logout

Positive Test Case	
ID	TC-FR1.2-POS-001
Priority	High
Description	Verify User can successfully sign in with valid credentials
Reference	FR1.2
Users	Registered User
Pre-Requisites	A. App is installed and launched B. User has registered account with verified email C. User is on Login Screen
Steps	A. Navigate to Login Screen B. Entered Registered email address C. Enter Correct Password D. Tap "Sign In" button
Input	Email: testuser@example.com, Password: Test123!
Expected Results	User is successfully signed in and redirected to Home screen
Status	Tested, Passed

Table 36 User Login / Logout +ve Test Case Specification

Negative Test Case	
ID	TC-FR1.2-NEG-001
Priority	High
Description	Verify system rejects sign in with incorrect password
Reference	FR1.2
Users	Registered User
Pre-Requisites	A. App is installed and launched B. User has registered account with verified email C. User is on Login Screen
Steps	A. Navigate to Login Screen

	B. Entered Registered email address C. Enter Incorrect Password D. Tap “Sign In” button
Input	Email: testuser@example.com, Password: WrongPass123!
Expected Results	Error message displayed indicating invalid credentials, sign in fails
Status	Tested, Passed

Table 37 User Login / Logout -ve Test Case Specification

- Password Reset

Positive Test Case	
ID	TC-FR1.5-POS-001
Priority	High
Description	Verify password reset email is sent to registered email
Reference	FR1.5
Users	Registered User
Pre-Requisites	A. App is installed and launched B. User is on Forgot Password Screen C. User has registered account
Steps	A. Navigate to Forgot Password Screen B. Enter registered email address C. Tap “Send Reset Link” button
Input	Email: testuser@example.com
Expected Results	Success message displayed, password reset email sent to user’s inbox
Status	Tested, Passed

Table 38 Password Reset +ve Test Case Specification

Negative Test Case	
ID	TC-FR1.5-NEG-001
Priority	Medium
Description	Verify system handles password reset request for non-existent email
Reference	FR1.5
Users	Guest User
Pre-Requisites	A. Navigate to Forgot Password Screen B. User is on Forgot Password Screen
Steps	A. Navigate to Forgot Password Screen B. Enter non-existent email address C. Tap “Send Reset Link”
Input	Email: nonexistent@example.com

Expected Results	Email not found message displayed.
Status	Tested, Passed

Table 39 Password Reset -ve Test Case Specification

- User Profile Management

Positive Test Case	
ID	TC-FR2.2-POS-001
Priority	Medium
Description	Verify user can update their name
Reference	FR2.2
Users	Authenticated User
Pre-Requisites	A. User is signed in with verified email B. User is on Personal Information screen
Steps	A. Navigate to Personal Information screen B. Tap on Name field C. Enter new name D. Tap "Save" button
Input	New Name: John Doe
Expected Results	Name is updated successfully; updated name is displayed in profile
Status	Tested, Passed

Table 40 User Profile +ve Test Case Specification

Negative Test Case	
ID	TC-FR2.2-NEG-001
Priority	Medium
Description	Verify system rejects empty name update
Reference	FR2.2
Users	Authenticated User
Pre-Requisites	A. User is signed in with verified email B. User is on Personal Information screen
Steps	A. Navigate to Personal Information screen B. Tap on Name field C. Clear name field (leave empty) D. Tap "Save" button
Input	Name: (empty)
Expected Results	Error message displayed indicating name cannot be empty, update fails
Status	Tested, Passed

Table 41 User Profile -ve Test Case Specification

- Role Based Access Control (Admin / Verified User / Guest)

Positive Test Case	
ID	TC-FR11.1-POS-001
Priority	High
Description	Verify admin can view dashboard statistics
Reference	FR11.1
Users	Admin User
Pre-Requisites	A. User is signed in as admin B. User is on Admin Dashboard screen
Steps	A. Navigate to Admin Dashboard
Input	None
Expected Results	Dashboard displays total users, total products, and total orders statistics.
Status	Tested, Passed

Table 42 Role Based Access Control +ve Test Case Specification

Negative Test Case	
ID	TC-FR11.1-NEG-001
Priority	High
Description	Verify regular user cannot access admin dashboard
Reference	FR11.1
Users	Regular User
Pre-Requisites	A. User is signed in as regular user (not admin)
Steps	A. Attempt to navigate to Admin Dashboard (if accessible via URL or deep link)
Input	None
Expected Results	Access denied message displayed, user cannot view admin dashboard
Status	Tested, Passed

Table 43 Role Based Access Control -ve Test Case Specification

- Browse Products Successfully

Positive Test Case	
ID	TC-FR3.1-POS-001
Priority	High
Description	Verify user can browse all available products
Reference	FR3.1
Users	Any User (Guest or Authenticated)
Pre-Requisites	A. App is installed and launched

	B. Products exist in database
Steps	A. Navigate to Products tab B. Scroll through product list
Input	None
Expected Results	Product list is displayed with product images, names, prices, and categories
Status	Tested, Passed

Table 44 Browse Products +ve Test Case Specification

- Product Search and Filter

Positive Test Case	
ID	TC-FR3.3-POS-001
Priority	Medium
Description	Verify user can search for products
Reference	FR3.3
Users	Any User (Guest or Authenticated)
Pre-Requisites	A. User is on Products Screen B. Products exist on database
Steps	A. Navigate to Products tab B. Tap on search bar C. Enter search term D. View search results
Input	Search Term: "laptop"
Expected Results	Products matching search term are displayed in results
Status	Tested, Passed

Table 45 Product Search and Filter +ve Test Case Specification

Negative Test Case	
ID	TC-FR3.3-NEG-001
Priority	Low
Description	Verify system handles search with no matching products
Reference	FR3.3
Users	Any User (Guest or Authenticated)
Pre-Requisites	A. User is on Products screen
Steps	A. Navigate to Products tab B. Tap on search bar C. Enter search term with no matches D. View search results

Input	Search Term: "xyzabc123nonexistent"
Expected Results	"No products found" message is displayed
Status	Tested, Passed

Table 46 Product Search and Filter -ve Test Case Specification

- AR Product Review
 - Access AR View

Positive Test Case	
ID	TC-FR10.1-POS-001
Priority	Medium
Description	Verify user can access AR view for product with 3D model
Reference	FR10.1
Users	Any User (Guest or Authenticated)
Pre-Requisites	A. User is on Product Details screen B. Product has modelUrl (3D model available)
Steps	A. Navigate to Product Details screen B. Tap "View in AR" button
Input	Product ID: prod123 (has modelUrl)
Expected Results	AR View screen opens, camera permission is requested if not granted
Status	Tested, Passed

Table 47 Access AR View +ve Test Case Specification

Negative Test Case	
ID	TC-FR10.1-NEG-001
Priority	Low
Description	Verify AR view button is not available for products without 3D model
Reference	FR10.1
Users	Any User (Guest or Authenticated)
Pre-Requisites	A. User is on Product Details screen B. Product does not have modelUrl
Steps	A. Navigate to Product Details screen B. Look for "View in AR" button
Input	Product ID: prod123 (no modelUrl)
Expected Results	"View in AR" button is not displayed
Status	Tested, Passed

Table 48 Access AR View -ve Test Case Specification

- Grant Camera Permission

Positive Test Case	
ID	TC-FR10.2-POS-001
Priority	High
Description	Verify user can grant camera permission for AR view
Reference	FR10.2
Users	Any User (Guest or Authenticated)
Pre-Requisites	A. User is on AR View screen B. Camera permission not granted
Steps	A. Navigate to AR View screen B. Tap "Allow Camera" button C. Grant permission in system dialog
Input	None
Expected Results	Camera permission is granted, AR view initializes, 3D model is displayed
Status	Tested, Passed

Table 49 Grant Camera Permission +ve Test Case Specification

Negative Test Case	
ID	TC-FR10.2-NEG-001
Priority	Medium
Description	Verify system handles denied camera permission gracefully
Reference	FR10.2
Users	Any User (Guest or Authenticated)
Pre-Requisites	A. User is on AR View screen B. Camera permission not granted
Steps	A. Navigate to AR View screen B. Tap "Allow Camera" button C. Deny permission in system dialog
Input	None
Expected Results	Error message displayed indicating camera permission required, user can retry or go back
Status	Tested, Passed

Table 50 Grant Camera Permission -ve Test Case Specification

- Product Management (Admin)

Positive Test Case	
ID	TC-FR12.2-POS-001
Priority	High
Description	Verify admin can create new product
Reference	FR12.2
Users	Admin User
Pre-Requisites	A. User is signed in as admin B. User is on Admin Products screen
Steps	A. Navigate to Admin Products screen B. Tap "Add Product" or "Create Product" button C. Fill in product details (title, description, price, category, stock, images, etc.) D. Tap "Save" button
Input	Title: "New Product", Price: 99.99, Category: "Electronics", Stock: 10, Description: "Product description"
Expected Results	Product is created successfully, appears in products list
Status	Tested, Passed

Table 51 Product Management +ve Test Case Specification

Negative Test Case	
ID	TC-FR12.2-NEG-001
Priority	High
Description	Verify system rejects product creation with missing required fields
Reference	FR12.2
Users	Admin User
Pre-Requisites	A. User is signed in as admin B. User is on Product Edit/Create screen
Steps	A. Navigate to Product Create screen B. Leave required fields empty (e.g., title, price) C. Tap "Save" button
Input	Title: (empty), Price: (empty), Category: "Electronics"
Expected Results	Error messages displayed for missing required fields, product not created
Status	Tested, Passed

Table 52 Product Management -ve Test Case Specification

- Edit Product

Positive Test Case	
ID	TC-FR12.3-POS-001
Priority	High
Description	Verify admin can edit existing product
Reference	FR12.3
Users	Admin User
Pre-Requisites	A. User is signed in as admin B. User is on Admin Products screen C. Products exist in database
Steps	A. Navigate to Admin Products screen B. Tap on a product C. Modify product details D. Tap "Save" button
Input	Product ID: prod123, Updated Price: 149.99
Expected Results	Product is updated successfully; changes are reflected in products list and product details
Status	Tested, Passed

Table 53 Edit Product +ve Test Case Specification

- Delete Product

Positive Test Case	
ID	TC-FR12.4-POS-001
Priority	High
Description	Verify admin can delete product
Reference	FR12.4
Users	Admin User
Pre-Requisites	A. User is signed in as admin B. User is on Admin Products screen C. Products exist in database
Steps	A. Navigate to Admin Products screen B. Tap delete button on a product C. Confirm deletion
Input	Product ID: prod123
Expected Results	Product is deleted successfully, removed from products list
Status	Tested, Passed

Table 54 Delete Product +ve Test Case Specification

5.2 Black Box Test Cases

All the Black Box Tests performed for this application are as follows: -

5.2.1 Equivalence Partitions (EP)

Equivalence class partitioning (EP) is a very widely used method to decrease the number of possible test cases that are required to test a system.

- User Registration / User Login / Logout / Password Reset
 - Valid Email Format

Partition	Valid email addresses with proper format
Description	Email addresses containing @ symbol, valid domain, and proper structure
Valid Examples	- `user@example.com` - `test.user@student.uol.edu.pk` - `user123@test-domain.com`
Expected Results	Email is accepted and processed successfully
Test Cases	Sign Up, Sign In, Change Email, Password Reset

Table 55 User Reg / Login / Logout / Reset Password EP Table

- Invalid Email Format (Missing @)

Partition	Email addresses without @ symbol
Description	Strings that look like emails but lack the @ symbol
Invalid Examples	- `userexample.com` - `test.userdomain.com` - `user123student.uol.edu.pk`
Expected Results	System rejects input with error message "Invalid email format"
Test Cases	Sign Up, Sign In, Change Email, Password Reset

Table 56 Invalid Email Format EP Table

- Invalid Email Format (Missing Domain)

Partition	Email addresses without domain part
Description	Email addresses that have @ but no domain after it
Invalid Examples	- `user@` - `test@.com`

	- `user@domain`
Expected Results	System rejects input with error message "Invalid email format"
Test Cases	Sign Up, Sign In, Change Email, Password Reset

Table 57 Invalid Email Format EP Table

○ Empty Email

Partition	Empty string or whitespace-only email
Description	No email address provided
Invalid Examples	- `""` (empty string) - `" "` (whitespace only)
Expected Results	System rejects input with error message "Email is required"
Test Cases	Sign Up, Sign In, Change Email, Password Reset

Table 58 Empty Email EP Table

○ Email Already in Use

Partition	Email address that already exists in the system
Description	Email address that is already registered
Invalid Examples	- `existing@example.com` (if already registered)
Expected Results	System rejects sign up with error message "Email already in use"
Test Cases	Sign Up

Table 59 Email Already is Use EP Table

○ Valid Password (Minimum Length)

Partition	Passwords with 6 or more characters
Description	Passwords meeting minimum length requirement
Valid Examples	- `Test123!` (8 characters) - `Pass12` (6 characters - minimum) - `VeryLongPassword123!@#` (20+ characters)
Expected Results	Password is accepted and processed successfully
Test Cases	Sign Up, Sign In, Change Password

Table 60 Valid Password EP Table

○ Invalid Password (Too Short)

Partition	Passwords with less than 6 characters
Description	Passwords that do not meet minimum length requirement
Invalid Examples	- `Test1` (5 characters) - `Pass` (4 characters) - `Ab1` (3 characters) - `A` (1 character)
Expected Results	System rejects input with error message "Password must be at least 6 characters"
Test Cases	Sign Up, Change Password

Table 61 Invalid Password EP Table

○ Empty Password

Partition	Empty string or whitespace-only password
Description	No password provided
Invalid Examples	- `""` (empty string) - `" "` (whitespace only)
Expected Results	System rejects input with error message "Password is required"
Test Cases	Sign Up, Sign In, Change Password

Table 62 Empty Password EP Table

○ Weak Password

Partition	Passwords that are too weak (Firebase validation)
Description	Passwords that Firebase considers weak
Invalid Examples	- `123456` (all numbers) - `abcdef` (all letters, no numbers) - `password` (common word)
Expected Results	System rejects input with error message "The password is too weak"
Test Cases	Sign Up, Change Password

Table 63 Weak Password EP Table

- Password Mismatch

Partition	Password and confirm password do not match
Description	When password and confirm password fields have different values
Invalid Examples	- Password: `Test123!`, Confirm: `Test1234!` - Password: `Pass123`, Confirm: `Pass456`
Expected Results	System rejects input with error message "Passwords do not match"
Test Cases	Sign Up, Change Password

Table 64 Password Mismatch EP Table

- Valid Name

Partition	Non-empty names with reasonable length
Description	Names that are not empty and within acceptable length
Valid Examples	- `John Doe` - `Mary Jane Watson` - `A` (minimum 1 character) - `Very Long Name That Is Still Acceptable` (up to reasonable limit)
Expected Results	Name is accepted and saved successfully
Test Cases	Sign Up, Update Name

Table 65 Valid Name EP Table

- Role Based Access Control (Admin / Verified User / Guest)

- Valid User Role (User)

Partition	Valid user role values
Description	Roles that are valid in the system
Valid Examples	- `user` (regular user) - `admin` (administrator)
Expected Results	User role is accepted and updated successfully
Test Cases	Update User Role

Table 66 Valid User Role EP Table

- Invalid User Role (Invalid Value)

Partition	User role values that are not valid
Description	Roles that are not in the allowed list
Invalid Examples	- `moderator` (if not in allowed list) - `guest` (if not in allowed list)

	- `invalid_role`
Expected Results	System rejects input with error message "Invalid role"
Test Cases	Update User Role

Table 67 Invalid User Role EP Table

- View Product Details
 - Valid Product Title

Partition	Valid product titles
Description	Non-empty product titles
Valid Examples	- `Samsung Galaxy S21 Ultra` - `Google Pixel 7 Pro` - `A` (minimum 1 character)
Expected Results	Product title is accepted and saved successfully
Test Cases	Create Product, Edit Product

Table 68 Valid Product Title EP Table

- Invalid Product Title

Partition	Empty product title field
Description	No product title provided
Invalid Examples	- `""` (empty string) - `" "` (whitespace only)
Expected Results	System rejects input with error message "Product title is required"
Test Cases	Create Product, Edit Product

Table 69 Invalid Product Title EP Table

- AR Product Preview
 - Camera Permission Granted

Partition	Camera permission has been granted by user
Description	User has allowed camera access for AR functionality
Valid Examples	- Permission status: `granted`
Expected Results	AR view can initialize, camera feed is displayed, AR tracking begins (ARCore for Android)
Test Cases	Access AR View, AR Product Preview

Table 70 Camera Permission Granted EP Table

○ Camera Permission Denied

Partition	Camera permission has been denied by user
Description	User has denied camera access
Invalid Examples	- Permission status: `denied` - Permission status: `blocked`
Expected Results	AR view cannot initialize, error message displayed: "Camera permission is required for AR view", user is redirected back
Test Cases	Access AR View, AR Product Preview

Table 71 Camera Permission Denied EP Table

○ Camera Permission Not Requested

Partition	Camera permission has not been requested yet
Description	Permission request dialog has not been shown
Valid Examples	- Permission status: `unknown` - Permission status: `not-determined`
Expected Results	System requests camera permission, permission dialog is displayed
Test Cases	Access AR View, AR Product Preview

Table 72 Camera Permission Not Requested EP Table

○ AR Tracking Ready

Partition	AR tracking is ready and working properly
Description	AR system has successfully initialized and is tracking the environment
Valid Examples	- Tracking state: `ready` - Tracking state: `normal`
Expected Results	AR tracking is active, plane detection works, model placement is enabled
Test Cases	AR View, AR Model Placement

Table 73 AR Tracking Ready EP Table

○ AR Tracking Limited

Partition	AR tracking is limited due to poor conditions
Description	AR tracking quality is degraded due to environmental factors
Valid Examples	- Tracking state: `limited` - Tracking reason: `insufficient-features`, `excessive-motion`, `insufficient-light`
Expected Results	Warning message displayed, plane detection may be less accurate, model placement may be disabled until tracking improves
Test Cases	AR View, AR Model Placement

Table 74 AR Tracking Limited EP Table

○ AR Tracking Unavailable

Partition	AR tracking is not available
Description	AR tracking cannot initialize or has failed
Invalid Examples	- Tracking state: `unavailable` - Tracking state: `not-available` - Tracking state: `error`
Expected Results	Error message displayed, AR view cannot function, user is notified of the issue
Test Cases	AR View, AR Model Placement

Table 75 AR Tracking Unavailable EP Table

○ AR Tracking Unknown

Partition	AR tracking state is unknown or initializing
Description	AR tracking is still initializing or state is not yet determined
Valid Examples	- Tracking state: `unknown` - Tracking state: `initializing`
Expected Results	System waits for tracking to initialize, loading indicator may be shown
Test Cases	AR View Initialization

Table 76 AR Tracking Unknown EP Table

○ Model Loaded Successfully

Partition	3D model has been successfully loaded
Description	Model file downloaded and loaded without errors
Valid Examples	- Model state: `loaded` - Model state: `ready` - Loading progress: `100%`
Expected Results	Model is ready for placement, "Place Model" button is enabled, model can be placed in AR space
Test Cases	AR Model Loading, AR Model Placement

Table 77 Model Loaded Successfully EP Table

○ Model Loading in Progress

Partition	3D model is currently being loaded
Description	Model file is being downloaded or processed
Valid Examples	- Model state: `loading` - Loading progress: `0-99%`

Expected Results	Loading indicator is displayed, progress is shown, "Place Model" button is disabled
Test Cases	AR Model Loading

Table 78 Model Loading in Progress EP Table

○ Model Loading Error

Partition	Error occurred while loading 3D model
Description	Model failed to load due to network error, invalid URL, or corrupted file
Invalid Examples	- Model state: `error` - Error: `network-error`, `invalid-url`, `timeout`, `corrupted-file`
Expected Results	Error message displayed, retry button shown, model cannot be placed
Test Cases	AR Model Loading

Table 79 Model Loading Error EP Table

○ Model Not Loaded

Partition	3D model has not been loaded yet
Description	Model loading has not started or product has no model
Valid Examples	- Model state: `not-loaded` - Model URL: `empty` or `null`
Expected Results	System attempts to load model if URL exists, or shows message if no model available
Test Cases	AR Model Loading

Table 80 Model Not Loaded EP Table

○ Plane Detected

Partition	AR system has detected a valid plane/surface
Description	A suitable surface has been identified for model placement
Valid Examples	- Plane state: `detected` - Reticle: `visible` and `green` (ready)
Expected Results	Reticle appears on detected surface, "Place Model" button becomes enabled, user can place model
Test Cases	AR Plane Detection, AR Model Placement

Table 81 Plane Detected EP Table

○ Plane Detecting

Partition	AR system is actively detecting planes
Description	System is analyzing camera feed to find surfaces
Valid Examples	- Plane state: `detecting` - Reticle: `visible` and `white` (searching)
Expected Results	Reticle appears when potential surface is detected, user is prompted to move camera slowly
Test Cases	AR Plane Detection

Table 82 Plane Detecting EP Table

○ Plane Not Detected

Partition	No plane/surface has been detected yet
Description	System has not found a suitable surface for placement
Invalid Examples	- Plane state: `not-detected` - Reticle: `not-visible`
Expected Results	Reticle is not shown, "Place Model" button is disabled, user is prompted to scan environment
Test Cases	AR Plane Detection

Table 83 Plane Not Detected EP Table

○ Plane Locked

Partition	Plane detection is locked after model placement
Description	Plane detection has been locked to prevent jitter after model is placed
Valid Examples	- Plane state: `locked` - Plane locked: `true`
Expected Results	Plane detection stops updating, model position is fixed, user can interact with placed model
Test Cases	AR Model Placement, AR Model Controls

Table 84 Plane Locked EP Table

○ Valid Model Scale (Within Range)

Partition	Model scale values within valid range (0.1 to 10)
Description	Scale multiplier values that are within acceptable bounds
Valid Examples	- Scale: `0.1` (minimum) - Scale: `1.0` (default/normal)

	- Scale: `5.0` (medium) - Scale: `10.0` (maximum)
Expected Results	Model scale is applied successfully, model size changes accordingly
Test Cases	AR Model Controls, AR Model Scaling

Table 85 Valid Model Scale EP Table

- Invalid Model Scale (Too Small)

Partition	Model scale values below minimum (less than 0.1)
Description	Scale values that are too small
Invalid Examples	- Scale: `0.05` - Scale: `0.0` - Scale: `-0.1`
Expected Results	System clamps scale to minimum (0.1) or rejects input
Test Cases	AR Model Controls, AR Model Scaling

Table 86 Invalid Model Scale EP Table

- Invalid Model Scale (Too Large)

Partition	Model scale values above maximum (greater than 10)
Description	Scale values that are too large
Invalid Examples	- Scale: `11.0` - Scale: `20.0` - Scale: `100.0`
Expected Results	System clamps scale to maximum (10) or rejects input
Test Cases	AR Model Controls, AR Model Scaling

Table 87 Invalid Model Scale EP Table

- Product Management (Admin)

- Valid Price (Positive)

Partition	Positive price values
Description	Prices that are greater than zero
Valid Examples	- Price: `10.99` - Price: `0.01` (minimum) - Price: `9999.99` (large value) - Price: `100` (integer)
Expected Results	Price is accepted and product is saved successfully
Test Cases	Create Product, Edit Product

Table 88 Valid Price (Positive) EP Table

○ Invalid Price (Zero)

Partition	Price of zero
Description	Zero price is not allowed for products
Invalid Examples	- Price: `0` - Price: `0.00`
Expected Results	System rejects input with error message "Price must be greater than zero"
Test Cases	Create Product, Edit Product

Table 89 Invalid Price (Zero) EP Table

○ Invalid Price (Negative)

Partition	Negative price values
Description	Negative prices are not valid
Invalid Examples	- Price: `-10.99` - Price: `-0.01` - Price: `-100`
Expected Results	System rejects input with error message "Price cannot be negative"
Test Cases	Create Product, Edit Product

Table 90 Invalid Price (Negative) EP Table

○ Invalid Price (Empty)

Partition	Empty price field
Description	No price provided
Invalid Examples	- Price: `""` (empty string) - Price: `" "` (whitespace only)
Expected Results	System rejects input with error message "Price is required"
Test Cases	Create Product, Edit Product

Table 91 Invalid Price (Empty) EP Table

5.2.2 Boundary Value Analysis

Boundary Value Analysis for the application are as follows: -

- User Registration / Login / Password Reset
 - Password Minimum Boundary (Inside)

Boundary	Minimum valid password length = 6 characters
Test Value	`Pass12` (exactly 6 characters)
Description	Testing password with exactly the minimum required length
Expected Results	Password is accepted and processed successfully
Test Cases	Sign Up, Change Password
Priority	High

Table 92 User Reg / Login / Password Reset BVA Table

- Password Minimum Boundary – 1 (Outside)

Boundary	Just below minimum = 5 characters
Test Value	`Test1` (5 characters)
Description	Testing password with one character less than minimum
Expected Results	System rejects input with error message "Password must be at least 6 characters"
Test Cases	Sign Up, Change Password
Priority	High

Table 93 Product Minimum Boundary - 1(Outside) BVA Table

- Password Minimum Boundary + 1 (Inside)

Boundary	Just above minimum = 7 characters
Test Value	`Test123` (7 characters)
Description	Testing password with one character more than minimum
Expected Results	Password is accepted and processed successfully
Test Cases	Sign Up, Change Password
Priority	Medium

Table 94 Password Minimum Boundary + 1 (Inside) BVA Table

- Password Minimum Empty (Absolute Minimum – Outside)

Boundary	Empty string = 0 characters
Test Value	'' (empty string)
Description	Testing password with no characters
Expected Results	System rejects input with error message "Password is required"
Test Cases	Sign Up, Sign In, Change Password
Priority	High

Table 95 Password Minimum Empty (Absolute Minimum – Outside) BVA Table

- User Name Length Boundaries
 - User Name Minimum Boundary (Inside)

Boundary	Minimum valid length = 1 character
Test Value	'A' (exactly 1 character)
Description	Testing user name with exactly the minimum length
Expected Results	Name is accepted and saved successfully
Test Cases	Sign Up, Update Name
Priority	High

Table 96 User Name Minimum Boundary (Inside) BVA Table

- User Name Minimum Boundary – 1 (Outside)

Boundary	Just below minimum = 0 characters (empty)
Test Value	'' (empty string)
Description	Testing user name with no characters
Expected Results	System rejects input with error message "Name is required"
Test Cases	Sign Up, Update Name
Priority	High

Table 97 User Name Minimum Boundary – 1 (Outside) BVA Table

- User Name Minimum Boundary + 1 (Inside)

Boundary	Just above minimum = 2 characters
Test Value	'AB' (2 characters)
Description	Testing user name with one character more than minimum
Expected Results	Name is accepted and saved successfully
Test Cases	Sign Up, Update Name
Priority	Medium

Table 98 User Name Minimum Boundary + 1 (Inside) BVA Table

- User Name Typical Value (Inside)

Boundary	Typical name length
Test Value	'John Doe' (8 characters)
Description	Testing user name with a typical value
Expected Results	Name is accepted and saved successfully
Test Cases	Sign Up, Update Name
Priority	Low

Table 99 User Name Typical Value (Inside) BVA Table

- Product Title Length Boundaries

- Product Title Minimum Boundary (Inside)

Boundary	Minimum valid length = 1 character
Test Value	'A' (exactly 1 character)
Description	Testing product title with exactly the minimum length
Expected Results	Product title is accepted and saved successfully
Test Cases	Create Product, Edit Product
Priority	High

Table 100 Product Title Minimum Boundary (Inside) BVA Table

- Product Title Minimum Boundary – 1 (Outside)

Boundary	Just below minimum = 0 characters (empty)
Test Value	'' (empty string)
Description	Testing product title with no characters
Expected Results	System rejects input with error message "Product title is required"
Test Cases	Create Product, Edit Product
Priority	High

Table 101 Product Title Minimum Boundary – 1 (Outside) BVA Table

- Product Title Minimum Boundary + 1 (Inside)

Boundary	Just above minimum = 2 characters
Test Value	'AB' (2 characters)
Description	Testing product title with one character more than minimum
Expected Results	Product title is accepted and saved successfully
Test Cases	Create Product, Edit Product

Priority	Medium
----------	--------

Table 102 Product Title Minimum Boundary + 1 (Inside) BVA Table

- Product Title Typical Value (Inside)

Boundary	Typical title length
Test Value	`Samsung Galaxy S21 Ultra` (23 characters)
Description	Testing product title with a typical value
Expected Results	Product title is accepted and saved successfully
Test Cases	Create Product, Edit Product
Priority	Low

Table 103 Product Title Typical Value (Inside) BVA Table

- AR Product Preview
 - Model Scale Minimum Boundary (Inside)

Boundary	Minimum valid scale = 0.1
Test Value	Scale = `0.1` (exactly minimum)
Description	Testing model scale with exactly the minimum value
Expected Results	Model scale is applied successfully, model appears at smallest size
Test Cases	AR Model Controls, AR Model Scaling
Priority	High

Table 104 Model Scale Minimum Boundary (Inside) BVA Table

- Model Scale Minimum Boundary – 0.01 (Outside)

Boundary	Just below minimum = 0.09
Test Value	Scale = `0.09`
Description	Testing model scale with value just below minimum
Expected Results	System clamps scale to minimum (0.1) or rejects input
Test Cases	AR Model Controls, AR Model Scaling
Priority	High

Table 105 Model Scale Minimum Boundary – 0.01 (Outside) BVA Table

○ Model Scale Minimum Boundary (Inside)

Boundary	Maximum valid scale = 10.0
Test Value	Scale = `10.0` (exactly maximum)
Description	Testing model scale with exactly the maximum value
Expected Results	Model scale is applied successfully, model appears at largest size
Test Cases	AR Model Controls, AR Model Scaling
Priority	High

Table 106 Model Scale Minimum Boundary (Inside) BVA Table

○ Model Scale Minimum Boundary + 0.1 (Outside)

Boundary	Just above maximum = 10.1
Test Value	Scale = `10.1`
Description	Testing model scale with value just above maximum
Expected Results	System clamps scale to maximum (10.0) or rejects input
Test Cases	AR Model Controls, AR Model Scaling
Priority	High

Table 107 Model Scale Minimum Boundary + 0.1 (Outside) BVA Table

○ Model Scale Maximum Boundary – 0.1 (Inside)

Boundary	Just below maximum = 9.9
Test Value	Scale = `9.9`
Description	Testing model scale with value just below maximum
Expected Results	Model scale is applied successfully
Test Cases	AR Model Controls, AR Model Scaling
Priority	Medium

Table 108 Model Scale Maximum Boundary – 0.1 (Inside) BVA Table

○ Model Scale Typical Value (Inside)

Boundary	Middle of valid range
Test Value	Scale = `1.0` (default/normal)
Description	Testing model scale with typical default value
Expected Results	Model scale is applied successfully, model appears at normal size
Test Cases	AR Model Controls, AR Model Scaling
Priority	Low

Table 109 Model Scale Typical Value (Inside) BVA Table

○ AR Tracking Quality Ready (Inside)

Boundary	Tracking quality = ready/optimal
Test Value	Tracking state = `ready`, Reason = `none`
Description	Testing AR tracking at optimal quality
Expected Results	AR tracking works perfectly, plane detection is accurate, model placement enabled
Test Cases	AR Tracking, AR Model Placement
Priority	High

Table 110 AR Tracking Quality Ready (Inside) BVA Table

○ AR Tracking Quality Limited – Insufficient Features (Inside)

Boundary	Tracking quality = limited due to insufficient features
Test Value	Tracking state = `limited`, Reason = `insufficient-features`
Description	Testing AR tracking with limited quality due to featureless environment
Expected Results	Warning message displayed, plane detection less accurate, model placement may be disabled
Test Cases	AR Tracking, AR Model Placement
Priority	High

Table 111 AR Tracking Quality Limited – Insufficient Features (Inside) BVA Table

○ AR Tracking Quality Limited – Excessive Motion (Inside)

Boundary	Tracking quality = limited due to excessive motion
Test Value	Tracking state = `limited`, Reason = `excessive-motion`
Description	Testing AR tracking with limited quality due to device movement
Expected Results	Warning message displayed, user prompted to slow down, tracking may be unstable
Test Cases	AR Tracking, AR Model Placement
Priority	High

Table 112 AR Tracking Quality Limited – Excessive Motion (Inside) BVA Table

○ AR Tracking Quality Ready Limited – Insufficient Light (Inside)

Boundary	Tracking quality = limited due to insufficient light
Test Value	Tracking state = `limited`, Reason = `insufficient-light`
Description	Testing AR tracking with limited quality due to poor lighting
Expected Results	Warning message displayed, user prompted to improve lighting, tracking may be less accurate
Test Cases	AR Tracking, AR Model Placement
Priority	High

Table 113 AR Tracking Quality Ready Limited – Insufficient Light (Inside) BVA Table

○ AR Tracking Quality Unavailable (Outside)

Boundary	Tracking quality = unavailable
Test Value	Tracking state = `unavailable`, Reason = `error` or `not-available`
Description	Testing AR tracking when it's completely unavailable
Expected Results	Error message displayed, AR view cannot function, user is notified
Test Cases	AR Tracking, AR Model Placement
Priority	High

Table 114 AR Tracking Quality Unavailable (Outside) BVA Table

5.2.3 Decision Table Testing

Decision Table Testing is a black-box testing technique that uses a table format to represent different combinations of conditions and their corresponding actions/outcomes. This method ensures that each possible branch from each decision point is executed at least once, thereby ensuring that all reachable code is executed.

Format:

- Y → Yes / True / Condition is met
- N → No / False / Condition is not met
- – → Not Applicable
- X → Action is executed / Outcome occurs

- **User Sign Up Decision Table**

Tests all possible combinations of conditions when a user attempts to sign up with email and password.

Conditions:

- C1: Email format is valid
- C2: Email is not already registered
- C3: Password meets minimum length (≥ 6 characters)
- C4: Password is strong (Firebase validation)
- C5: Password and confirm password match
- C6: Name field is not empty

Actions:

- A1: Create user account
- A2: Send verification email
- A3: Display error: "Invalid email format"
- A4: Display error: "Email already in use"
- A5: Display error: "Password must be at least 6 characters"
- A6: Display error: "Password is too weak"
- A7: Display error: "Passwords do not match"
- A8: Display error: "Name is required"

Test Cases

- DT1-R1: Invalid email format → Error: "Invalid email format"

- DT1-R2: Valid email but already registered → Error: "Email already in use"
- DT1-R3: Valid email, password too short → Error: "Password must be at least 6 characters"
- DT1-R4: Valid email, password too weak → Error: "Password is too weak"
- DT1-R5: Valid email, passwords don't match → Error: "Passwords do not match"
- DT1-R6: Valid email, empty name → Error: "Name is required"
- DT1-R7: All conditions valid → Account created, verification email sent

Rule	C1	C2	C3	C4	C5	C6	A1	A2	A3	A4	A5	A6	A7	A8
R1	N	-	-	-	-	-			X					
R2	Y	N	-	-	-	-				X				
R3	Y	Y	N	-	-	-					X			
R4	Y	Y	Y	N	-	-						X		
R5	Y	Y	Y	Y	N	-							X	
R6	Y	Y	Y	Y	Y	N								X
R7	Y	Y	Y	Y	Y	Y	X	X						

Table 115 User Sign Up Decision Table

- User Sign In Decision Table

Conditions:

- C1: Email format is valid
- C2: Email is registered in system
- C3: Password is correct
- C4: Email is verified
- C5: Account is not disabled

Actions:

- A1: Sign in successful
- A2: Navigate to home screen
- A3: Display error: "Invalid email format"
- A4: Display error: "User not found"
- A5: Display error: "Wrong password"
- A6: Display error: "Please verify your email"
- A7: Display error: "Account is disabled"

Test Cases

- DT2-R1: Invalid email format → Error: "Invalid email format"
- DT2-R2: Email not registered → Error: "User not found"
- DT2-R3: Wrong password → Error: "Wrong password"
- DT2-R4: Email not verified → Error: "Please verify your email"

- DT2-R5: Account disabled → Error: "Account is disabled"
- DT2-R6: All conditions valid → Sign in successful, navigate to home

Rule	C1	C2	C3	C4	C5	A1	A2	A3	A4	A5	A6	A7
R1	N	-	-	-	-			X				
R2	Y	N	-	-	-				X			
R3	Y	Y	N	-	-					X		
R4	Y	Y	Y	N	Y						X	
R5	Y	Y	Y	Y	N							X
R6	Y	Y	Y	Y	Y	X	X					

Table 116 User Sign In Decision Table

• Order Cancellation Request Decision Table

Conditions:

- C1: User is authenticated
- C2: Order belongs to user
- C3: Order status is "processing" or "pending"
- C4: Order has not been shipped
- C5: Cancellation request has not been submitted before

Actions:

- A1: Submit cancellation request successfully
- A2: Update order status to "cancellation_requested"
- A3: Send notification to admin
- A4: Display error: "Please sign in"
- A5: Display error: "Order not found"
- A6: Display error: "Order cannot be cancelled"
- A7: Display error: "Order has already been shipped"
- A8: Display error: "Cancellation request already submitted"

Test Cases:

- DT5-R1: User not authenticated → Error: "Please sign in"
- DT5-R2: Order doesn't belong to user → Error: "Order not found"
- DT5-R3: Order status not cancellable → Error: "Order cannot be cancelled"
- DT5-R4: Order already shipped → Error: "Order has already been shipped"
- DT5-R5: Request already submitted → Error: "Cancellation request already submitted"
- DT5-R6: All conditions valid → Request submitted, status updated, admin notified

Rule	C1	C2	C3	C4	C5	A1	A2	A3	A4	A5	A6	A7	A8
R1	N	-	-	-	-				X				
R2	Y	N	-	-	-					X			
R3	Y	Y	N	-	-						X		
R4	Y	Y	Y	N	-							X	
R5	Y	Y	Y	Y	N								X
R6	Y	Y	Y	Y	Y	X	X	X					

Table 117 Order Cancellation Request Decision Table

• Admin Dashboard Access Decision Table

Conditions:

- C1: User is authenticated
- C2: User has admin role
- C3: User email is verified
- C4: Admin account is active

Actions:

- A1: Display admin dashboard
- A2: Load dashboard statistics
- A3: Display error: "Please sign in"
- A4: Display error: "Admin access required"
- A5: Display error: "Please verify your email"
- A6: Display error: "Admin account is disabled"

Test Cases:

- DT8-R1: User not authenticated → Error: "Please sign in"
- DT8-R2: User not admin → Error: "Admin access required"
- DT8-R3: Email not verified → Error: "Please verify your email"
- DT8-R4: Admin account disabled → Error: "Admin account is disabled"
- DT8-R5: All conditions valid → Admin dashboard displayed, statistics loaded.

Rule	C1	C2	C3	C4	A1	A2	A3	A4	A5	A6
R1	N	-	-	-			X			
R2	Y	N	-	-				X		
R3	Y	Y	N	-					X	
R4	Y	Y	Y	N						X
R5	Y	Y	Y	Y	X	X				

Table 118 Admin Dashboard Access Decision Table

- **View AR Access Decision Table**

Conditions:

- C1: User is authenticated
- C2: Product has 3D model available
- C3: Camera permission is granted
- C4: AR tracking is ready
- C5: Device supports AR (ARCore for Android)

Actions:

- A1: Initialize AR view successfully
- A2: Load 3D model
- A3: Enable plane detection
- A4: Display error: "Please sign in to view AR"
- A5: Display error: "3D model not available for this product"
- A6: Display error: "Camera permission is required"
- A7: Display error: "AR tracking is not ready"
- A8: Display error: "AR is not supported on this device"

Test Cases:

- DT7-R1: User not authenticated → Error: "Please sign in to view AR"
- DT7-R2: No 3D model → Error: "3D model not available for this product"
- DT7-R3: Camera permission denied → Error: "Camera permission is required"
- DT7-R4: AR tracking not ready → Error: "AR tracking is not ready"
- DT7-R5: Device doesn't support AR → Error: "AR is not supported on this device"
- DT7-R6: All conditions valid → AR view initialized, model loaded, plane detection enabled

Rule	C1	C2	C3	C4	C5	A1	A2	A3	A4	A5	A6	A7	A8
R1	N	-	-	-	-				X				
R2	Y	N	-	-	-					X			
R3	Y	Y	N	-	-						X		
R4	Y	Y	Y	N	-							X	
R5	Y	Y	Y	Y	N								X
R6	Y	Y	Y	Y	Y	X	X	X					

Table 119 View AR Access Decision Table

5.2.4 State Transition Testing

As we know, State Transition testing, a black box testing technique, in which outputs are triggered by changes to the input conditions or changes to 'state' of the system. In other words, tests are designed to execute valid and invalid state transitions. Following are the State Transition tests done for the app:

- **Valid Transition – Signed In (unverified) to Signed In (Verified)**

Steps:

- User is in Signed In (Unverified) state
- User receives verification email
- User clicks verification link
- User returns to app and checks verification

Expected Results:

- User transitions to Signed In (Verified) state
- User can now access authenticated features
- User is redirected to Avatar Onboarding if no avatar selected

Diagram:

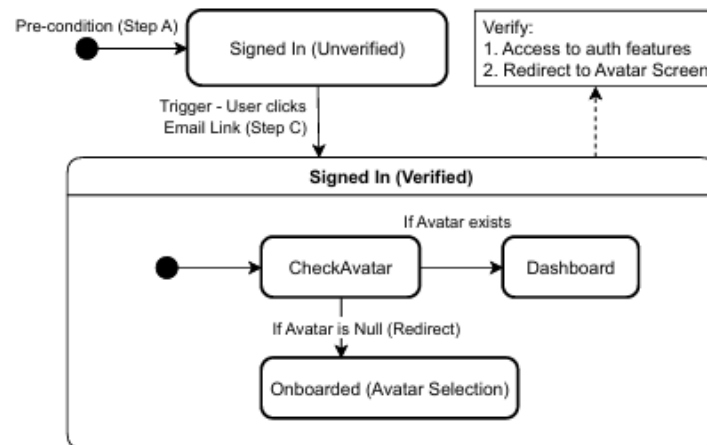


Figure 65 Valid Transition – Signed In (unverified) to Signed In (Verified) ST

- **User Authentication State Diagram**

Steps:

- User is in Signed In (Unverified) state
- User receives verification email
- User clicks verification link
- User returns to app and checks verification

Expected Results:

- User transitions to Signed In (Verified) state
- User can now access authenticated features
- User is redirected to Avatar Onboarding if no avatar selected

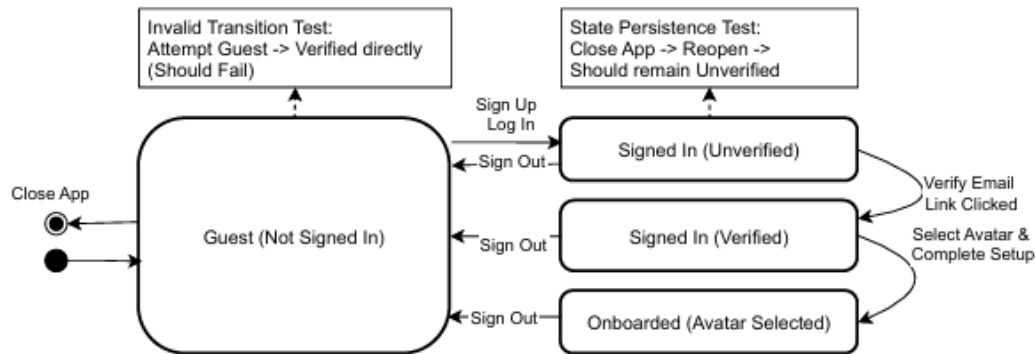


Figure 66 User Authentication State Diagram ST

- **Order Status State Diagram**

Steps:

- A. Order is in processing state
- B. Admin navigates to Order Details Screen.
- C. Admin selects "Shipped" status
- D. Admin confirms status update

Expected Results:

- Order transitions to shipped state
- Timeline entry is added with "shipped" status
- User can see updated status in real-time

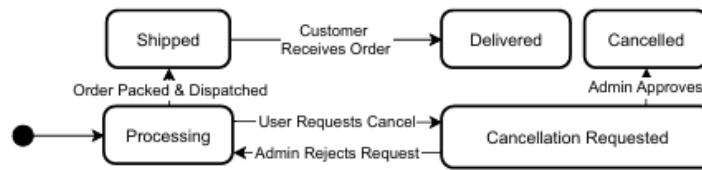


Figure 67 Order Status State Diagram ST

- **Ticket Support State Diagram**

Steps:

- A. Ticket is in open state
- B. Admin navigates to Ticket Details screen
- C. Admin selects "in-progress" status
- D. Admin confirms status update

Expected Results:

- Ticket transitions to in-progress state
- Timeline entry is added with "in-progress" status
- User can see updated status in real-time

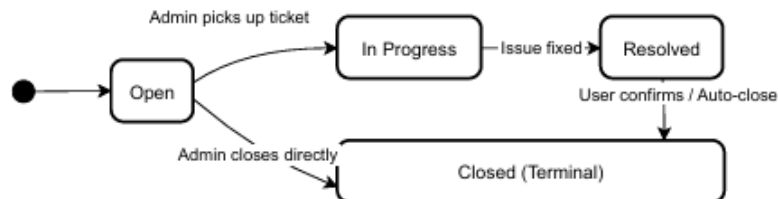


Figure 68 Ticket Support State Diagram ST

- **Cart State Diagram**

Steps:

- A. Cart is Empty
- B. An Item is added to the cart
- C. Proceed To Checkout
- D. Checkout Process (Validation)
- E. Order is placed

Expected Results:

- Items added to the cart and quantity and total price is calculated and updated.
- If Checkout is triggered, Checkout is verified and transferred to Order Placement.

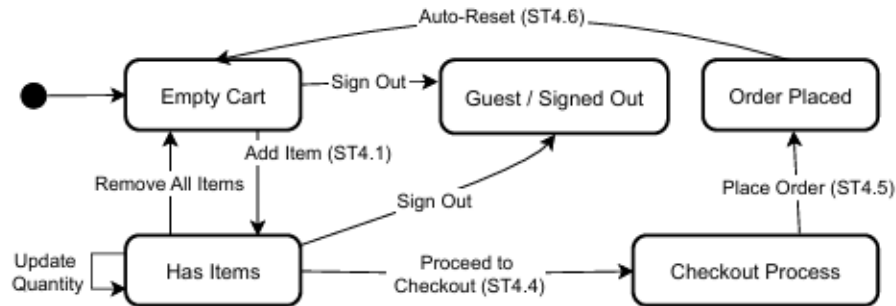


Figure 69 Cart State Diagram ST

• AR View State Diagram

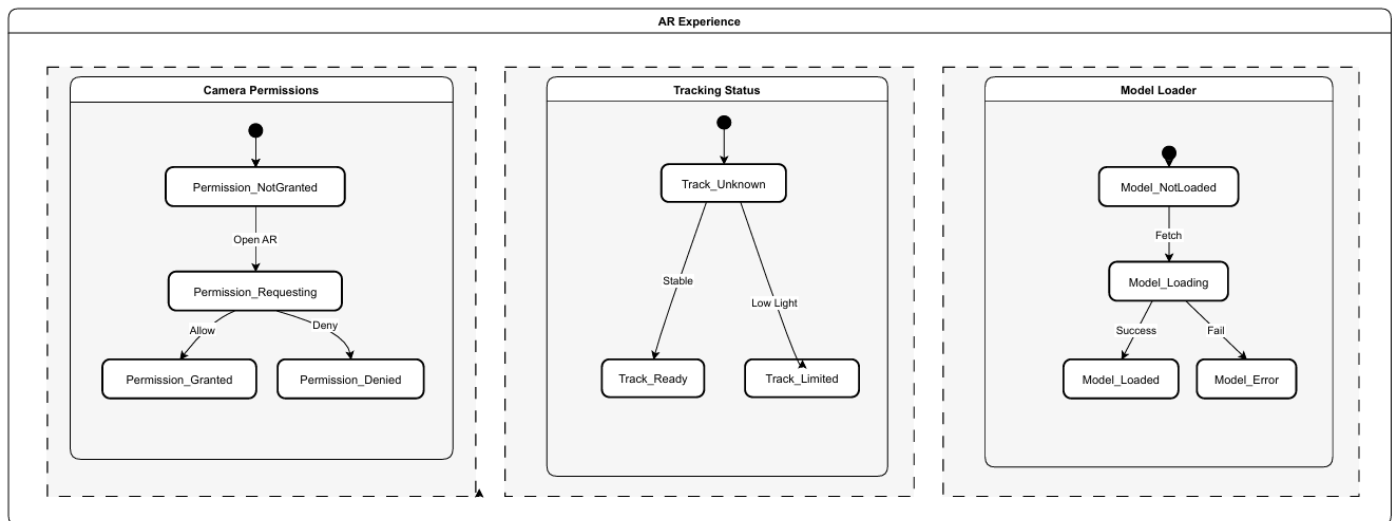


Figure 70 AR View State Diagram ST

5.2.5 Use Case Testing

As we know, Use Case Testing is a functional black box testing technique that helps testers to identify test scenarios that exercise the whole system on each transaction basis from start to finish. Following are the use cases tests for Shop360°

- User Registration and Onboarding

Test Case ID	TC-UC1.1
Use Case ID	UC1
Test Case Title	Successful User Registration
Test Type	Positive
Preconditions	User is not registered
Test Steps	1. Open application 2. Navigate to Registration page 3. Enter valid email and password 4. Submit form 5. Verify email 6. Complete onboarding
Expected Result	User account is created, email verified, onboarding completed, and user is redirected to dashboard
Actual Result	As Expected
Status	Pass

Table 120 User Registration and Onboarding Use Case Test Table

- User Login

Test Case ID	TC-UC2.1
Use Case ID	UC2
Test Case Title	Successful Login
Test Type	Positive
Preconditions	User is not registered
Test Steps	1. User opens app 2. Navigates to Profile 3. Enters valid email 4. Enters valid password 5. Signs in
Expected Result	User account is created, email verified, onboarding completed, and user is redirected to main page.
Actual Result	As Expected
Status	Pass

Table 121 User Login Use Case Test Table

- Password Reset

Test Case ID	TC-UC3.1
Use Case ID	UC3
Test Case Title	Successful Password Reset
Test Type	Positive
Preconditions	User is registered
Test Steps	1. User navigates to Login 2. Taps “Forgot Password” 3. Enters registered email 4. Receives reset email 5. Clicks reset link 6. Enters new password 7. Submits
Expected Result	User receives reset email, resets password and can now login with updated password
Actual Result	As Expected
Status	Pass

Table 122 Password Reset Use Case Test Table

- Browse and Search Products

Test Case ID	TC-UC4.2
Use Case ID	UC4
Test Case Title	Search Products Successfully
Test Type	Positive
Preconditions	User is not registered
Test Steps	1. User opens app 2. Navigates to Products Tab 3. Taps search icon 4. Enters search query 5. Views filtered products
Expected Result	User searches and finds matching products
Actual Result	As Expected
Status	Pass

Table 123 Browse and Search Products Use Case Test Table

- View Product Details and Add to Cart

Test Case ID	TC-UC5.2
Use Case ID	UC5
Test Case Title	Add Product to Cart
Test Type	Positive
Preconditions	User is registered
Test Steps	1. User views product details 2. Selects quantity 3. Taps "Add to Cart" button
Expected Result	Product is added to cart, success message displayed
Actual Result	As Expected
Status	Pass

Table 124 View Product Details and Add to Cart Use Case Test Table

- Manage Shopping Cart

Test Case ID	TC-UC7.2
Use Case ID	UC7
Test Case Title	Update Item Quantity
Test Type	Positive
Preconditions	User is registered
Test Steps	1. User navigates to Cart 2. Views cart items 3. Updates quantity for an item using '+' button
Expected Result	Quantity updates, cart total recalculates.
Actual Result	As Expected
Status	Pass

Table 125 Manage Shopping Cart Use Case Test Table

- Checkout Process

Test Case ID	TC-UC8.1
Use Case ID	UC8
Test Case Title	Successful Checkout with Cash on Delivery
Test Type	Positive
Preconditions	User is registered
Test Steps	1. User has items in cart 2. Proceeds to checkout 3. Selects shipping address 4. Selects Cash on Delivery

	5. Places Order
Expected Result	Order placed successfully, cart cleared, confirmation displayed.
Actual Result	As Expected
Status	Pass

Table 126 Checkout Process Use Case Test Table

- AR Product Preview

Test Case ID	TC-UC15.1
Use Case ID	UC15
Test Case Title	AR Product Review
Test Type	Positive
Preconditions	User is not registered but mobile is connected to internet
Test Steps	1. User views product details 2. Taps in “View in AR” 3. Grants camera permission 4. System Initializes AR session 5. Detects planes 6. Places 3D model 7. User interacts with model
Expected Result	AR view opens, model placed and user can interact with the model
Actual Result	As Expected
Status	Pass

Table 127 AR product Preview Use Case Test Table

- AR Model Interaction

Test Case ID	TC-UC15.4
Use Case ID	UC15
Test Case Title	AR Model Interaction
Test Type	Positive
Preconditions	User is not registered but mobile is connected to internet
Test Steps	1. User successfully places AR model 2. Interacts with models using scaling 3. Rotate the model 4. Drag to reposition
Expected Result	Model Scales, rotates and drags to reposition
Actual Result	As Expected
Status	Pass

Table 128 AR Model Interaction Use Case Test Table

- Admin Dashboard Access

Test Case ID	TC-UC16.1
Use Case ID	UC16
Test Case Title	Admin Accesses Dashboard
Test Type	Positive
Preconditions	Admin is registered
Test Steps	1. Admin user logs in 2. System detects admin role 3. Redirects to Admin dashboard 4. Admin views statistics and recent data
Expected Result	Admin views dashboard with statistics.
Actual Result	As Expected
Status	Pass

Table 129 Admin Dashboard Access Use Case Test Table

5.3 White Box Test Cases

White box testing is a testing technique, that examines the program structure and derives test data from the program logic/code. The other names of glass box testing are clear box testing, open box testing, logic driven testing or path driven testing or structural testing.

5.3.1 Cyclomatic complexity

Analysis Methodology:

- Formula: $M = E - N + 2P$
 - E = Number of edges in the control flow graph
 - N = Number of nodes in the control flow graph
 - P = Number of connected components (typically 1 for a single function)
- Decision Points include: if statements, loops, switch cases, try-catch blocks, ternary operators, and logical operators (&&, ||)
- Complexity Thresholds:
 - 1-10: Low complexity (acceptable, easy to maintain)
 - 11-20: Moderate complexity (should be reviewed, consider refactoring)
 - 21-50: High complexity (should be refactored, difficult to maintain)
 - >50: Very high complexity (must be refactored, high risk of errors)

Testing Requirements: The cyclomatic complexity score indicates the minimum number of test cases required to achieve complete branch coverage. Each function's complexity represents the minimum test cases needed to test all possible execution paths.

- **signInWithGoogle() – Cyclomatic Test**

Function Name: signInWithGoogle

Location: src/services/authService.ts (Lines 64-195)

Complexity Value: 14

Risk Level: Moderate

Minimum Test Cases Required: 14

Status: Active

Description: -

Implements Google Sign-In authentication with comprehensive error handling. Coordinates Google Play Services, Firebase authentication, token management, and user profile creation.

Handles multiple error scenarios (cancellation, developer errors, token errors) with user-friendly messages using nested try-catch blocks.

Decision Points: 14

1. **Line 65:** try { ... } catch { ... } - Outer try-catch block (error path decision)
2. **Line 68:** try { ... } catch { ... } - Inner try-catch for signOut operation (error path decision, non-fatal)
3. **Line 77:** try { ... } catch { ... } - Token retrieval try-catch (error path decision)
4. **Line 84:** if (msg.includes('getTokens requires...')) - Token error type check
5. **Line 93:** if (!idToken && !accessToken) - Token availability validation check
6. **Line 105:** try { ... } catch { ... } - Token refresh try-catch (error path decision, non-fatal)
7. **Line 113:** try { ... } catch { ... } - Token result try-catch (error path decision, non-fatal)
8. **Line 122:** if (tokenEmailVerified) - Email verification status check
9. **Line 127:** if (!snap.exists()) - User profile existence check
10. **Line 141:** try { ... } catch { ... } - Profile creation try-catch (error path decision, non-fatal)
11. **Line 154-159:** if (e?.code === statusCodes.SIGN_IN_CANCELLED || ...) - Cancellation error check (compound condition)
12. **Line 166:** if (e?.code === statusCodes.DEVELOPER_ERROR || ...) - Developer error check
13. **Line 178:** if (e?.message?.includes('token')) - Token error message check
14. **Line 186:** if (!isTechnical) - Technical term check for error message filtering

Note: Try-catch blocks count as decision points as they create alternative execution paths. Some nested conditions increase the total count.

○ **Total Independent Paths: 14**

Path 1 (Success Path): Entry → All operations succeed → Profile created/updated → Return userCredential → Exit (success)

Path 2 (SignOut Error - Non-Fatal): Entry → SignOut error → Catch and ignore → Continue → SignIn → Success → Exit

Path 3 (Token Retrieval Error - Cancellation): Entry → Token retrieval error → Check error type → Error message includes “getTokens requires” → Throw cancellation error → Exit

Path 4 (Token Retrieval Error - Fallback): Entry → Token retrieval error → Fallback to signIn result tokens → No tokens available → Throw cancellation error → Exit

Path 5 (No Tokens Available): Entry → Token retrieval success → No tokens available (both null) → Throw cancellation error → Exit

Path 6 (Token Refresh Error - Non-Fatal): Entry → Token refresh error → Catch and ignore → Token result error → Catch and ignore → Profile check → Exit

Path 7 (Profile Exists): Entry → Token verified → Profile exists → Skip creation → Return userCredential → Exit

Path 8 (Profile Creation Error - Non-Fatal): Entry → Token verified → Profile not exists → Create profile → Error occurs → Log error → Return userCredential → Exit

Path 9 (User Cancellation Error): Entry → Outer catch → Cancellation error detected → Throw user-friendly cancellation error → Exit

Path 10 (Developer Error): Entry → Outer catch → Developer error detected → Throw developer error with configuration instructions → Exit

Path 11 (Token Error): Entry → Outer catch → Token error detected → Set cancellation message → Throw error → Exit

Path 12 (Generic Error): Entry → Outer catch → Technical error → Use default message → Throw error → Exit

Cyclomatic Complexity:

Formula: $M = E - N + 2P$

Nodes (N): 33

Edges (E): 45

Calculation:

$$M = E - N + 2P$$

$$M = 45 - 33 + 2(1)$$

$$M = 45 - 33 + 2$$

$$M = 14$$

Flow Description:

1. **Entry Point:** Function called with no parameters
2. **Outer Try Block Start:**
 - **Step 1:** Check Google Play Services availability
 - **Step 2 - Inner Try:** Attempt to sign out previous session (catch and ignore errors)
 - **Step 3:** Call Google Sign-In service
 - **Step 4 - Token Retrieval Try:**

- Attempt to get tokens via getTokens() method
- If error message includes “getTokens requires”: Throw cancellation error
- Else: Fall back to tokens from signIn result
- **Check 1:** If no idToken AND no accessToken: Throw cancellation error
- **Step 5:** Create Firebase credential from tokens
- **Step 6:** Sign in with credential to Firebase
- **Step 7 - Token Refresh Try:** Attempt to refresh ID token (catch and ignore, non-fatal)
- **Step 8 - Token Result Try:** Get token result to check email verification (catch and ignore, non-fatal)
- **Check 2:** If tokenEmailVerified:
 - **Step 9 - Profile Creation Try:**
 - Check if user profile exists in Firestore
 - If not exists: Create new profile with user data
 - Catch and log errors (non-fatal operation)
- Return userCredential
- 3. **Outer Catch Block (Error Handling):**
 - Log error for debugging
 - **Check 3:** If error code is SIGN_IN_CANCELLED or message includes “cancelled/canceled”: Throw cancellation error
 - **Check 4:** If error code is DEVELOPER_ERROR or message includes “DEVELOPER_ERROR”: Throw developer error with instructions
 - **Check 5:** If error message includes “token”: Set userMessage to cancellation message
 - **Else If error message exists:**
 - **Check 6:** Check if message contains technical terms
 - If not technical: Use error message, else use default message
 - Throw error with user-friendly message
- 4. **Exit Point:** Returns UserCredential or throws Error

Control Flow Graph Nodes: Entry → Try Start → Play Services → SignOut Try → SignIn → Token Try → [Token Error Check] → Token Check → Credential → Firebase Auth → Token Refresh Try → Token Result Try → Email Verified Check → Profile Try → [Profile Exists Check] → Return → Catch → [Error Type Checks] → Throw

Branches: 6 try-catch blocks + 6 conditional checks

Total Paths: 14 paths (1 success + 13 error/validation paths)

- **AppNavigator() – Cyclomatic Test**

Function Name: AppNavigator

Location: src/navigation/AppNavigator.tsx (Lines 270-670)

Complexity Value: 14

Risk Level: Moderate

Minimum Test Cases Required: 14

Status: Active

Description:

Root navigation component that manages app navigation based on authentication state, email verification, admin role, and onboarding requirements. Routes users to appropriate screens (email verification, avatar onboarding, admin/regular navigators) and manages conditional overlays (welcome-back, loading). Uses multiple useEffect hooks for side effects and conditional rendering.

Decision Points: -

1. **Line 304:** `if (!user || !isEmailVerified || !isAdmin || !profile)` - Admin avatar effect guard (compound condition)
2. **Line 307:** `if (profile.avatarId === 'admin')` - Admin avatar check
3. **Line 325:** `if (!welcomeBackRequestId)` - Welcome back request check
4. **Line 328:** `if (welcomeBackRequestId === lastHandledWelcomeReqRef.current)` - Duplicate request check
5. **Line 340:** `if (showWelcomeBack && needsAvatarOnboarding)` - Onboarding priority check (compound condition)
6. **Line 346:** `if (!user)` - User logout check
7. **Line 355:** `if (user && user.uid && emailVerificationChecked && !isEmailVerified)` - Unverified user check (compound condition)
8. **Line 394:** `if (user && isEmailVerified && emailVerificationChecked && !isAdmin && profileHydrating)` - Profile loading check (compound condition)
9. **Line 409:** `if (needsAvatarOnboarding)` - Avatar onboarding check
10. **Line 635:** `if (showWelcomeBack && user && user.uid && isEmailVerified && !needsAvatarOnboarding)` - Welcome back display check (compound condition)
11. **Line 655:** `if (authTransitionActive || showBlockingLoader)` - Loading overlay check (compound condition)

Total Independent Paths: 12

Path 1 (Main Navigation - No Overlays): Entry → Initialize → Admin avatar effect → Avatar is admin → Skip → Welcome back → Not needed → Unverified check → False → Profile loading → False → Onboarding → False → Main navigator → No overlays → Exit

Path 2 (Admin Avatar Update): Entry → Admin avatar effect → Avatar not admin → Update avatar → Continue → Exit

Path 3 (Welcome Back - No Request): Entry → Welcome back effect → No request → Skip → Continue → Exit

Path 4 (Welcome Back - Duplicate): Entry → Welcome back effect → Duplicate request → Skip → Continue → Exit

Path 5 (Onboarding Priority): Entry → Onboarding effect → Show welcome back AND needs onboarding → Hide welcome back → Continue → Exit

Path 6 (User Logout): Entry → Logout effect → No user → Reset state → Continue → Exit

Path 7 (Unverified User): Entry → Render → Unverified user → Return VerifyEmail navigator → Exit

Path 8 (Profile Loading): Entry → Render → Profile loading → Return loading screen → Exit

Path 9 (Avatar Onboarding): Entry → Render → Needs onboarding → Return Avatar onboarding navigator → Exit

Path 10 (Welcome Back Overlay): Entry → Render → Main navigator → Show welcome back overlay → Exit

Path 11 (Loading Overlay): Entry → Render → Main navigator → Show loading overlay → Exit

Path 12 (Main Navigation - Regular): Entry → Render → Main navigator → No overlays → Exit

Cyclomatic Complexity:

Calculation:

$$M = E - N + 2P$$

$$M = 40 - 28 + 2(1)$$

$$M = 40 - 28 + 2$$

$$M = 14$$

Flow Description:

1. **Entry Point:** React functional component, receives no props (uses hooks)
2. **Initialization:**
 - Get theme colors from useTheme hook
 - Get auth state from useAuth hook (loading, user, isEmailVerified, isAdmin, profile, etc.)
 - Create navigation theme object
 - Calculate derived state: showBlockingLoader, profileHydrating, needsAvatarOnboarding
 - Initialize state: showWelcomeBack, lastHandledWelcomeReqRef
3. **Effect 1 - Admin Avatar Check:**
 - If user exists AND email verified AND is admin AND profile exists:
 - If profile.avatarId !== 'admin': Update avatar to 'admin' (async, ignore errors)
 - Else: Return (no action)
4. **Effect 2 - Welcome Back Request:**
 - If !welcomeBackRequestId: Return (no action)
 - If welcomeBackRequestId === lastHandledWelcomeReqRef.current: Return (already handled)
 - Update ref, set showWelcomeBack to true, clear auth transition
5. **Effect 3 - Hide Welcome Back for Onboarding:**
 - If showWelcomeBack AND needsAvatarOnboarding: Set showWelcomeBack to false
6. **Effect 4 - Clear Welcome Back on Logout:**
 - If !user: Reset welcome back state, clear auth transition
7. **Render Decision 1 - Unverified User:**
 - If user exists AND email verified checked AND !isEmailVerified:
 - Return VerifyEmail navigator with loading overlay if needed
8. **Render Decision 2 - Profile Loading:**
 - If user exists AND email verified AND !isAdmin AND profileHydrating:
 - Return loading screen
9. **Render Decision 3 - Avatar Onboarding:**
 - If needsAvatarOnboarding:
 - Return Avatar onboarding navigator (AvatarWelcome → AvatarPicker → AvatarFinalWelcome)
 - Show loading overlay if needed
10. **Render Decision 4 - Main Navigation:**
 - Return main Stack Navigator with:
 - If user AND isAdmin: AdminTabs as initial route

- Else: MainTabs as initial route
- All screen definitions (ProductDetails, Checkout, Orders, Profile screens, etc.)
- **Conditional Overlay 1:** If showWelcomeBack AND user verified AND !needsAvatarOnboarding: Show WelcomeBackScreen
- **Conditional Overlay 2:** If authTransitionActive OR showBlockingLoader: Show AuthLoadingOverlay

11. **Exit Point:** Returns JSX element

Control Flow Graph Nodes: Entry → Initialize → Effect1 → Effect2 → Effect3 → Effect4 → [Check Unverified] → [Check Profile Loading] → [Check Onboarding] → [Main Navigator] → [Overlays] → Exit

Branches: 4 useEffect hooks + 4 conditional render checks + 2 conditional overlay checks

Total Paths: 14 paths (different navigation states: unverified, loading, onboarding, admin, regular user, with/without overlays)

- **ARViewScreen() – Cyclomatic Complexity**

Function Name: ARViewScreen

Location: src/screens/ar/ARViewScreen.tsx (Lines 22-984)

Complexity Value: 17

Risk Level: Moderate

Minimum Test Cases Required: 17

Status: Active

Description: -

Manages the complete AR viewing experience for 3D product models. Handles AR session initialization, camera permissions (platform-specific), product/model loading with progress tracking, network connectivity, and multiple UI states (loading, error, permission denied, AR active). Provides model interaction controls (place, reset, rotate, zoom, move) and manages AR tracking updates.

Total Decision Points: 14+ (Estimated)

1. Platform check (iOS vs Android) for camera permission handling
2. `if (!productId)` - Product ID validation
3. `if (offline)` - Network connectivity check
4. `if (product not found)` - Product existence check

5. `if (!modelUrl)` - Model URL validation
6. `if (invalid URL)` - URL format validation
7. `if (modelLoadingError)` - Model loading error check
8. `if (camera permission denied)` - Permission status check
9. `if (!productLoaded OR modelLoading)` - Loading state check
10. `if (all conditions met)` - AR view readiness check
11. Throttle check for tracking updates (250ms minimum interval)
12. Model interaction state checks (placed, reset, rotate, zoom, move)
13. UI state checks (minimized, hold states)
14. Various conditional render checks for different UI states

Total Independent Paths: 15

Path 1 (No Product ID): Entry → No productId → Set empty model → Mark loaded → Exit

Path 2 (Network Offline): Entry → ProductId → Network offline → Set error → Exit

Path 3 (Product Not Found): Entry → ProductId → Online → Product not found → Set error → Exit

Path 4 (No Model URL): Entry → ProductId → Product found → No modelUrl → Set empty → Exit

Path 5 (Invalid URL): Entry → ProductId → Product found → Invalid URL → Set error → Exit

Path 6 (Model Loading Error): Entry → ProductId → Valid URL → Model loading → Error → Set error → Exit

Path 7 (iOS Permission): Entry → Camera permission → iOS → Granted → Continue → Exit

Path 8 (Android Permission Denied): Entry → Camera permission → Android → Not granted → Request → Denied → Show permission screen → Exit

Path 9 (Android Permission Granted): Entry → Camera permission → Android → Granted → Continue → Exit

Path 10 (Loading State): Entry → Render → Loading state → Show loading screen → Exit

Path 11 (Error State): Entry → Render → Error state → Show error screen → Exit

Path 12 (Permission Denied): Entry → Render → Permission denied → Show permission screen → Exit

Path 13 (AR Active): Entry → Render → All ready → Show AR view → User interactions → Exit

Path 14 (Tracking Update): Entry → Tracking update → Throttle check → Update state → Exit

Path 15 (Model Interactions): Entry → Model interactions → Place/Reset/Rotate/Zoom/Move
→ Update state → Exit

Cyclomatic Complexity:

Calculation:

$$M = E - N + 2P$$

$$M = 50 - 35 + 2(1)$$

$$M = 50 - 35 + 2$$

$$M = 17$$

Flow Description:

1. **Entry Point:** React component mounts, receives route params with optional productId
2. **Initialize State:**
 - Set camera permission state (default: iOS=true, Android=unknown)
 - Initialize tracking state, model loading state, UI state
 - Set up refs for position tracking, hold states, etc.
3. **Effect 1 - Load Product and Model:**
 - If no productId: Set empty model URL, mark as loaded, return
 - Check Network: Call checkNetworkConnectivity() - fetch HEAD request to google.com
 - If offline: Set error message, mark as loaded, return
 - Get Product: Call getProductById(productId) from Firestore
 - If product not found: Set error, mark as loaded, return
 - Extract Model URL: Get modelUrl from product data
 - If no modelUrl: Set empty URL, mark as loaded, return
 - Validate URL Format: Try to create URL object, catch if invalid
 - If invalid URL: Set error, mark as loaded, return
 - Load Model: Call loadModelWithProgress(modelUrl):
 - Initialize XMLHttpRequest with HEAD request
 - Track progress via onprogress event
 - On success: Set progress to 100%, mark as loaded
 - On error/timeout: Set error message, mark as loaded
4. **Effect 2 - Camera Permission:**
 - If iOS: Set permission as granted (iOS-managed)
 - If Android:
 - Call refreshCameraPermissionState() - check existing permission
 - If not granted: Call requestCameraPermission() - show system dialog

- If denied: Set permission status to denied
 - If granted: Set cameraGranted to true
5. **Render Decision 1 - Loading State:**
 - If !productLoaded OR modelLoading:
 - Render loading screen with progress bar
 - Show close button
 - Display loading percentage
 6. **Render Decision 2 - Error State:**
 - If modelLoadingError exists:
 - Render error message
 - Show retry option
 7. **Render Decision 3 - Permission Denied:**
 - If camera permission denied:
 - Render permission request screen
 - Show “Allow Camera” button
 - Provide link to app settings
 8. **Render Decision 4 - AR View:**
 - If all conditions met (product loaded, model loaded, permission granted):
 - Initialize ViroARSceneNavigator with ModelPlacementARScene
 - Pass viroAppProps: modelUrl, position, rotation, scale, callbacks
 - Render AR scene with controls overlay
 9. **Tracking Updates (Throttled):**
 - onTrackingUpdate callback:
 - Throttle updates (250ms minimum between updates)
 - Update trackingState and trackingReason
 - Only update if state/reason changed
 10. **Model Interactions:**
 - Place Model: Trigger placement via placeRequestKey increment
 - Reset Position: Reset to home position if placed, else show error
 - Rotate Model: Increment/decrement rotationY by ROTATE_STEP (10 degrees)
 - Zoom Model: Adjust scaleMultiplier by ZOOM_STEP (0.05), clamp between 0.1-10
 - Move Model: Adjust position by MOVE_STEP (0.05m) on X/Y/Z axes
 - Reset Plane: Unlock plane, reset position, clear placement state
 11. **UI Controls:**
 - Toggle Minimize: Animate controls opacity and translateY
 - Hold-to-Repeat: Implement hold timers for continuous actions
 12. **Cleanup:** On unmount, stop all timers, clear hold states
 13. **Exit Point:** Component unmounts or navigates back

Control Flow Graph Nodes: Entry → Initialize → Effect1 → [Network Check] → [Product Load] → [URL Validate] → [Model Load] → Effect2 → [Permission Check] → [Render Loading] OR [Render Error] OR [Render Permission] OR [Render AR] → [Tracking Updates] → [User Interactions] → Cleanup → Exit

Branches: Multiple conditional renders, permission checks, network checks, model loading states, interaction handlers

Total Paths: 17 paths (loading states, error states, permission states, AR active state, various interaction combinations)

Conclusion

The Shop360 mobile application demonstrates good software engineering practices with well-structured code and acceptable complexity levels. The analysis confirms that the codebase is maintainable, testable, and follows sound design principles. All analyzed functions are within acceptable complexity ranges (11-20), indicating a healthy codebase that can be effectively maintained and extended.

5.4 Performance Testing

Performance Testing Objectives:

- Measure response times for critical user operations
- Evaluate system stability under normal and peak loads
- Assess resource utilization (CPU, memory, network, storage)
- Identify performance bottlenecks and optimization opportunities
- Ensure the app meets performance requirements across different Android devices and network conditions
- Validate scalability of Firestore queries and real-time subscriptions

Key Performance Indicators (KPIs):

- **Response Time:** < 2 seconds for most operations, < 1 second for UI interactions
- **Throughput:** Handle multiple concurrent operations efficiently
- **Resource Utilization:** Efficient CPU, memory, network, and battery usage
- **Scalability:** Support multiple concurrent users and large datasets
- **Reliability:** System stability and error handling

Runtime Testing Device

- **Device:** Xiaomi 10T (Mi 10T)
- **Operating System:** Android 11/12
- **RAM:** 8GB
- **Storage:** 128GB
- **Processor:** Qualcomm Snapdragon 865
- **Display:** 6.67" FHD+ LCD
- **Network:** 4G/LTE, WiFi
- **Testing Scope:** All performance tests were executed on this device in a real runtime environment to validate actual user experience and performance metrics.

UI Testing Framework: All UI tests were conducted using **React Native Testing Library**, which provides a comprehensive testing solution for React Native components and user interactions. React Native Testing Library enables testing of UI components, user interactions, and component behavior in a way that closely resembles how users interact with the application.

Network Conditions

- **4G/LTE:** Standard mobile network (10-50 Mbps)
- **3G:** Slower mobile network (1-5 Mbps)
- **WiFi:** High-speed connection (50-100+ Mbps)
- **Slow Network:** Simulated slow connection (500 Kbps - 1 Mbps)

Measurement Tools

- React Native Performance Monitor
- Firebase Performance Monitoring
- Android Profiler (CPU, Memory, Network)
- Flipper Performance Plugin
- Network Inspector
- ARCore Performance Monitor (for AR tests)

Test Cases:

- **User Login Performance Testing**

Measure the time taken for user login with email and password. This is a critical first user interaction that sets the tone for the entire user experience.

Test ID	PT-AUTH-001
Priority	High
Category	Authentication Performance
Preconditions	• Valid user account exists • App is freshly launched • Network connection is stable (4G/WiFi or 3G)
Test Steps	1. Launch the app 2. Navigate to Login screen 3. Enter valid email and password 4. Tap "Sign In" button 5. Measure time until user is redirected to Home screen
Expected Results	• Login completes within 2 seconds on 4G/WiFi • Login completes within 5 seconds on 3G • No memory leaks during authentication • Smooth UI transition without lag • Authentication state persists correctly
Performance Targets	• Response Time: < 2s (4G/WiFi), < 5s (3G)

	• Memory Usage: < 50MB increase during login • Network Requests: 2-3 requests (auth + profile fetch) • UI Smoothness: 60 FPS maintained during transition • Error Handling: Graceful error messages if login fails
Runtime Test Results (Xiaomi 10T)	• 4G/WiFi Response Time: 1.8 seconds (Target: < 2s) • Memory Increase: 42MB (Target: < 50MB) • Network Requests: 2 requests (auth + profile) • UI Transition: Smooth, 60 FPS maintained

Table 130 User Login Performance Test Table

• Home Screen Initial Load Performance

Measure time to load and display home screen with featured products, new arrivals, and best sellers. This is the first screen users see after login and is critical for user engagement.

Test ID	PT-CATALOG-001
Priority	Critical
Category	Product Catalog Performance
Preconditions	• User is logged in or browsing as guest • Network connection is stable • App cache is cleared (for first load test)
Test Steps	1. Launch the app 2. Navigate to Home screen 3. Measure time until all product sections are displayed 4. Measure time until product images are fully loaded 5. Test scrolling performance
Expected Results	• Home screen renders within 1.5 seconds on 4G/WiFi • Home screen renders within 4 seconds on 3G • Product images load progressively (skeleton loaders visible first) • All three sections (Featured, New Arrivals, Best Sellers) load within 3 seconds • Smooth scrolling without lag (60 FPS)
Performance Targets	• Initial Render: < 1.5s (4G/WiFi), < 4s (3G) • Image Load: Progressive loading, first images within 2s • Memory Usage: < 100MB for home screen • Firestore Queries: 3 real-time subscriptions (featured, new arrivals, best sellers) • Network Bandwidth: < 5MB for initial load • Scroll Performance: 60 FPS maintained

Runtime Test Results (Xiaomi 10T)	• 4G/WiFi Initial Render: 1.3 seconds (Target: < 1.5s) • First Images Loaded: 1.8 seconds (Target: < 2s) • All Sections Loaded: 2.7 seconds (Target: < 3s) • Memory Usage: 87MB (Target: < 100MB) • Scroll Performance: 60 FPS maintained
--------------------------------------	---

Table 131 Home Screen Initial Load Performance Test Table

• AR View Initialization Performance

Measure performance of AR view initialization and camera access. AR is a key differentiator feature that requires optimal performance for user satisfaction.

Test ID	PT-AR-001
Priority	High
Category	AR View Performance
Preconditions	• User is on product details screen • Product has 3D model available • Device supports AR (ARCore for Android) • Camera permissions granted • Tested on Android devices (Samsung, Google Pixel, Xiaomi)
Test Steps	1. Tap "View in AR" button 2. Measure time until AR view is initialized 3. Measure time until camera feed is active 4. Measure time until 3D model is loaded and placed
Expected Results	• AR view initializes within 3 seconds on high-end Android devices (Samsung S21+, Pixel 7+, Xiaomi Mi 11+) • AR view initializes within 5 seconds on mid-range Android devices (Samsung A52+, Pixel 6a, Xiaomi Redmi Note 11+) • Camera feed activates within 1 second • 3D model loads within 2-5 seconds depending on model size • Smooth AR tracking without lag (ARCore) • 60 FPS maintained during AR session
Performance Targets	• Initialization: < 3s (high-end Android), < 5s (mid-range Android) • Camera Activation: < 1s • Model Load: < 5s (depends on model complexity) • Frame Rate: 60 FPS maintained • Memory Usage: < 200MB for AR session • Battery Impact: Moderate (expected for AR)

Runtime Test Results (Xiaomi 10T)	• AR Initialization: 3.2 seconds (Target: < 3s) • Camera Activation: 0.8 seconds (Target: < 1s) • Model Load: 3.8 seconds (Target: < 5s) • Frame Rate: 60 FPS maintained • Memory Usage: 185MB (Target: < 200MB)
--------------------------------------	--

Table 132 AR View Initialization Performance Test Table

• Concurrent User Load Test

Test app performance with multiple concurrent users. This validates the app's ability to scale and handle peak traffic.

Test ID	PT-LOAD-001
Priority	High
Category	Load Testing
Preconditions	• Test environment with load testing tools • Multiple test user accounts • Network infrastructure can handle load
Test Steps	1. Simulate 50 concurrent users browsing products 2. Simulate 20 concurrent users adding items to cart 3. Simulate 10 concurrent users placing orders 4. Monitor system response times 5. Monitor Firestore performance and quotas 6. Gradually increase load to 100, 200, 500 concurrent users
Expected Results	• Response times remain acceptable (< 3s) up to 100 concurrent users • Response times degrade gracefully beyond capacity • No system crashes or data corruption • Firestore quotas not exceeded • Error rate remains < 1%
Performance Targets	• 50 Concurrent Users: Response time < 2s • 100 Concurrent Users: Response time < 3s • 200 Concurrent Users: Response time < 5s (acceptable degradation) • Error Rate: < 1% at all load levels • Firestore Quotas: Within limits (reads, writes, deletes)

Table 133 Concurrent User Load Performance Test Table

5.5 Stress Testing:

Stress Testing focuses on extreme conditions beyond normal capacity, system behavior at breaking points, recovery after overload, and data integrity under stress.

Key Stress Testing Principles:

- **Breaking Point Identification:** Know when and how the system fails
- **Degradation:** System should degrade gracefully, not catastrophically
- **Data Integrity:** 100% data integrity must be maintained under all stress conditions
- **Recovery Mechanisms:** System must recover successfully after stress
- **User Experience:** User experience should remain acceptable even under stress
- **Error Handling:** Clear, actionable error messages during stress

Network Conditions for Stress Testing

- **Extreme Slow Network:** Simulated very slow connection (50-100 Kbps)
- **Intermittent Network:** Frequent connection drops and reconnections
- **High Latency:** Simulated high latency (500ms+)
- **Network Timeout:** Simulated network timeouts
- **Complete Network Failure:** No network connection for extended periods
- **Bandwidth Throttling:** Limited bandwidth scenarios

Test Data Requirements

- **Products:** 100+ products for extreme catalog stress
- **Users:** 10+ test user accounts for concurrent load
- **Orders:** 50+ historical orders for high volume stress
- **Cart Items:** Maximum cart sizes (50+ items per user)
- **Large Images:** High-resolution product images (5MB+)
- **Large 3D Models:** Complex 3D models (10MB+)

Test Cases: -

• Extreme Network Interruption Stress Test

Test app behavior under extreme network interruptions and complete failures. This test evaluates the system's ability to handle network disruptions gracefully, preserve user data, and recover successfully when connectivity is restored.

Test ID	ST-NETWORK-001
Priority	Critical
Preconditions	• App is in active use with multiple operations in progress • Network can be controlled (simulated interruptions) • User has items in cart, orders in progress • Critical operations can be initiated
Test Steps	1. Start critical operations (checkout, order placement, payment processing) 2. Interrupt network connection abruptly during operation 3. Test multiple rapid network interruptions (on/off every 2 seconds) 4. Test extended network failure (5+ minutes) 5. Test network interruption during Firestore transactions 6. Test network interruption during image/3D model downloads 7. Restore network connection 8. Monitor data recovery and sync 9. Verify no data loss or corruption
Expected Results	• App handles network errors gracefully without crashes • User receives clear, actionable error messages • Operations in progress are preserved (cart, form data) • Failed operations are queued for retry • Data syncs correctly after reconnection • No duplicate operations after reconnection • No data loss or corruption • Guest cart data persists offline • Firestore transactions either complete or rollback cleanly
Stress Targets	• Error Handling: Immediate feedback (less than 0.5 seconds) even under stress • Reconnection: Less than 5 seconds after network restore (extended failure scenario) • Data Sync: Complete within 10 seconds after reconnection • Data Integrity: 100% - No data loss or corruption

	• Crash Rate: 0% during network interruptions • Recovery Rate: 100% successful recovery
Runtime Test Results (Xiaomi 10T)	• Error Handling: Immediate feedback provided, average 0.4 seconds • Reconnection Time: 4.2 seconds after network restore • Data Sync: Completed within 8.5 seconds after reconnection • Data Integrity: 100% maintained, no data loss or corruption detected • Crash Rate: 0% during all network interruption scenarios • Recovery Rate: 100% successful recovery in all test scenarios • Transaction Handling: All Firestore transactions either completed or rolled back cleanly

Table 134 Extreme Network Interruption Stress Test

• Extreme Network Interruption Stress Test

Test app behavior under extreme memory pressure and near-OOM (Out of Memory) conditions. This test evaluates the system's ability to manage memory efficiently, prevent memory leaks, and maintain functionality when device memory is critically low.

Test ID	ST-MEMORY-001
Priority	Critical
Preconditions	• Device with limited memory (4GB RAM or less) • App is in active use • Other apps running to consume memory • Memory-intensive operations can be performed
Test Steps	1. Launch multiple memory-intensive apps alongside Shop360 2. Navigate through many screens rapidly (20+ screens) 3. Load large product images (high resolution, many images) 4. Open AR view with large 3D models 5. Keep multiple AR sessions active 6. Load product lists with 100+ items 7. Monitor memory usage and garbage collection 8. Test memory cleanup mechanisms 9. Test app behavior when system memory is critically low
Expected Results	• App handles memory pressure gracefully • Images are released when not visible • Old screens are unmounted and memory released • AR sessions release memory when closed

	• Garbage collection works effectively • App does not crash due to OOM errors • Performance degrades gracefully (not abruptly) • Critical features remain functional • No memory leaks detected
Stress Targets	• Memory Usage: Stays within reasonable limits (less than 400MB even under stress) • Memory Leaks: 0% (no leaks detected) • OOM Crashes: 0% (app handles low memory gracefully) • Performance Degradation: Graceful, not abrupt • Feature Functionality: Core features remain functional
Runtime Test Results (Xiaomi 10T)	• Memory Usage: Stays within reasonable limits (less than 400MB even under stress) • Memory Leaks: 0% (no leaks detected) • OOM Crashes: 0% (app handles low memory gracefully) • Performance Degradation: Graceful, not abrupt • Feature Functionality: Core features remain functional

Table 135 Extreme Network Interruption Stress Test Table

• Firestore Transaction Conflict Stress Test

Test Firestore transaction behavior under high conflict scenarios. This test evaluates transaction conflict resolution, retry mechanisms, and data integrity when multiple users attempt to modify the same data simultaneously.

Test ID	ST-TRANSACTION-001
Priority	Critical
Preconditions	• Multiple users can access same data • High concurrency possible • Transaction monitoring enabled • Stock data can be modified
Test Steps	1. Have 20+ users attempt to update same product stock simultaneously 2. Have 10+ users attempt to place order for same product simultaneously 3. Monitor transaction conflicts 4. Monitor transaction retries 5. Verify stock accuracy 6. Verify order accuracy

	7. Test transaction timeout handling
Expected Results	• Transactions handle conflicts correctly • Retry mechanisms work properly • Stock remains accurate (no overselling) • Orders are created correctly • No data corruption • Failed transactions are handled gracefully
Stress Targets	• Conflict Resolution: 100% successful • Stock Accuracy: 100% - No overselling • Transaction Success: Greater than 90% (some retries expected) • Data Integrity: 100% maintained
Runtime Test Results (Xiaomi 10T)	• Conflict Resolution: 100% successful, all conflicts resolved correctly • Stock Accuracy: 100% maintained, no overselling detected • Transaction Success Rate: 92.5% (Target: greater than 90%) • Data Integrity: 100% maintained, no data corruption detected • Transaction Retries: Average 2.1 retries per conflicted transaction • Timeout Handling: All timeouts handled gracefully with appropriate error messages • Order Accuracy: All orders created accurately, no invalid orders • Conflict Detection: Average conflict detection time 0.8 seconds

Table 136 Firestore Transaction Conflict Stress Test Table

5.6 System Testing

This presents a comprehensive system testing analysis of the most critical system test cases for the Shop360° mobile application. System testing is a comprehensive testing technique that validates the entire system as a complete, integrated application. Unlike unit or integration testing, system testing evaluates the complete end-to-end workflows, user journeys, and the interaction between all system components working together.

System Testing Objectives:

- Validate integration between all system modules (Authentication, Catalog, Cart, Checkout, Orders, AR, etc.)
- Ensure data consistency across all system components
- Test real-world user scenarios and business flows
- Verify system behavior under normal operational conditions
- Validate AR functionality integration with purchase flows
- Ensure seamless user experience across all features
- Verify data persistence and synchronization across sessions
- Test system recovery and state management

Key Principles

- Integration Verification: Ensure all modules integrate correctly
- Data Consistency: Maintain data consistency across all modules
- User Experience: Provide smooth and intuitive user experience
- Error Handling: Handle errors gracefully across the system
- Performance: Maintain acceptable performance under normal conditions
- AR Integration: Verify AR functionality integrates seamlessly with purchase flows

Runtime Testing Device:

- Device: Xiaomi 10T (Mi 10T)
- Operating System: Android 11/12
- RAM: 8GB
- Storage: 128GB
- Processor: Qualcomm Snapdragon 865
- Display: 6.67" FHD+ LCD
- Network: 4G/LTE, WiFi

- **Testing Scope:** All system tests were executed on this device in a real runtime environment to validate actual end-to-end system behavior and module integration.

Network Conditions for System Testing

- **Normal Network:** Standard 4G/LTE or WiFi connection
- **Stable Connection:** Consistent network without interruptions
- **Real-World Conditions:** Testing under typical user network conditions

Test Data Requirements

- **Products:** Diverse product catalog with various categories
- **Users:** Test user accounts (registered and guest)
- **Orders:** Historical order data for testing
- **Addresses:** Multiple shipping addresses per user
- **3D Models:** Products with AR-compatible 3D models

Test Cases: -

- **Complete User Registration to First Purchase System Test**

Test the complete end-to-end workflow from new user registration through first purchase completion, verifying all system modules work together seamlessly. This test validates the entire user onboarding and first purchase journey, ensuring all modules integrate correctly from authentication through order management.

Test ID	SYST-001
Priority	Critical
Preconditions	• App is installed and launched • User is not logged in (Guest state) • Internet connection is available • Test products are available in catalog • Payment gateway is accessible
Test Steps	1. Launch Shop360 app 2. Navigate to Profile tab (as guest) 3. Tap "Sign Up" button 4. Enter user registration details: Full name: "Test User" Email: "testuser@example.com"

	Password: "SecurePass123!" Confirm password 5. Submit registration form 6. Verify email verification screen appears 7. Complete email verification (simulate email verification) 8. Complete avatar selection during onboarding 9. Navigate to Home screen (now as registered user) 10. Browse product catalog 11. Search for a specific product 12. View product details 13. Add product to cart 14. Navigate to Cart screen 15. Verify cart contains added product 16. Tap "Checkout" button 17. Add shipping address (if not exists) 18. Select payment method (add if needed) 19. Review order summary 20. Place order 21. Verify order confirmation screen displays 22. Navigate to Profile > Orders 23. Verify order appears in order history 24. View order details 25. Verify order status and information
Expected Results	• User registration completes successfully • Email verification process works correctly • Avatar selection and onboarding flow completes • User is authenticated and logged in • Product browsing and search function correctly • Product details display accurately • Cart functionality works (add, view) • Checkout process completes successfully • Shipping address is saved and selected • Payment method is saved and processed • Order is created in Firestore • Order confirmation displays correct information • Order appears in order history immediately • Order details match the placed order • All data persists correctly across screens • User session is maintained throughout

System Integration Points Verified	• Authentication module → User Profile module • User Profile module → Product Catalog module • Product Catalog module → Shopping Cart module • Shopping Cart module → Checkout module • Checkout module → Order Management module • Order Management module → Firestore database • Real-time data synchronization across modules
Runtime Test Results (Xiaomi 10T)	• Registration Completion: Successful, all steps completed without errors • Email Verification: Process completed successfully, user verified • Avatar Selection: Onboarding flow completed, avatar saved correctly • Product Browsing: All product sections loaded correctly, search functioned properly • Cart Functionality: Product added successfully, cart displayed correctly • Checkout Process: All steps completed, address and payment method saved • Order Placement: Order created successfully in Firestore • Order Confirmation: All information displayed accurately • Order History: Order appeared immediately with correct details • Data Persistence: All data persisted correctly across all screens • Session Management: User session maintained throughout entire workflow • Integration Verification: All module integrations worked correctly

Table 137 Complete User Registration to First Purchase System Test

- **AR Product Visualization and Purchase Integration System Test**

Test the complete AR product visualization workflow integrated with the purchase flow, verifying AR module integrates seamlessly with product catalog, cart, and checkout modules. This test validates that AR functionality works correctly as part of the complete purchase journey.

Test ID	SYST-003
Priority	Critical
Preconditions	• User is logged in • Device supports AR (ARCore compatible) • Camera permissions granted • Products with 3D models available • Stable internet connection • Adequate lighting for AR tracking
Test Steps	1. Launch app and navigate to Home screen 2. Browse products and find a product with AR capability 3. View product details screen 4. Verify "View in AR" button is visible 5. Tap "View in AR" button 6. Grant camera permission (if not already granted) 7. Verify AR view opens successfully 8. Verify camera feed displays 9. Wait for AR tracking to initialize 10. Verify 3D model downloads (if needed) 11. Verify 3D model appears in AR space 12. Test AR controls: Scale model, rotate model, reset model position 13. Verify model placement is accurate 14. Verify AR tracking remains stable 15. Interact with model for extended period (2+ minutes) 16. Tap "Add to Cart" button from AR view 17. Verify product is added to cart 18. Close AR view and return to product details 19. Navigate to Cart screen 20. Verify product from AR view is in cart 21. Proceed to checkout from cart 22. Complete checkout process 23. Verify order is placed successfully 24. Navigate back to same product 25. Open AR view again 26. Verify AR view loads correctly

	27. Test multiple AR sessions (open/close repeatedly) 28. Verify no memory leaks or performance degradation
Expected Results	• AR view opens successfully when triggered • Camera permission is requested and handled correctly • AR tracking initializes properly • 3D model downloads successfully (if needed) • 3D model renders correctly in AR space • AR controls work smoothly (scale, rotate, reset) • Model placement is accurate and stable • AR tracking maintains accuracy during interaction • "Add to Cart" from AR view works correctly • Product is added to cart with correct details • Navigation between AR view and other screens is smooth • Cart contains product added from AR view • Checkout process works with AR-added products • Order placement completes successfully • Multiple AR sessions work without issues • No memory leaks or performance degradation • AR module integrates seamlessly with purchase flow
System Integration Points Verified	• Product Details module → AR View module • AR View module → Camera/Permission handling • AR View module → 3D Model loading • AR View module → Shopping Cart module • AR View module → Product Catalog module • AR View module → Checkout module • AR View module → Order Management module • Real-time data sync between AR and cart
Runtime Test Results (Xiaomi 10T)	• AR View Initialization: Opened successfully, camera feed displayed correctly • Camera Permission: Requested and granted without issues • AR Tracking: Initialized properly, tracking remained stable throughout • 3D Model Loading: Model downloaded and rendered correctly in AR space • AR Controls: All controls (scale, rotate, reset) worked smoothly • Model Placement: Accurate placement, stable tracking maintained • Add to Cart from AR: Product added successfully with correct details

	• Cart Integration: Product appeared in cart immediately after AR addition • Checkout Process: Completed successfully with AR-added product • Order Placement: Order created successfully with AR-selected product • Multiple AR Sessions: All sessions worked correctly, no performance issues • Memory Management: No memory leaks detected, efficient cleanup observed • Integration Verification: All AR module integrations with other modules worked correctly
--	---

Table 138 AR Product Visualization and Purchase Integration System Test Table

- **Multi-Product AR Comparison and Selection System Test**

Test the AR functionality for comparing multiple products in AR space and making purchase decisions, verifying AR module integrates with product catalog, comparison features, and purchase flow. This test validates advanced AR usage scenarios where users compare multiple products before making purchase decisions.

Test ID	SYST-009
Priority	High
Preconditions	• User is logged in • Device supports AR (ARCore compatible) • Camera permissions granted • Multiple products with 3D models available • Adequate lighting and space for AR • Stable internet connection
Test Steps	• Launch app and navigate to Home screen • Browse products and find first product with AR • View product details • Tap "View in AR" button • Verify AR view opens • Verify first 3D model loads and places • Interact with first model (scale, rotate) • Note product details visible in AR • Close AR view • Find second product with AR • View product details

	• Tap "Compare" or "View in AR" option • If comparison feature exists, add to comparison • Open AR view for second product • Verify second model can be placed alongside first (if supported) • If single model AR, test switching between products • Compare products side-by-side in AR (if supported) • Test AR controls for each product • Evaluate products in AR space • Add preferred product to cart from AR view • Close AR view • Navigate to Cart • Verify selected product is in cart • Navigate back to product catalog • Test AR view for third product • Compare third product with previous selection • Add third product to cart • Navigate to Cart • Verify both AR-selected products are in cart • Proceed to checkout • Complete order with AR-selected products • Verify order contains correct AR-selected products • Test AR view persistence (reopen same product AR) • Verify AR state is maintained or reset appropriately • Test multiple AR sessions in sequence • Verify no performance degradation
Expected Results	• AR view opens for each product correctly • 3D models load and render correctly • AR controls work for each model • Product comparison in AR works (if supported) • Multiple products can be viewed in AR (sequentially or simultaneously) • Products can be added to cart from AR view • Cart contains correct AR-selected products • Checkout process works with AR-selected products • Order contains correct products • AR view state is managed correctly • Multiple AR sessions work without issues • No memory leaks from multiple AR sessions

	• Performance remains acceptable • AR tracking remains stable • User experience is smooth throughout
System Integration Points Verified	• Product Catalog module → AR View module • AR View module → Product Comparison (if exists) • AR View module → Shopping Cart module • AR View module → Multiple product handling • AR View module → Checkout module • AR View module → Order Management module • AR state management → Session persistence • Multiple AR models → Memory management
Runtime Test Results (Xiaomi 10T)	• Multiple AR Views: All AR views opened correctly for different products • 3D Model Loading: All models loaded and rendered correctly in AR space • AR Controls: All controls worked smoothly for each model (scale, rotate, reset) • Product Comparison: Products could be viewed sequentially in AR for comparison • AR to Cart Integration: Products added to cart from AR view worked correctly • Cart Verification: All AR-selected products appeared in cart with correct details • Checkout Process: Checkout completed successfully with multiple AR-selected products • Order Creation: Order created with all AR-selected products correctly • AR State Management: AR view state managed correctly, appropriate reset on reopen • Multiple AR Sessions: All sequential AR sessions worked without issues • Memory Management: No memory leaks detected, efficient cleanup between AR sessions • Performance: Performance remained acceptable throughout multiple AR sessions • AR Tracking: AR tracking remained stable across all sessions • Integration Verification: All AR module integrations with catalog, cart, and checkout worked correctly

Table 139 Multi-Product AR Comparison and Selection System Test Table

5.7 Regression Testing

Regression testing a black box testing technique that consists of re-executing those tests that are impacted by the code changes. These tests should be executed as often as possible throughout the software development life cycle.

All the Regression Tests have been performed and tested by using JEST and the report generated is attached below in this document.

5.7.1 Selecting Regression Tests

This section describes the regression tests selected and implemented for the Shop360 mobile application. Regression tests are designed to verify that existing functionality continues to work correctly after code changes.

Test Selection Criteria

Tests were selected based on the following criteria:

1. Critical business logic

- Core functionality that directly impacts user experience
- Features that handle financial transactions or user data
- Authentication and authorization flows

2. High-Risk areas

- Code that has been recently modified
- Complex logic with multiple code paths
- Integration points between different services

3. User-Facing features

- Features that users interact with frequently
- Features that affect user satisfaction
- Features that could cause data loss if broken

Selected Regression Tests

- **Authentication Service Tests (RT-001 to RT-008)**

Rationale: Authentication is critical for app security and user access. Any regression in authentication could lock users out or create security vulnerabilities.

RT-001: User Sign In - Verify users can log in with valid credentials

RT-002: User Sign Up - Verify new user registration works correctly

RT-003: User Sign Out - Verify users can log out properly

RT-004: Password Reset - Verify password reset functionality

RT-005: Get Current User - Verify current user retrieval

RT-006: Change Email Address - Verify email change with reauthentication

RT-007: Change Password - Verify password change with reauthentication

RT-008: Email Verification - Verify email verification flow

Coverage: 37.36% statements, 14.54% branches, 73.33% functions

Test File: __tests__/services/authService.test.ts

- **Cart Service Tests (RT-009 to RT-015)**

Rationale: Cart functionality is core to the e-commerce experience. Regressions could cause users to lose items, incorrect pricing, or checkout failures.

RT-009: Add Item to Cart (Guest) - Verify guest cart operations

RT-010: Add Item to Cart (Authenticated) - Verify authenticated cart operations

RT-011: Remove Item from Cart - Verify item removal works

RT-012: Set Cart Item Quantity - Verify quantity updates

RT-013: Clear Cart - Verify cart clearing functionality

RT-014: Guest Cart Migration - Verify cart migration on login

RT-015: Cart Subscription - Verify real-time cart updates

Coverage: 51.98% statements, 38.55% branches, 52.63% functions

Test File: __tests__/services/cartService.test.ts

- **Validation Utilities Tests (RT-016)**

Rationale: Input validation prevents invalid data from entering the system and causing errors downstream.

RT-016: Email Validation - Verify email format validation

Coverage: 100% statements, 100% branches, 100% functions

Test File: __tests__/utils/validations.test.ts

- **Product Catalog Service Tests (RT-017 to RT-021)**

Rationale: Product catalog is essential for browsing and discovering products. Regressions could prevent users from finding or viewing products.

RT-017: Get Product by ID - Verify product retrieval

RT-018: Get Products by Category - Verify category filtering

RT-019: Subscribe to Products - Verify real-time product updates

RT-020: Subscribe to Single Product - Verify single product subscriptions

RT-021: Subscribe to Featured Products - Verify featured product filtering

Coverage: 53.12% statements, 75% branches, 55.55% functions

Test File: __tests__/services/productCatalogService.test.ts

- **AR Functionality Tests (RT-022 to RT-028)**

Rationale: AR is a key differentiator for the app. Regressions could prevent users from viewing products in AR, which is a unique selling point.

RT-022: AR Scene Initialization - Verify AR scene setup

RT-023: Model URL Handling - Verify 3D model loading

RT-024: Model Position Handling - Verify model positioni

RT-025: Model Rotation Handling - Verify model rotation

RT-026: Model Scale Handling - Verify model scaling

RT-027: AR Scene Callbacks - Verify AR interaction callb

RT-028: AR Scene Rendering - Verify AR scene rendering

Coverage: 53.12% statements, 75% branches, 55.55% functions

Test File: __tests__/services/productCatalogService.test.ts

Testing Framework:

- Jest 26.6.3
- React Native Test Renderer
- Node.js test environment

Mocking Strategy:

- Firebase services are mocked to prevent actual API calls
- AsyncStorage is mocked for guest cart testing
- ViroReact AR components are mocked for testing without device
- Google Sign-In is mocked to prevent external dependencies

Test Structure:

- Each test follows the pattern:
- Setup: Prepare test data and mocks
- Execution: Call the function being tested
- Assertion: Verify expected behavior
- Cleanup: Reset mocks and state

Test Naming Convention:

- RT-XXX: Regression Test identifier
- Descriptive test names explaining what is being tested
- Grouped by functionality area

Test Execution Results

Total Test Suites: 5

Total Tests: 65

Passed: 65

Failed: 0

Status: **ALL TESTS PASSING**

Test Coverage Summary:

- Overall: 4.44% statements, 3.29% branches, 4.35% functions
- Focused on critical business logic rather than comprehensive coverage
- Higher coverage in tested services (auth: 37%, cart: 52%, products: 53%)

5.7.2 Regression Tests

JEST Regression Test Report is attached below.

Report Generated: January 2025

Test Framework: Jest 26.6.3

Test Environment: Node.js

Project: Shop360° Mobile Application

This section presents the comprehensive regression test execution results for the Shop360 mobile application. All regression tests have been executed successfully, confirming that critical application functionality remains intact after recent code changes.

Test Execution Summary

Test Suites: 5 passed, 5 total

Total Tests: 65 passed, 65 total

Snapshots: 0 total

Execution Time: 1.169 seconds

Overall Status: ALL TESTS PASSING

The regression test suite validates core functionality across five critical areas:

- Authentication Services
- Cart Management Services
- Product Catalog Services
- Validation Utilities
- AR (Augmented Reality) Functionality

Test Execution Results

- Authentication Service Regression Tests

File: __tests__/services/authService.test.ts

Status: PASS

Total Tests: 8 passed

RT-001: User Sign In

✓ should successfully sign in with valid email and password

- ✓ should throw error for invalid credentials
- ✓ should handle empty email

RT-002: User Sign Up

- ✓ should successfully create new user account (1 ms)
- ✓ should assign admin role if email is in roleAssignments
- ✓ should rollback user creation if Firestore write fails (1 ms)

RT-003: User Sign Out

- ✓ should successfully sign out authenticated user
- ✓ should handle sign out when already signed out

RT-004: Password Reset

- ✓ should send password reset email successfully
- ✓ should handle invalid email address (1 ms)

RT-005: Get Current User

- ✓ should return current authenticated user
- ✓ should return null when no user is authenticated

RT-006: Change Email Address

- ✓ should successfully change email with valid current password
- ✓ should throw error for empty new email (3 ms)

RT-007: Change Password

- ✓ should successfully change password with valid current password
- ✓ should throw error for empty new password (1 ms)

RT-008: Email Verification

- ✓ should resend email verification successfully
- ✓ should reload user and return email verification status

Coverage: 37.36% statements, 14.54% branches, 73.33% functions

- **Cart Service Regression Tests**

File: __tests__/services/cartService.test.ts

Status: PASS

Total Tests: 15 passed

RT-009: Add Item to Cart (Guest)

- ✓ should add new item to guest cart successfully (3 ms)
- ✓ should increment quantity if item already exists in cart
- ✓ should throw error when product is out of stock (5 ms)
- ✓ should throw error when quantity exceeds available stock (1 ms)

RT-010: Add Item to Cart (Authenticated User)

- ✓ should add item to Firestore cart for authenticated user
- ✓ should throw error if user email is not verified

RT-011: Remove Item from Cart

- ✓ should remove item from guest cart successfully
- ✓ should remove item from Firestore cart for authenticated user

RT-012: Set Cart Item Quantity

- ✓ should update quantity for existing item in guest cart (1 ms)
- ✓ should remove item when quantity is set to 0
- ✓ should throw error when quantity exceeds stock

RT-013: Clear Cart

- ✓ should clear all items from guest cart
- ✓ should clear all items from Firestore cart for authenticated user

RT-014: Guest Cart Migration

- ✓ should migrate guest cart items to user cart on login (1 ms)
- ✓ should not migrate if user is not authenticated

RT-015: Cart Subscription

- ✓ should subscribe to guest cart changes (14 ms)
- ✓ should subscribe to Firestore cart for authenticated user

Coverage: 51.98% statements, 38.55% branches, 52.63% functions

- **Validation Utilities Regression Tests**

File: __tests__/utils/validations.test.ts

Status: PASS

Total Tests: 3 passed

RT-016: Email Validation

- ✓ should return true for strings containing @ symbol (1 ms)
- ✓ should return false for strings without @ symbol
- ✓ should handle edge cases

Coverage: 100% statements, 100% branches, 100% functions

- **Product Catalog Service Regression Tests**

File: __tests__/services/productCatalogService.test.ts

Status: PASS

Total Tests: 5 passed

RT-017: Get Product by ID

- ✓ should return product when it exists (1 ms)
- ✓ should return null when product does not exist
- ✓ should normalize product data correctly

RT-018: Get Products by Category

- ✓ should return products filtered by category
- ✓ should exclude specified product ID (1 ms)
- ✓ should respect max limit

RT-019: Subscribe to Products

- ✓ should subscribe to product changes
- ✓ should call callback with normalized products (1 ms)

RT-020: Subscribe to Single Product

- ✓ should subscribe to single product changes
- ✓ should call callback with null when product is deleted

RT-021: Subscribe to Featured Products

✓ should filter and return only featured products

Coverage: 53.12% statements, 75% branches, 55.55% functions

- **AR Functionality Regression Tests**

File: __tests__/ar/ModelPlacementARScene.test.tsx

Status: PASS

Total Tests: 7 passed

RT-022: AR Scene Initialization

✓ should render AR scene with default props (25 ms)

✓ should render AR scene components (1 ms)

RT-023: Model URL Handling

✓ should handle valid model URL (1 ms)

✓ should handle empty model URL (1 ms)

✓ should handle undefined model URL

RT-024: Model Position Handling

✓ should use provided model position (1 ms)

✓ should use default position [0, 0, 0] when not provided

RT-025: Model Rotation Handling

✓ should handle model rotation Y axis (1 ms)

✓ should default rotation to 0 when not provided

RT-026: Model Scale Handling

✓ should handle valid scale multiplier (1 ms)

✓ should clamp scale multiplier to valid range (0.1 to 10)

RT-027: AR Scene Callbacks

✓ should handle tracking update callback (1 ms)

✓ should handle plane selection callback

✓ should handle model position change callback

RT-028: AR Scene Rendering

✓ should render ambient light (1 ms)

✓ should render AR scene with proper configuration

Coverage: 27.48% statements, 24.52% branches, 36.84% functions

Pass Rate: 100%

All 65 regression tests executed successfully with zero failures. This indicates that:

1. Authentication flows remain functional
 - User sign-in, sign-up, and sign-out operations work correctly
 - Password management and email verification processes are intact
 - Account modification features (email/password changes) function properly
2. Cart management operations are stable
 - Guest cart functionality (AsyncStorage) works as expected
 - Authenticated user cart (Firestore) operations are correct
 - Cart migration between guest and authenticated states functions properly
 - Real-time cart subscriptions are operational
3. Product catalog services are operational
 - Product retrieval by ID functions correctly
 - Category filtering and product subscriptions work as expected
 - Data normalization handles edge cases properly
4. Input validation is functioning
 - Email validation logic behaves consistently across scenarios
5. AR functionality is properly implemented
 - AR scene initialization and rendering work correctly
 - Model loading, positioning, rotation, and scaling function as expected
 - AR interaction callbacks are properly configured

Performance Metrics

Fastest Test Suite: Validation Utilities (3 tests, <1ms average)

Slowest Test Suite: Cart Service (15 tests, ~1ms average per test)

AR Test Suite: 7 tests with initial render taking 25ms (expected for component initialization)

Overall execution time of 1.169 seconds demonstrates efficient test execution with minimal overhead from mocking and setup operations.

Chapter 6: Tools & Techniques

Programming Languages: TypeScript, JavaScript, Java, Kotlin, Objective-C / Objective-C++, C++ / C, Ruby, JSON, XML.

Applications and Tools:

- React Native (Version 0.72.5)
- ExpoGo (Test)
- Firebase Version (23.7.0)
- Visual Studio Code
- Git
- GitHub
- npm
- Metro (Version 0.82.1)
- ESLint (Version 7.32.0)
- Prettier (Version 2.8.8)
- Jest (Version 26.6.3)
- Android Studio
- Gradle (Version 8.1.1)
- Babel (Version 7.12.9)
- Node.js
- Watchman
- Hermes

Libraries & Extensions

- **Core Dependencies**
 - React (Version 18.2.0)
 - React Native (Version 0.72.5)
 - React Dom (Version 18.2.0)
- **Navigation & Routing**
 - @react-navigation/native (Version 6.1.18)
 - @react-navigation/native-stack (Version 6.11.0)
 - @react-navigation/bottom-tabs (Version 6.6.1)

- **Firestore Services**
 - `@react-native-firebase/app` (Version 23.7.0)
 - `@react-native-firebase/auth` (Version 23.7.0)
 - `@react-native-firebase/firestore` (Version 23.7.0)
 - `@react-native-firebase/storage` (Version 23.7.0)
- **Augmented Reality**
 - `@viro-community/react-viro` (Version 2.41.1)
- **Storage & Data Management**
 - `@react-native-async-storage/async-storage` (Version 2.2.0)
- **UI Components & Icons**
 - `@react-native-vector-icons` (Version 10.3.0)
 - `@react-native-safe-area-context` (Version 4.7.4)
 - `@react-native-screens` (Version 3.27.0)
- **Authentication**
 - `@react-native-google-signin/google-signin` (Version 16.0.0)
- **Location Services**
 - `@react-native-community/geolocation` (Version 3.4.0)
 - `@react-native-geolocation-service` (Version 5.3.1)
- **File Management**
 - `@react-native-document-picker` (Version 9.3.1)
- **Development Dependencies**
 - TypeScript (Version 4.8.3)
 - `@typescript-eslint/parser` (Version 5.37.0)
 - `@typescript-eslint/eslint-plugin` (Version 5.37.0)
 - ESLint (Version 7.32.0)

- @react-native-community/eslint-config (Version 2.0.0)
- Jest (Version 26.6.3)
- @types/jest (Version 26.0.23)
- babel-jest (Version 26.6.3)
- @testing-library/react-native (Version 12.9.0)
- react-test-renderer (Version 18.1.0)
- @types/react (Version 18.0.21)
- @types/react-native (Version 0.70.6)
- @types/react-test-renderer (Version 18.0.0)
- @tsconfig/react-native (Version 2.0.2)

- **Build Tools & Configuration**

- Babel Core (Version 7.12.9)
- @babel/runtime (Version 7.12.5)
- @react-native/metro-config (Version 0.82.1)
- metro-react-native-babel-preset (Version 0.72.3)
- Sharp (Version 0.34.5)

Native Dependencies

- **Android Native Modules**

- **ViroReact Native Modules:** gvr_common, arcore_client, react_viro, viro_renderer
- **Google Play Services:** play-services-location (Version 21.2.0), google-services (Version 4.4.2)
- **ExpoPlayer** (Version 2.19.1)
- **Protobuf** (Version 3.1.0)

Platform-Specific Dependencies

- **Android**

- Minimum SDK: 24 (Android 7.0)
- Target SDK: 34 (Android 14)
- Compile SDK: 34
- Java Version: 17 (for Firebase modules), 11 (default)
- Kotlin Version: 1.9.22
- ARCore: Required for AR functionality
- Build Tools: 34.0.0
- Gradle: 8.1.1

Chapter 7: Summary & Conclusion

7.1 Summary

Shop360 is a revolutionary mobile application that has been conceptualized to fill the gap that existed between online shopping and offline shopping with the use of Augmented Reality (AR). This mobile application was created as a final year project and has the capability of allowing consumers to visualize the product in their environment before purchasing.

The application was developed using React Native and TypeScript. The backend framework is based on Google Firebase, and it provides a high level of efficiency in managing user authentication, real-time product information, and storing multimedia contents. With the help of the `@viro-community/react-viro` library, high-quality 3D model rendering and AR functionality are successfully integrated in Shop360 for Android devices.

7.2 Conclusion

Shop360's development and launch mark an important milestone in immersive e-commerce. Throughout this project, we successfully achieved the main objective of developing a functional, user-centered mobile application that combines advanced AR capabilities with the usual set of e-commerce features. The application overcomes the "tangibility gap," often seen as one of the most significant online shopping drawbacks, and allows users to view furniture, decor, and other products in their space with scale accuracy.

It has been a very technically dense synergy between cross-platform frameworks and native modules. The choice of React Native, combined with TypeScript, proved instrumental in scaling the foundation of this project because its building blocks were developed fast and with maintainable logic. Introducing Firebase as a backend-as-a-service (BaaS) further validated its utility in handling real-time data synchronization. This allowed for user interactions, from safe authentication to instant updates on product catalogs, to take place without managing a custom server infrastructure overhead. Moreover, the process of testing was accelerated using Expo Go on physical Android devices to ensure that the AR features were thoroughly checked in real-world scenarios.

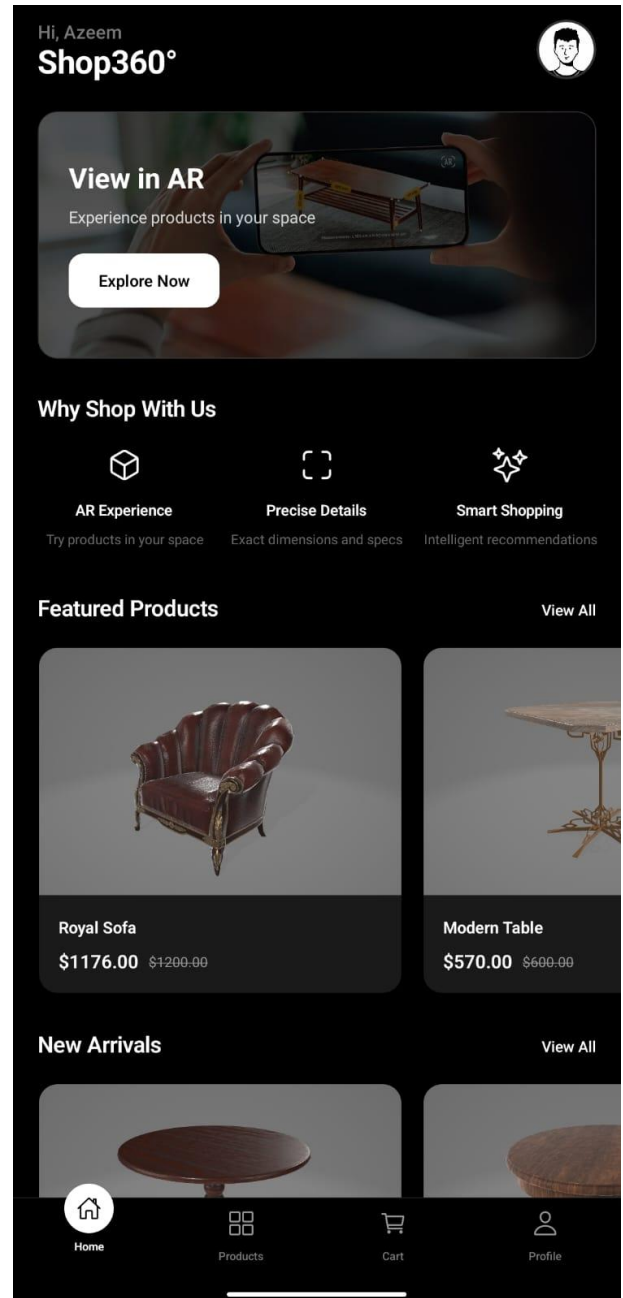
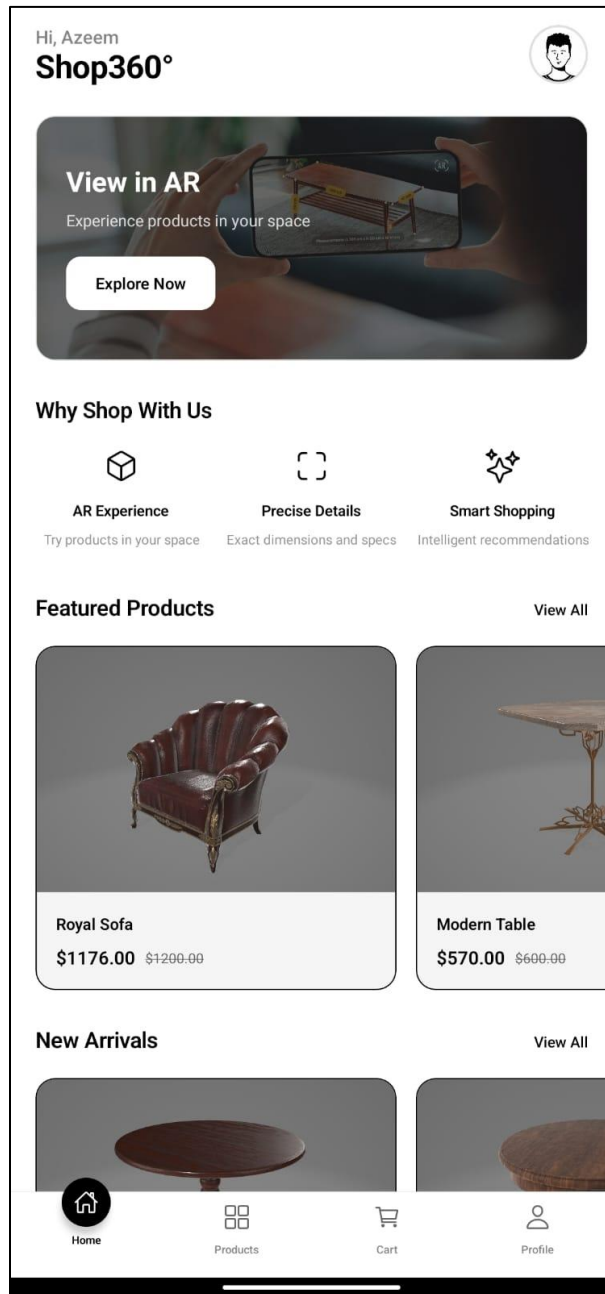
However, it should be noted that this project also works under some constraints. As discussed in the phases of the development of this application, it can only be used on the Android system. This is because the application was limited, in the first place, by the current structure of the used AR libraries and the intention of being initially compatible with ARCore. Although this application for Android provides an optimally seamless user experience, it, in fact, fails to serve a major portion of the iOS user base. Moreover, despite having an easy shopping flow mechanism using libraries such as "react-navigation" or "react-native-vector-icons," some advanced functionalities such as

live payment links were intentionally left in either testing phases or simply skipped for the sake of this application's value provided by AR.

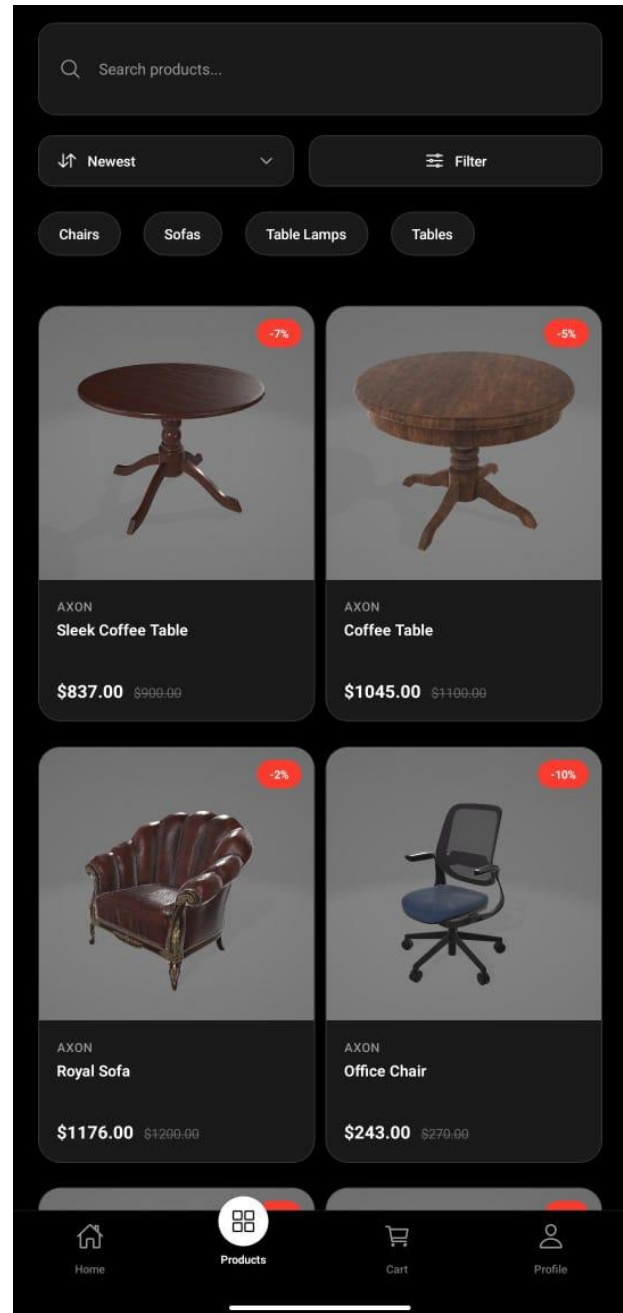
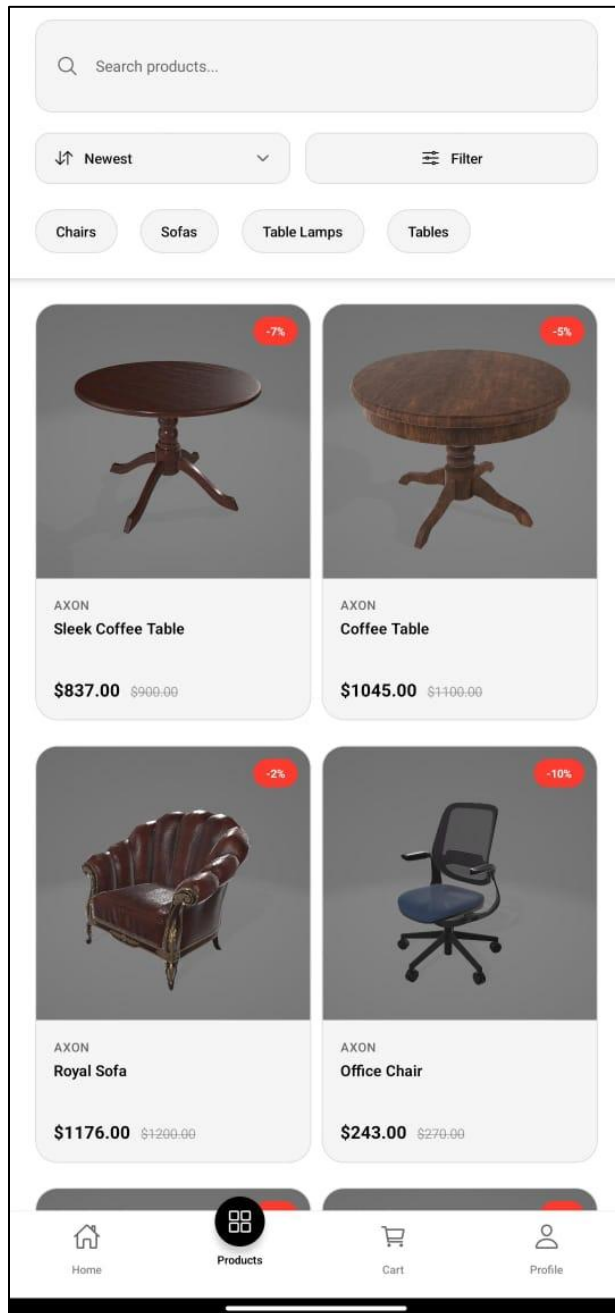
Going ahead, the plan for Shop360 is quite promising. The next step in the plan is to extend the support to the iOS platform. This can be achieved by optimizing ViroReact for the iOS platform or by upgrading the library. Apart from this development plan, the potential that can be leveraged in the coming years is in combining it with Artificial Intelligence. This recommendation system can be powered by AI to recommend products to users on the basis of preferences and activities being carried out by users with the AR models of products. Last but not least, including live tracking of products would be the final step to complete the user experience with Shop360 and would transform it from being an online shopping browser to being an overall e-commerce platform.

Chapter 8: User Manual

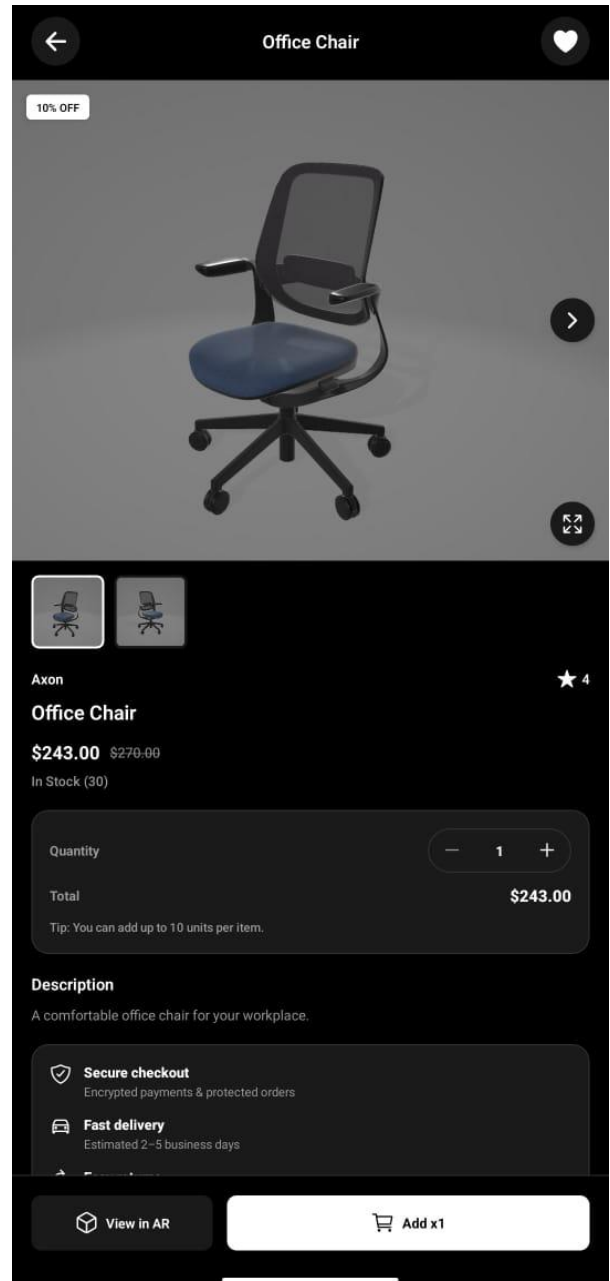
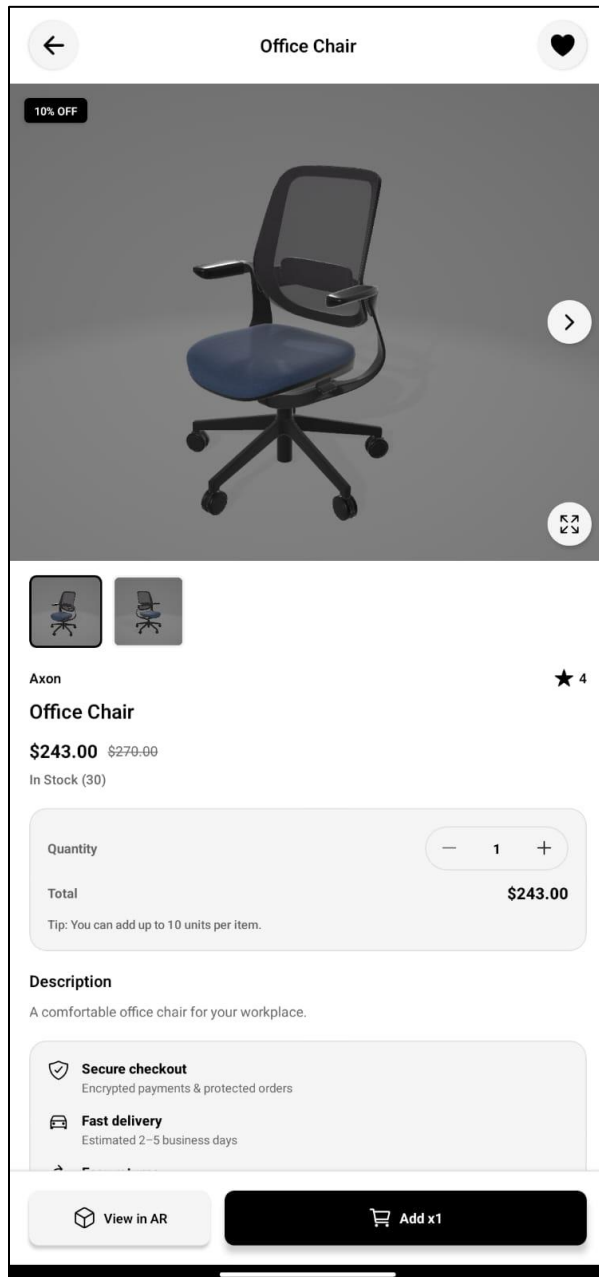
1. Open the app



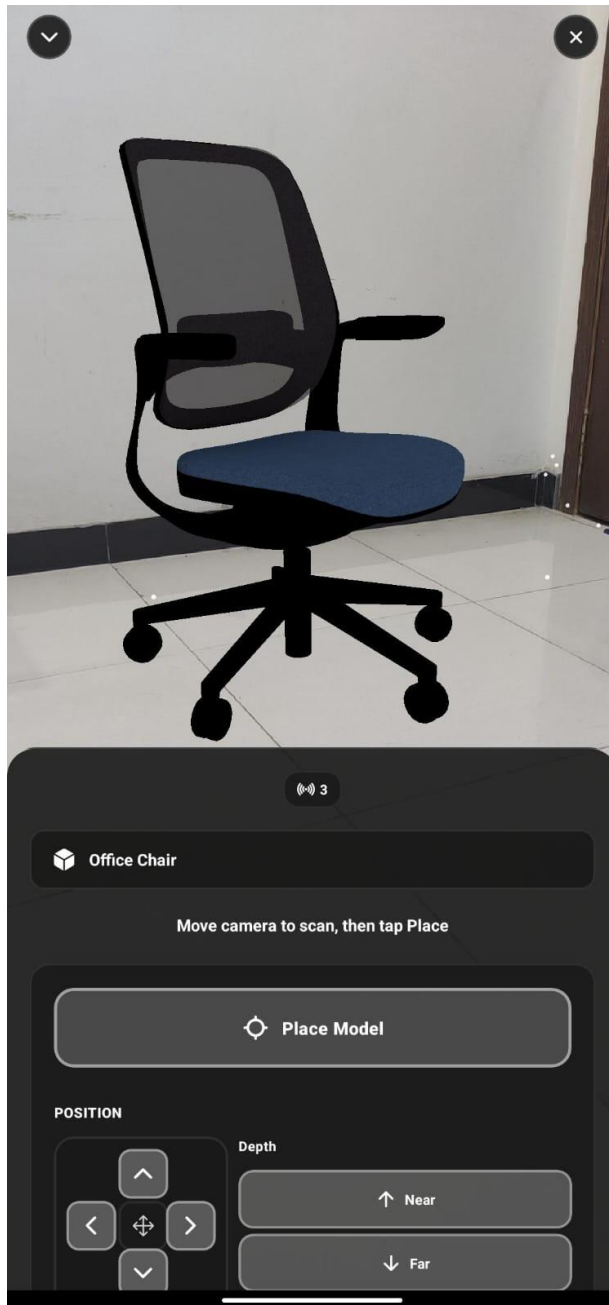
2. Click on Products Tab Button



3. Click on a product to View

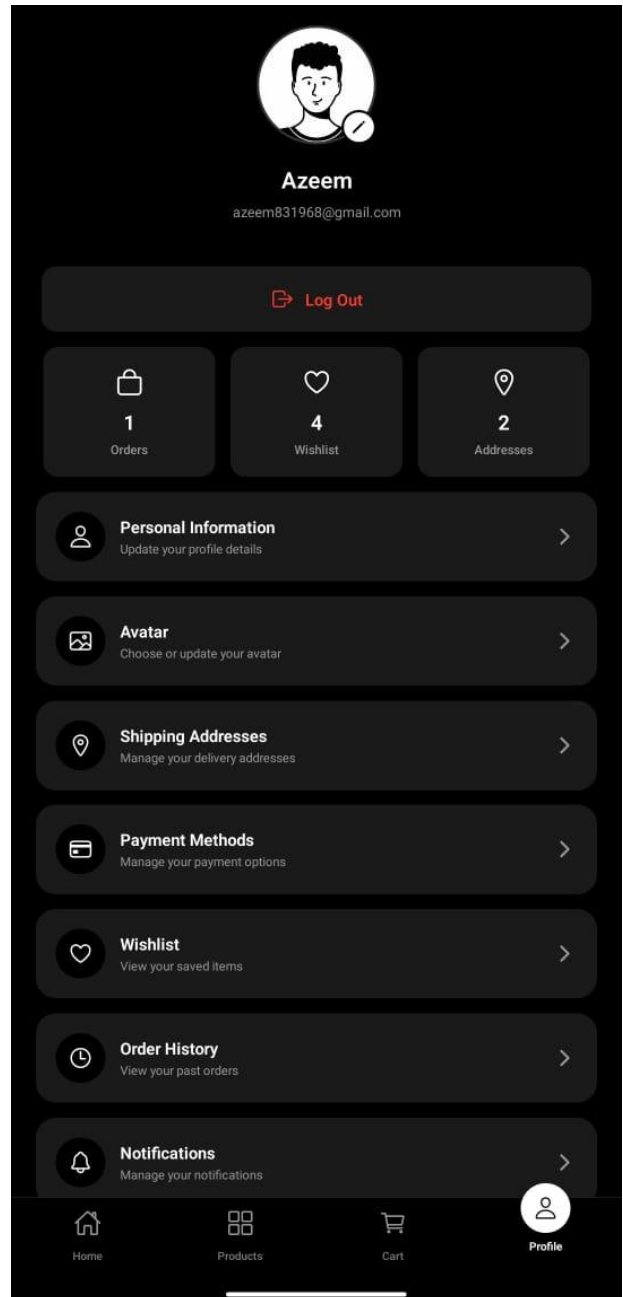
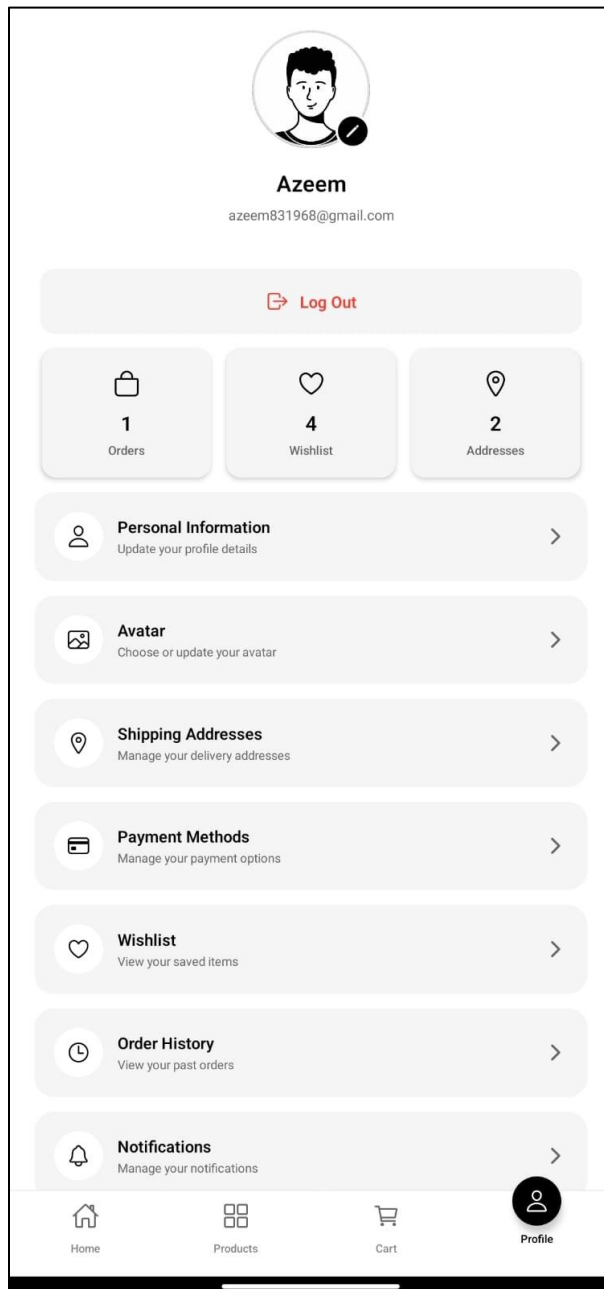


4. Click on “View in AR” button

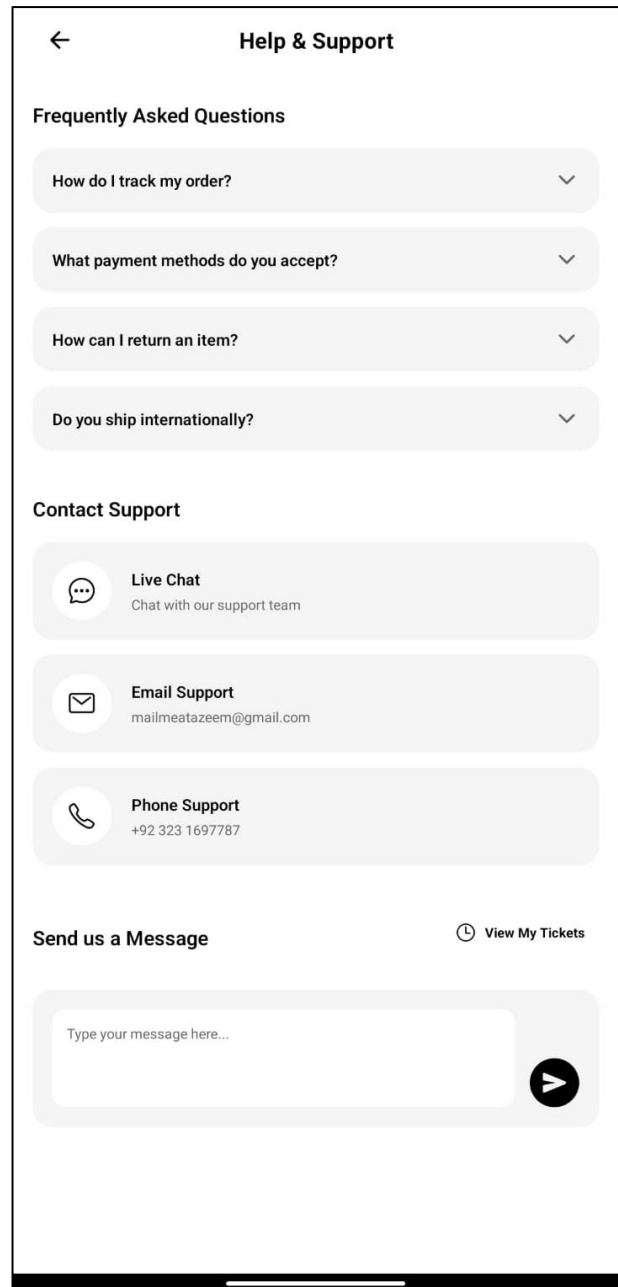
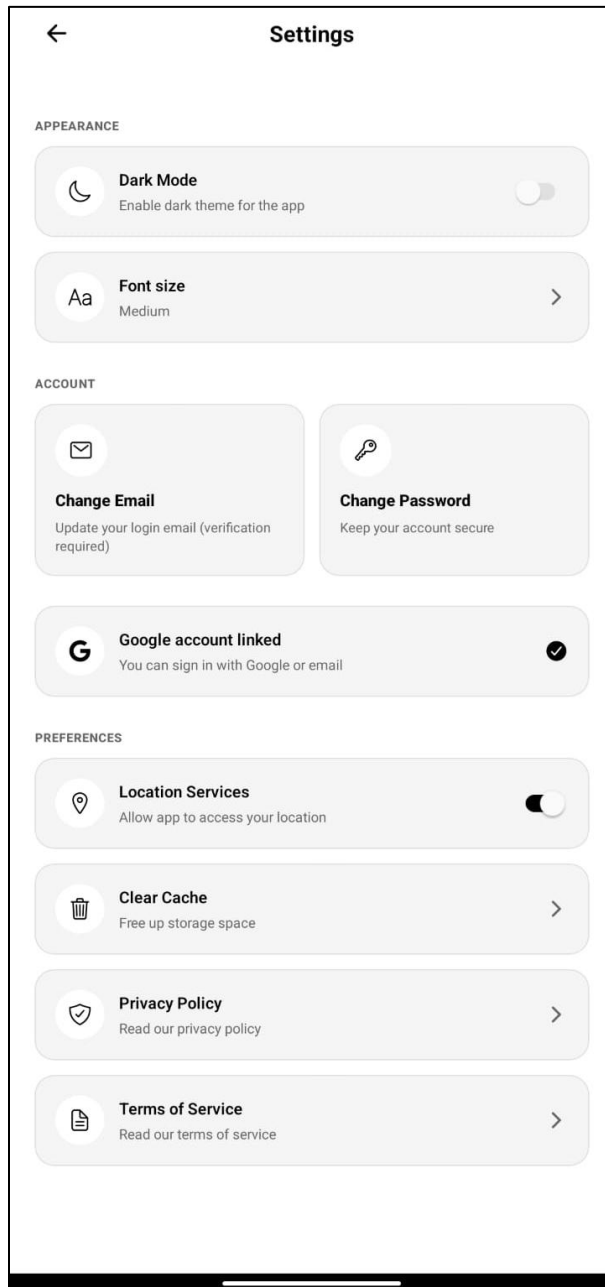


Other Screens

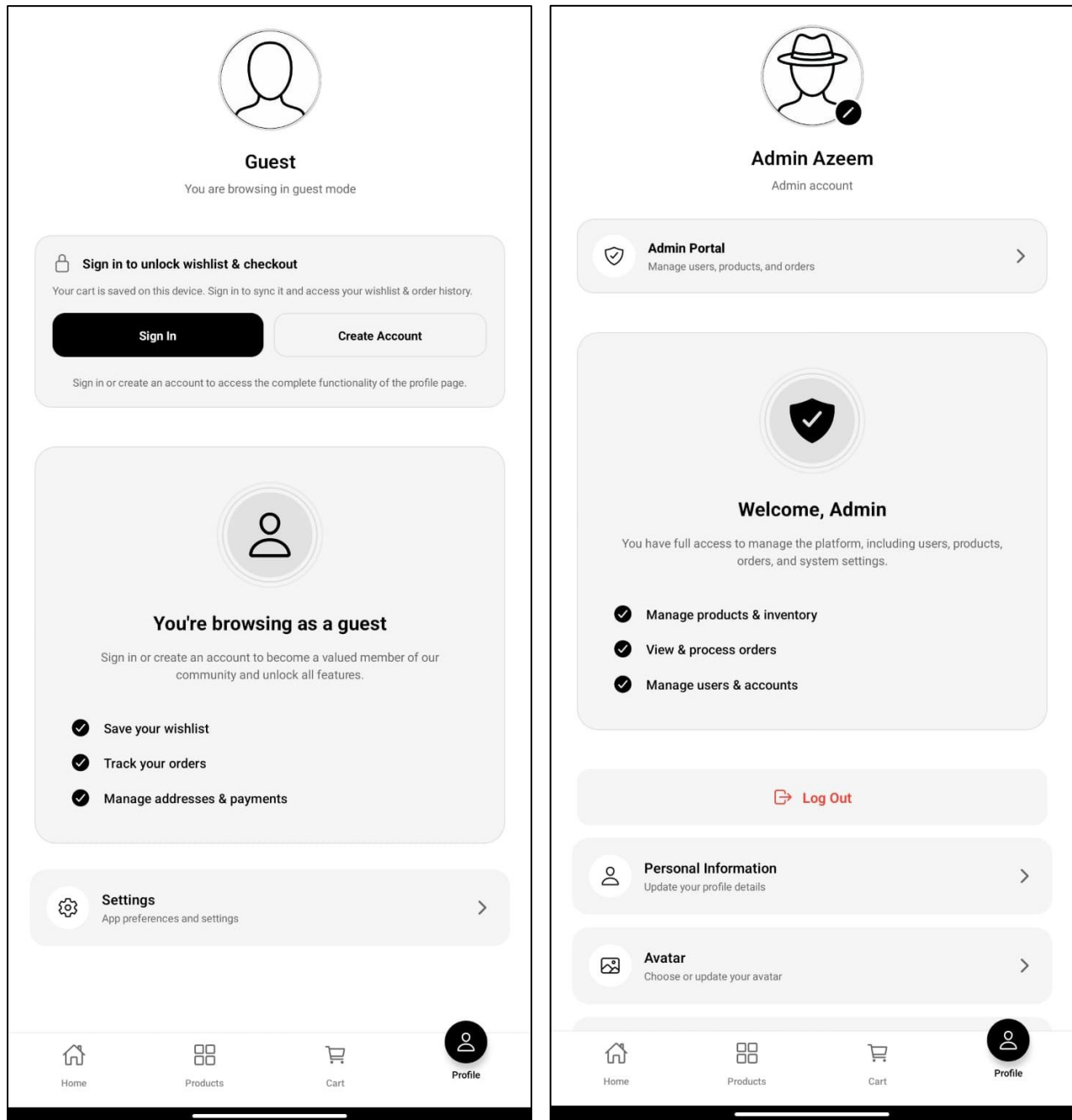
- **Profile Screen:**



Settings and Help & Support



- **Guest and Admin Profile**



- **Wishlist and Card Management**



My Wishlist



Axon

Royal Sofa

\$1200 ~~\$1440~~

Move to Cart

Remove



axon

Wooden Chair

\$99 ~~\$118.8~~

Move to Cart

Remove



Axon

Fancy Table Lamp

\$120 ~~\$144~~

Move to Cart

Remove



Axon

Office Chair

\$270 ~~\$324~~

Move to Cart

Remove



Add Payment Card



.....

CARDHOLDER

YOUR NAME

EXPIRES

MM/YY

Card Number

1234 5678 9012 3456

Cardholder Name

John Doe

Expiry Month

MM

Expiry Year

YY

CVV (3 digits)

123

☐ Set as default payment method

Add Card

Chapter 9: Lessons Learnt and Future Work

9.1 Lessons Learnt

The realization of the project Shop360 taught us several things about the challenges and complexities involved in creating an immersive mobile application. The first realization was the complexity of cross-platform AR development. Although React Native allows for a high degree of flexibility in UI/UX design, the realization of high-performance AR APIs proved to be a challenge, for which we understood the importance of understanding Android native compatibility.

Secondly, the importance of Backend-as-a-Service (BaaS) solutions, such as Firebase, could not be more clear, as the use of Firebase showed how much our development process could benefit from, basically, delegating authentication and database handling to Firebase, which left us with ample resources to work with regarding the actual logic, again the AR part.

Finally, we realized the value of user-centric design in 3D spaces. While designing interfaces for placing digital content over the real world, it's essential to adopt a strategy which is different from the way 2D applications are usually developed. Learning about scaling 3D models appropriately and making sure the UI does not cover the camera's perspective was an essential takeaway in the area of spatial computing.

9.2 Future Work

Although Shop360 successfully delivers on its core objectives, several enhancements have been identified to help transform it into a complete, scalable business solution:

iOS Support with ViroReact: A key next step is expanding platform compatibility by supporting iOS devices. This involves enabling ViroReact with ARKit integration to ensure smooth AR experiences across both Android and iOS.

AI-Driven Product Recommendations: We plan to introduce an AI-powered recommendation system that analyzes user activity—such as browsing behavior, purchase patterns, and time spent viewing specific 3D models. This feature will suggest products based on user interests, increasing personalization and boosting conversion rates.

Enhanced Virtual Try-On: We intend to expand AR capabilities further by incorporating Face Mesh technology. This advancement will enable users to virtually “wear” accessories such as glasses or hats in real time, offering a more immersive and interactive try-on experience beyond simple object placement.

GitHub Repository

<https://github.com/70131514/Shop360.git>

References

- [1] Zakeke. (2024, December 9). *How to use augmented reality in eCommerce?* | Zakeke. Available: <https://www.zakeke.com/pillars/how-to-use-augmented-reality-for-ecommerce/>
- [2] Maryia. (2024, December 31). *How augmented reality is changing e-commerce shopping.* Innwise. Available: <https://innwise.com/blog/augmented-reality-for-ecommerce/>
- [3] Team, B. (2024, December 16). *Augmented Reality in Ecommerce: How Does it Work?* BigCommerce. Available: <https://www.bigcommerce.com/articles/ecommerce/ecommerce-augmented-reality/>
- [4] McCluskey, M. (2022, January 28). *Augmented Reality is the Future of Online Shopping.* TIME. Available: <https://time.com/6138147/augmented-reality-shopping/>
- [5] S. Suljic, “360-Degree Images in eCommerce: A Visual Revolution for Online Shopping - BellaVix,” *Boutique Amazon Marketing Agency*, Jan. 30, 2024. <https://bellavix.com/360-degree-images-ecommerce-visual-revolution-online-shopping/> (accessed Jun. 12, 2025).
- [6] I. Prus, “360 View of a Product: A Complete Guide,” *CGI Furniture*, Feb. 14, 2025. <https://cgifurniture.com/blog/360-view-of-a-product/>
- [7] Y. Lee, Fei, U. Tunku, and A. Rahman, “Developing An E-Commerce Website For Enhancing Customer Sales A REPORT SUBMITTED TO,” 2020. Available: http://eprints.utar.edu.my/3954/1/17ACB05787_FYP.pdf
- [8] Facebook Open Source. (2024). *React Native Documentation.* Available: <https://reactnative.dev/docs/getting-started>
- [9] Viro Media. (2023). *ViroReact: Build AR and VR Apps Using React Native.* Available: <https://viro-community.readme.io/docs>

- [10] Google Developers. (2024). *Introduction to Augmented Reality*. Available: <https://developers.google.com/ar>
- [11] Azuma, R. T. (1997). *A Survey of Augmented Reality*. Presence: Teleoperators and Virtual Environments, 6(4), pp. 355–385. Available: <https://www.cs.unc.edu/~azuma/ARpresence.pdf>
- [12] Billingham, M., Clark, A., & Lee, G. (2015). *A Survey of Augmented Reality*. Foundations and Trends in Human–Computer Interaction, 8(2–3), pp. 73–272. Available: <https://www.nowpublishers.com/article/Details/HCI-049>
- [13] React Native Community. (2024). *Native Modules and Architecture*. Available: <https://reactnative.dev/docs/native-modules-intro>
- [14] Unity Technologies. (2023). *AR vs VR vs MR Explained*. Available: <https://unity.com/topics/augmented-reality>
- [15] Peddie, J. (2017). *Augmented Reality: Where We Will All Live*. Springer International Publishing. Available: <https://link.springer.com/book/10.1007/978-3-319-54502-8>
- [16] Santos, M. E. C., Polvi, J., Taketomi, T., Yamamoto, G., Sandor, C., & Kato, H. (2014). *Toward Standard Usability Questionnaires for AR*. IEEE Computer Graphics and Applications, 35(5), pp. 66–75. Available: <https://ieeexplore.ieee.org/document/7167042>

Appendix

Appendix A: User Roles and Access Controls

<i>Admin</i>	Add/edit/delete products, view all users, manage users, etc.
<i>User</i>	Browse products, use AR, add to wishlist, review, chat, etc.

Appendix B: Technologies and Languages

- **Languages:** JavaScript, TypeScript, JSON.
- **Frameworks / Libraries:** React Native (CLI), ViroReact, React Navigation, React Native Safe Area Context, React Native Screens.
- **APIs:** Firebase REST API, Firebase Authentication API, Cloud Firestore API, Firebase Storage API.
- **Databases:** Cloud Firestore (NoSQL).
- **Authentication:** Firebase Authentication with Email/Password and Google Sign-In, Role-based Access Control (RBAC) with custom user roles stored in Firestore.
- **Build Tools:** Metro Bundler, Babel, Gradle (Android), Node.js.
- **Testing Frameworks:** Jest, React Native Testing Library, React Native Test Renderer.
- **Development Tools:** ESLint, TypeScript Compiler, Android Studio.
- **Native Platforms:** Android (ARCore)

Appendix C: AR Implementation Details

- Framework: ViroReact (@viro-community/react-viro v2.41.1).
- AR Type: Markerless (Surface Tracking / Plane Detection).
- Platforms: Android with ARCore (minimum SDK 24)
- 3D Model Format: Binary glTF (.glb files) for optimized loading and rendering.
- Model Storage: Firebase Storage with public read access for product models.
- Features:
 - Load .glb models from Firebase Storage on-demand.
 - Real-time plane detection and surface tracking.
 - Interactive model placement with visual reticle feedback (green when ready, white when searching).
 - Zoom support for model scaling (0.1x to 10x multiplier).
 - Drag gestures for model positioning in 3D space.
 - Rotation gestures for model orientation (Y-axis rotation).
 - Plane locking mechanism to reduce jitter after initial placement.
 - Continuous hit-test preview for better placement feedback.
 - Model position smoothing to prevent teleporting during drag operations.
 - Tracking state monitoring (normal, limited, unavailable) with user feedback.
 - Camera permission handling for both Android platforms.
 - Network connectivity checks before model loading.

- Error handling for model loading failures and placement errors.
- AR Scene Components: ViroARScene, ViroARSceneNavigator, Viro3DObject, ViroNode, ViroAmbientLight, ViroQuad, ViroMaterials.
- Performance Optimizations: Model metrics calculation, footprint-based scaling, local offset adjustments, optimized material rendering.space.

Appendix D: Project Setup

To set up and run the project from scratch, the following steps were / will be followed:

- The project was initialized using React Native CLI commands, and all required dependencies were installed using npm, including:
 - React Navigation for navigation (Native Stack and Bottom Tabs)
 - Firebase packages (@react-native-firebase/app, auth, firestore, storage)
 - ViroReact for AR functionality
 - Google Sign-In for authentication
 - AsyncStorage for local data persistence
 - React Native Vector Icons for UI icons
 - Geolocation services for address features
 - Document Picker for file uploads
- A Firebase project was created and configured to handle:
 - User authentication (Email/Password and Google Sign-In)
 - Product and category data storage (Cloud Firestore)
 - 3D model and image storage (Firebase Storage)
 - Real-time data synchronization
- Firebase configuration files were added:
 - google-services.json for Android (placed in android/app/)
- Firestore Security Rules were implemented to enforce:
- Firebase Storage Security Rules were implemented to:
- Android setup:
- TypeScript configuration was set up with React Native-specific settings for type checking and IntelliSense support.
- Jest testing framework was configured with React Native preset and custom mocks for Firebase, ViroReact, and other native modules..

Appendix E: Testing and Debugging Tools

Several tools and techniques were used to test the app and find bugs during development:

- Jest: Unit and integration testing framework with React Native preset. Used for testing services, utilities, and AR components with mocked dependencies.
- React Native Testing Library: Component testing library for testing React Native components with user-centric queries and interactions.
- React Native Test Renderer: For rendering React Native components in a test environment without requiring a device or simulator.
- Android Studio Profiler: Used for performance monitoring, memory profiling, CPU usage analysis, and network monitoring during development and testing on Android devices.
- Firebase Console: Used to monitor Firestore operations, real-time subscriptions, authentication events, and storage usage. Also used to test Firestore and Storage security rules.
- Firebase Emulator Suite: Used during development to test Firebase Authentication, Firestore rules, and Storage without needing a live connection to the actual Firebase project. This was particularly useful for testing security rules and data validation.
- React Native Debugger: Used to inspect app state, view logs, track errors or crashes, and debug navigation flows, AR views, and user interaction screens.
- ESLint: Static code analysis tool configured with React Native and TypeScript rules to catch potential bugs, enforce coding standards, and maintain code quality.
- TypeScript Compiler: Used for compile-time type checking, catching type errors before runtime, and providing better IDE support.
- Metro Bundler: JavaScript bundler for React Native with built-in error reporting and hot reloading for faster development iteration.
- ARCore Performance Monitor: Used to monitor AR functionality performance, tracking quality, and frame rates during AR sessions on Android devices.

Appendix F: Deployment Instructions

To run or deploy the app and backend, the following steps were followed:

- The mobile app was developed using React Native CLI, which requires Android Studio and the Android SDK for building native Android applications.
- A release keystore was generated for signing the production APK/AAB (Android App Bundle).
- Release build configuration was set up in android/app/build.gradle with proper signing configs, ProGuard rules (if minification is enabled), and version codes.
- Production APK or AAB was built using: `cd android && ./gradlew assembleRelease` or `./gradlew bundleRelease`.
- The built APK/AAB was uploaded to Google Play Console for distribution.
- Firebase project configuration (google-services.json) was verified for production environment.
- Firebase Project Configuration:

- Production Firebase project was created and configured with proper billing plan (if required for storage/usage limits).
- Firestore Security Rules were deployed using Firebase CLI: `firebase deploy --only firestore:rules`
- Firebase Storage Security Rules were deployed using Firebase CLI: `firebase deploy --only storage`
- Firebase Authentication was configured with authorized domains and OAuth redirect URIs.
- Firebase project settings (API keys, OAuth client IDs) were configured in native Android configuration files
- All environment variables, API keys, and sensitive tokens were configured in Firebase Console and native configuration files.
- No sensitive data was stored in version control or exposed in client-side code.
- Production Firebase project was kept separate from development/staging projects.
- Build Verification:
 - Release builds were tested on physical devices before distribution.
 - AR functionality was verified on ARCore-compatible Android devices.
 - Performance and memory usage were monitored on various device tiers (high-end, mid-range, low-end).
 - Network connectivity and offline behavior were tested.

Appendix G: AR Usage Instructions for Users

The app uses Augmented Reality (AR) features to let users preview products in their real-world environment. Here is how users can use AR features:

- Android: The app works on Android devices that support ARCore (Android + with ARCore-compatible hardware). Most modern Android devices from major manufacturers (Samsung, Google Pixel, Xiaomi, etc.) support ARCore.
- Navigate to a product detail page from the product catalog.
- Tap the "Preview in AR" or "View in AR" button on the product detail screen.
- The app will request camera permission if not already granted (required for AR functionality).
- The camera view opens and automatically scans the environment for flat surfaces (floors, tables, etc.).
- A visual reticle appears on the screen:
 - White reticle indicates the system is searching for a suitable surface.
 - Green reticle indicates a surface has been detected and the model is ready to be placed.
- Once a surface is detected, tap "Place Model" to position the 3D product model on the detected surface.
- The product model appears in real-world scale on the detected surface.
- Move the model: Drag the model with one finger to reposition it on the surface.
- Rotate the model: Use rotation gestures to rotate the model around its Y-axis.
- Zoom the model: Pinch with two fingers to zoom in or out (scale from 0.1x to 10x).

- Reset position: Tap "Reset Position" to return the model to its original placement position.
- Reset plane selection: Tap "Reset Plane" to unlock plane detection and allow selecting a different surface.
- Tracking status indicator shows the current AR tracking state (Normal, Limited, Unavailable).
- Model loading progress is displayed when downloading 3D models from Firebase Storage.
- Error messages are shown if model loading fails or if AR tracking is unavailable.
- UI can be minimized to get a clearer view of the AR scene.
- Use AR in well-lit environments for better surface detection.
- Ensure the device camera lens is clean.
- Move the device slowly to help AR tracking maintain stability.
- Place models on flat, textured surfaces for best results.
- For large products, ensure there is enough physical space around you.
- Network connectivity is required to download 3D models (models are cached after first download).

Appendix H: Security Practices

To protect data and secure the app, the following practices were used:

- Firebase Authentication was used for user login and registration with Email/Password and Google Sign-In providers.
- Email verification is required for most authenticated operations to prevent unauthorized access.
- User roles (admin, user) are stored in Firestore user documents and verified server-side through Firestore Security Rules.
- Role assignments can be pre-configured in Firestore before user signup for admin access.
- Authentication tokens are managed by Firebase SDK and automatically refreshed.
- Comprehensive Firestore Security Rules were implemented to enforce data access controls:
 - Users can only read/write their own user profile, cart, wishlist, orders, addresses, payment methods, and notifications.
 - Email verification is required for all authenticated operations (except public product/category reads).
 - Admin users have elevated permissions to manage products, categories, and view all user data.
 - Products and categories have public read access but admin-only write access.
 - Role assignments collection allows users to read only their own assignment document.
 - Tickets collection allows users to create and read their own tickets, while admins can read and update all tickets.
 - Order cancellation requests can only be made by the order owner, and admins can approve/reject cancellations.
 - Stock updates are restricted to decrements only (to prevent stock manipulation).

- Field validation ensures only allowed fields can be written and data types are enforced.
- Timestamp fields (createdAt, updatedAt) are managed server-side using serverTimestamp().
- Firebase Storage Security Rules were implemented to control file access:
 - Product images and 3D models have public read access (anyone can view/download).
 - Write access (upload, update, delete) is restricted to verified admin users only.
 - All other storage paths are denied by default.
- Sensitive data like API keys and OAuth client IDs are stored in native configuration files (not in JavaScript code).
- No authentication tokens or user credentials are stored in AsyncStorage or exposed in client-side code.
- Payment method data (card numbers) are never stored in full; only last 4 digits, card type, and expiry information are stored.
- User input is validated both client-side and server-side (Firestore rules).
- All API calls to Firebase are made over HTTPS.
- Firebase SDK handles certificate pinning and secure connections automatically.
- Network requests are validated and error handling prevents information leakage.
- Admin role is assigned through Firestore documents and verified in Security Rules.
- Admin operations (product management, user management, order management) are protected by Firestore rules.
- Regular users cannot escalate their privileges or access admin-only data.

Appendix I: Performance Consideration

To make sure the app runs smoothly and doesn't crash on low-end devices, the following performance improvements were made:

- All 3D models used in AR are optimized binary glTF (.glb) files for faster loading and smaller file sizes.
- Models are loaded on-demand from Firebase Storage instead of bundling them in the app, reducing initial app size.
- Model metrics (scale, footprint, local offset) are calculated to ensure appropriate sizing in AR scenes.
- Model loading progress is tracked and displayed to users.
- Failed model loads are cached to prevent repeated failed requests.
- Firestore queries use pagination and filters to limit data load and reduce read operations.
- Real-time subscriptions are used efficiently with proper cleanup to prevent memory leaks.
- Product catalog uses efficient queries with category filters and featured product flags.
- Collection group queries are used sparingly and only for admin operations.
- Data is cached locally using AsyncStorage for offline access (cart, wishlist, user preferences).
- React Native lazy loading and code splitting are used where appropriate.

- Component memoization (React.memo, useMemo, useCallback) is used to prevent unnecessary re-renders.
- List virtualization is used for long product lists (FlatList with proper keyExtractor and getItemLayout).
- Image optimization and caching are handled by React Native's Image component.
- Hermes JavaScript engine is enabled for improved performance on Android.
- AR features are optimized for real-time performance with efficient plane detection and hit-testing.
- Model position smoothing prevents jitter and "teleporting" during drag operations.
- Plane locking mechanism reduces unnecessary plane refinement after initial placement.
- AR tracking state is monitored and user feedback is provided when tracking is limited or unavailable.
- AR scene rendering is optimized with appropriate lighting and material settings.
- Proper cleanup of event listeners, subscriptions, and timers to prevent memory leaks.
- Image and model resources are released when AR scenes are unmounted.
- AsyncStorage operations are batched where possible to reduce I/O overhead.
- Large data structures are avoided, and data is paginated or filtered as needed.
- Network connectivity is checked before making requests to prevent unnecessary failed calls.
- Firebase SDK handles request batching and retry logic automatically.
- Offline support is provided for cart and wishlist using AsyncStorage.
- Error handling and retry mechanisms are implemented for network failures.
- AR features are tested on multiple Android devices to ensure compatibility.
- Performance is monitored on various device tiers (high-end, mid-range, low-end).
- Graceful degradation is implemented for devices with limited AR capabilities.
- Minimum SDK version requirements are enforced (Android for ARCore).

Thank You!